

Committing Authenticated Encryption: Generic Transforms with Hash Functions^{*}

Shan Chen¹  and Vukašin Karadžić² 

¹ Southern University of Science and Technology, Shenzhen, China^{**}

dragoncs16@gmail.com

² Technische Universität Darmstadt, Darmstadt, Germany

vukasin.karadzic@tu-darmstadt.de

Abstract. Recent applications and attacks have highlighted the need for authenticated encryption (AE) schemes to achieve the so-called committing security beyond privacy and authenticity. As a result, several generic solutions have been proposed to transform a non-committing AE scheme to a committing one, for both basic unique-nonce security and advanced misuse-resistant (MR) security. We observe that all existing practical generic transforms are subject to at least one of the following limitations: (i) not committing to the entire encryption context, (ii) involving non-standard primitives, (iii) not being a black-box transform, (iv) providing limited committing security. Furthermore, so far, there has been no generic transform that can directly elevate a basic (non-misuse-resistant) AE scheme to a committing AE scheme that offers MR security. Our work fills these gaps by developing black-box generic transforms that crucially rely on hash functions, which are well standardized and widely deployed.

First, we construct three basic transforms that combine AE with a single hash function, which we call **HtAE**, **AEaH** and **EtH**. They all guarantee strong security, and **EtH** can be applied to both AE and basic privacy-only encryption schemes. Next, for MR security, we propose two advanced hash-based transforms that we call **AEtH** and **chaSIV**. **AEtH** is an MRAE-preserving transform that adds committing security to an MR-secure AE scheme. **chaSIV** is the *first* generic transform that can directly elevate basic AE to one with both committing and MR security; moreover, **chaSIV** also works with arbitrary privacy-only encryption schemes. Both of them feature a simple design and ensure strong security.

For performance evaluation, we compare our transforms to similar existing ones, both in theory and through practical implementations using OpenSSL and wolfSSL libraries. The results show that our **AEaH** generally achieves the highest practical efficiency among basic transforms, while **AEtH** excels in MRAE-preserving transforms. Our MRAE-lifting transform **chaSIV** demonstrates comparable performance to MRAE-preserving ones and surpasses them for messages larger than approximately 360 bytes; for longer messages, surprisingly, it even outperforms the benchmark, non-committing standardized AES-GCM-SIV.

Keywords: Authenticated Encryption · Committing Security · Misuse Resistance · Generic Transform · Hash Function · Random Oracle Model

^{*} © IACR 2025. This is the full version of the EUROCRYPT 2025 article published by Springer-Verlag.

^{**} Shan Chen is affiliated with both the Research Institute of Trustworthy Autonomous Systems and the Department of Computer Science and Engineering of SUSTech.

Table of Contents

1	Introduction	3
1.1	Generic Transforms in Previous Works	3
1.2	Motivation	5
1.3	Our Contributions	6
2	Notation and Definitions	8
2.1	Pseudorandom Functions and Hash Functions	8
2.2	Symmetric Encryption	9
3	Basic Transforms	11
3.1	Hash-then-AE (HtAE)	11
3.2	AE-and-Hash (AEaH)	12
3.3	Encrypt-then-Hash (EtH)	13
3.4	Security Comparison	15
4	Advanced Transforms	15
4.1	MRAE-Preserving Transform AEtH	16
4.2	MRAE-Lifting Transform chaSIV	17
4.3	Security Comparison	18
5	Performance Evaluation	19
6	Conclusion	22
A	Performance Testing Details	25
B	Security Proofs	25
B.1	Proof of Theorem 1 (IND\$-CPA Security of HtAE)	25
B.2	Proof of Theorem 2 (INT-CTXT Security of HtAE)	26
B.3	Proof of Theorem 3 (CMT Security of HtAE)	27
B.4	Proof of Theorem 4 (CDY\$ Security of HtAE)	27
B.5	Proof of Theorem 5 (IND\$-CPA Security of AEaH)	28
B.6	Proof of Theorem 6 (INT-CTXT Security of AEaH)	29
B.7	Proof of Theorem 7 (CMT Security of AEaH)	31
B.8	Proof of Theorem 8 (CDY\$ Security of AEaH)	31
B.9	Proof of Theorem 9 (IND\$-CPA Security of EtH)	32
B.10	Proof of Theorem 10 (INT-CTXT Security of EtH)	34
B.11	Proof of Theorem 11 (CMT Security of EtH)	35
B.12	Proof of Theorem 12 (CDY\$ Security of EtH)	36
B.13	Proof of Theorem 13 (MR-IND\$-CPA Security of AEtH)	37
B.14	Proof of Theorem 14 (MR-INT-CTXT Security of AEtH)	39
B.15	Proof of Theorem 15 (CMT Security of AEtH)	40
B.16	Proof of Theorem 16 (CDY\$ Security of AEtH)	40
B.17	Proof of Theorem 17 (MR-IND\$-CPA Security of chaSIV)	42
B.18	Proof of Theorem 18 (MR-INT-CTXT Security of chaSIV)	44
B.19	Proof of Theorem 19 (CMT Security of chaSIV)	46
B.20	Proof of Theorem 20 (CDY\$ Security of chaSIV)	46

1 Introduction

Symmetric encryption is widely used in practice to ensure the privacy and authenticity of data. Such encryption schemes are often referred to as *authenticated encryption (AE)* [BN00], originally designed to address practical shortcomings of basic encryption schemes that offer only privacy. Their syntax also evolves from probabilistic [BDJR97] to nonce-based with associated data [Rog02], with nonces to avoid reliance on good randomness for their security and associated data to capture unencrypted but authenticated data.

In the past few years, a series of applications and attacks [ADG+22; DGRW18; GLR17; LGR21] motivated another level of security for symmetric encryption. Such so-called *committing security*, named and formalized in [BH22] and [CR22], requires the ciphertext to be a *commitment* to the encryption inputs, e.g., to the key K , to the message M , or even to all inputs including the nonce N and associated data AD . For example, the latter most secure case requires that no efficient adversary can find two distinct tuples $(K_1, N_1, AD_1, M_1) \neq (K_2, N_2, AD_2, M_2)$ such that they are encrypted to the same ciphertext. Authenticated encryption schemes that further satisfy committing security are often called *committing authenticated encryption*.

It has been shown in recent works [ADG+22; CR22; DGRW18; GLR17; LGR21; MLGR23] that almost none of the existing AE schemes achieves committing security, including the most popular ones, e.g., AES-GCM [Dwo07], ChaCha20/Poly1305 [Ber05; Ber08], CCM [WHF03], OCB [RBB03], AES-GCM-SIV [GLL19], etc. In order to construct committing AE schemes, one approach is to directly add committing security to existing AE schemes through dedicated modifications. For example, Bellare and Hoang [BH22] proposed specific adjustments to AES-GCM and AES-GCM-SIV, resulting in schemes that achieve both AE security (privacy and authenticity) and committing security. However, such dedicated schemes require a deep understanding of the modified constructions, which could make them difficult for practitioners, especially non-experts, to implement correctly, as committing AE schemes are not yet standardized.

The other approach is to build committing AE from *generic transforms*. Although some of the above dedicated schemes may perform better in practice, the generically transformed schemes enjoy good modularity and flexibility. Such schemes are usually simpler, and they allow the component primitives to be instantiated with any constructions that satisfy the required syntax and security.

In this work, we ask how easy one can generically transform authenticated encryption (and even basic privacy-only encryption) to committing authenticated encryption, using standardized primitives. Before delving into our constructions, we first discuss existing generic transforms proposed in previous works and motivate our work on generic transforms and the use of hash functions.

1.1 Generic Transforms in Previous Works

We summarize the existing generic transforms in Table 1. The study of committing security for authenticated encryption schemes is initialized by Farshim *et al.* [FOR17]. They proposed a generic transform EtM built from a pseudorandom generator (PRG), an AE scheme and a message authentication code (MAC). Their construction guarantees ciphertexts committing to the key by relying on non-standard security from PRG and MAC, e.g., PRG being right collision-resistant. Soon after, Grubbs *et al.* [GLR17] proposed two generic commit-then-encrypt constructions CtE1 and CtE2 that combine a commitment scheme with AE. Such schemes ensure committing security for the message and header (associated data), in order to meet the security goals of *message franking*, a mechanism that verifies abuse reports. Their CtE1 transform relies on standard commitments, while CtE2 requires a non-standard property (i.e., unique commitments) for its commitment component. Then, a follow-up work [DGRW18] on message franking introduced a more efficient transform based on a new primitive called *encryptment*. However, both [GLR17] and [DGRW18] lack committing security for the encryption key and nonce. Later, Albertini *et al.* [ADG+22] explored vulnerabilities in real-world systems resulting from lack of key commitment and proposed a simple generic solution called CommitKeyII. This construction makes use of a collision-resistant pseudorandom function (PRF), a non-standard primitive, and provides committing security for both the key and the nonce, but not for the message or the associated data.

To better mitigate the potential risks arising from insufficient committing security and to ease the burden on practitioners in understanding the nuances of different committing targets, recent efforts focus on building schemes with strong *context-committing* security, introduced in [MLGR23] as a synonym for the strong committing security defined in [BH22; CR22]. In this notion, the resulting ciphertext must commit to the entire encryption *context*: key, nonce, associated data, and message.

Construction	Committing Target	Transform Method	Non-Standard Primitive	Black-Box Transform
EtM [FOR17]	K	PRG-then-Encrypt-then-MAC	PRG, MAC	✓
CtE1/CtE2 [GLR17]	AD, M	Commit-then-AE	Commit for CtE2	✓
CE [DGRW18]	AD, M	Encryptment-then-AE	Encryptment	✓
CommitKeyII [ADG+22]	K, N	PRF-then-AE	PRF	✓
N1 [SW24]	K, N, AD, M	Encrypt-and-MAC (N1 [NRS14])	Encrypt, MAC	✓
EaM/SIV [BCC+24]	K, N, AD, M	(N1, A2)/A4 [NRS14]	Encrypt, MAC	✓
KEtM [BCC+24]	K, N, AD, M	KDF-then-(N2, A6) [NRS14]	KDF, MAC	✓
HtE $\circ$ UtC [BH22]	K, N, AD, M	Hash-then-PRF-then-AE	PRF	✓
HtE $\circ$ RtC [BH22]	K, N, AD, M	Hash-then-PRF-then-AE-then-Hash	PRF	✓
CTX [CR22]	K, N, AD, M	AE-then-Hash	-	✗
CTY [BH24]	K, N, AD, M	AE-then-Hash	-	✗
libsodium [lAut]	K, N, AD, M	AE-then-Hash	-	✗
PACT/comPACT [BBD24]	K, N, AD, M	AE-and-Hash-then- Encipher	AE	✓
HtAE (this work)	K, N, AD, M	Hash-then-AE	-	✓
AEaH (this work)	K, N, AD, M	AE-and-Hash	-	✓
EtH (this work)	K, N, AD, M	Encrypt-then-Hash	-	✓
AEtH (this work)	K, N, AD, M	AE-then-Hash	-	✓
chaSIV (this work)	K, N, AD, M	PRF-and-Hash-then-Encrypt	-	✓

Table 1: Comparison of practical generic transforms to construct committing AE. The “Non-Standard Primitive” column lists the component primitives that are tailored to ensure some form of committing security but are not yet standardized, whereas “-” indicates that all component primitives are already standardized. The **red-colored** component primitive imposes a hard limit on the achievable committing security of the associated transforms.

Bellare and Hoang [BH22] proposed the first generic transform that provides the above strong context-committing security. Their transform, which we denote by $\text{HtE} \circ \text{UtC}$, consists of two component transforms: first, UtC adds key commitment to an AE scheme, and then HtE elevates the key-committing AE to a context-committing one. The authors also proposed another transform $\text{HtE} \circ \text{RtC}$ to add context-committing security to a misuse-resistant AE scheme. The security of both UtC and RtC relies on that of a non-standard primitive that they introduced and named committing PRFs.³

In a work contemporary to [BH22], Chan and Rogaway [CR22] proposed a generic transform called CTX that also achieves context-committing security. CTX is very simple and efficient: it just adds a hash computation to an AE scheme. A recent work by Bellare and Hoang [BH24] improved CTX to an even more efficient transform called CTY, in which the associated data input to the underlying AE is omitted. However, both CTX and CTY require the underlying AE to produce a separate authentication tag, rendering them *not* black-box transforms. As a result, they cannot accommodate AE schemes that use the authentication tag for decryption or do not produce a separate authentication tag, including those based on design paradigms such as synthetic IV (SIV), MAC-then-Encrypt (MtE) and Encode-then-Encipher (EtE). The latter has become more important lately, as NIST plans to develop a new block cipher mode of operation called *accordion mode* [CDD+24], with one of its main applications being the construction of authenticated encryption schemes using the EtE paradigm.⁴ The Sodium library (libsodium) [lAut] used a transform similar to CTX and later adopted the CTY idea to drop the associated data from the AE scheme, differing from CTX/CTY only in that it keeps the authentication

³ We note that, in practice, committing PRFs may be instantiated with a standardized hash function. However, the corresponding PRF security notion is still non-standard. A similar caveat applies to the CtE2 construction: while its commitment component may also be realized via a hash function, its security relies on a non-standard requirement for commitment schemes (i.e., unique commitments).

⁴ EtE-based AE schemes are often designed to achieve the so-called *robust* AE security [HKR15], which is stronger than the classical AE security considered in this work. Recent works by Hoang and Menda [HM24], as well as Naito *et al.* [NST25], investigated dedicated EtE-based constructions that provide both robust AE and committing security.

tag in the ciphertext. By design, however, this transform still requires a separate authentication tag and is therefore non-black-box.

The above CTX/CTY limitation was also observed by a very recent work by Bhattacharjee *et al.* [BBD24]. To support arbitrary AE schemes, they proposed two generic transforms, PACT and comPACT, with PACT further preserving the advanced security of the underlying AE in the presence of nonce misuse. At the end of these transforms, instead of hashing the AE authentication tag as in CTX/CTY, they encrypt the authentication tag (or one block of the AE ciphertext) with a block cipher. As a result, PACT and comPACT incur no extra ciphertext expansion, i.e., the resulting ciphertext is of the same length as the AE ciphertext; their committing security, however, is also limited to at most half the block size (e.g., up to 64 bits when AES is used) due to birthday attacks, and it cannot be expanded. Furthermore, both PACT and comPACT require a non-standard ciphertext collision security from the underlying AE component, which—as recently shown in [BRSA26]—fails to hold for the standardized GCM [Dwo07] AE scheme.

There are two other recent works [BCC+24; SW24] that explored the possibility to develop generic transforms by generically composing an encryption scheme and a MAC. As shown in Table 1, their resulting transforms all involve primitives that have not been standardized. Naito *et al.* [NSS24] also proposed a generic transform, which exploits message redundancy to achieve beyond-birthday-bound committing security. Though theoretically intriguing, their transform is not well-suited for practical use, as the sufficient message redundancy required to achieve good committing security is typically unavailable. Also, their transform demands specific adjustments to the underlying encryption scheme based on the exact message redundancy, which is unrealistic or at least cumbersome in practice.

1.2 Motivation

Similar to classical AE generic composition (e.g., Encrypt-then-Mac) [BN00], black-box generic transforms for committing AE are important to investigate in both theory and practice. They provide modularity, flexibility, broader applicability, and robustness in cryptographic protocols. However, so far, no existing generic transform for committing AE serves as a good counterpart to Encrypt-then-MAC. From our discussion on previous works, all existing practical generic transforms suffer from at least one of the following limitations: (i) not context-committing, (ii) involving primitives not yet standardized, (iii) not being a black-box transform, (iv) providing limited committing security. Our goal is to develop practical black-box context-committing generic transforms using standard primitives, such that their committing security can be expanded as needed.

Furthermore, we observe that, all practical instantiations of the existing generic transforms that ensure context-committing security were proved secure under *idealized* assumptions.⁵ For example, security proofs of CTX/CTY instantiations were performed in the random oracle model, and the security of the practical committing PRF instantiation proposed by [BH22] (that underlies UtC and RtC transforms) was proved in the ideal cipher model.

Since idealized assumptions are currently inevitable for practical constructions and hash functions are well standardized and widely deployed, it is both natural and appealing to develop generic transforms that combine AE with hash functions and prove their security in the random oracle model. Additionally, given that hash-based commitment schemes are both straightforward and efficient, hash functions serve as attractive candidates for constructing simple and practical schemes with committing security. These considerations motivate our focus on using hash functions to develop generic transforms, which aligns with the design choices of existing transforms such as CTX/CTY and PACT/comPACT.

We emphasize that our work is not aimed at developing a committing AE scheme with the best practical performance, as generic transforms typically fall short in that regard, and dedicated schemes like those proposed in [BH22] are better suited for that purpose. Nevertheless, we strive to make our generic transforms as straightforward and efficient as possible, ensuring they are easy for practitioners to understand and implement, thereby facilitating the adoption and deployment of committing AE. The simplicity of our design is particularly useful, given that committing AE schemes are currently not yet standardized and practitioners may need to develop their own implementations.⁶ As a result, our work

⁵ Note that it is possible to prove instantiations of generic transforms secure in the standard model, e.g., HtE $\circ$ UtC with its underlying committing PRF instantiated with hardcore predicates [BH22; FOR17], but such constructions are only theoretical.

⁶ We remark that standardization can be a lengthy process. For example, NIST’s lightweight cryptography, started in 2015, selected Ascon in 2023, published a draft standard in 2024 [SMC+24], and will require more time for widespread adoption.

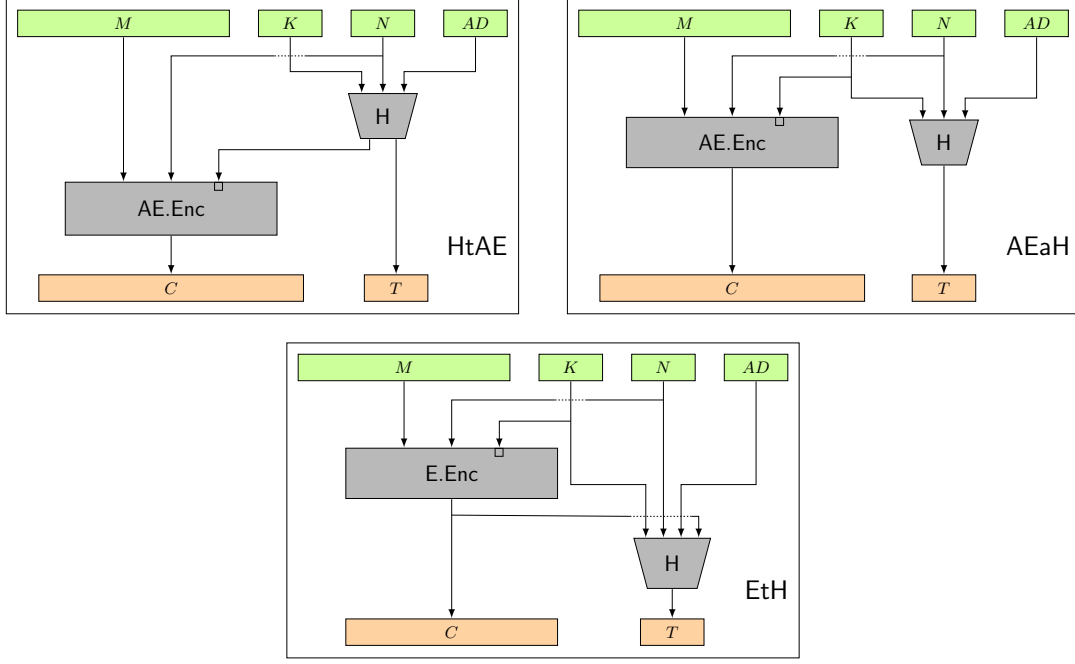


Fig. 1: Our three basic transforms HtAE, AEaH and EtH.

focuses on exploring various simple paradigms for constructing generic transforms using hash functions, rather than minimizing the ciphertext expansion (i.e., ciphertext length minus plaintext length) as was done in [BH24; CR22]. Even so, as we will show, some of our transforms do exhibit compact ciphertext expansion.

Finally, to the best of our knowledge, there is currently no generic transform that can directly elevate an AE scheme to a committing one with advanced misuse-resistant security. Such a transform can be highly useful in practice when misuse-resistant AE schemes are either unavailable (e.g., in Mozilla NSS or libsodium) or inefficiently implemented (e.g., in wolfSSL) within the chosen cryptographic library. Our work also aims to fill this gap.

1.3 Our Contributions

In this work, we propose five generic transforms based on hash functions. Among them, three yield committing authenticated encryption schemes with unique-nonce AE security, while the other two achieve advanced (nonce-)misuse-resistant (MR) AE security. Following the naming convention in [BNT19], we refer to the first three as *basic* transforms and the latter two as *advanced* transforms.

Basic transforms. As motivated above, we first explore black-box generic transforms that combine AE with a single hash function. To this end, similar to the classical generic composition approaches presented in the design of authenticated encryption schemes [BN00], we also consider three transform methods: Hash-then-AE (HtAE), AE-and-Hash (AEaH) and Encrypt-then-Hash (EtH). For each transform method, we propose a hash-based transform, and we naturally name them as HtAE, AEaH, EtH, respectively. Their encryption processes are illustrated in Figure 1.

As shown in the figure, all basic transforms follow a very straightforward and clean design. We note that the first two transforms are built from an AE scheme, while the last EtH transform can directly elevate a basic privacy-only encryption scheme to a committing AE scheme. From the figure, one can also observe that the encryption and hash computations in our second transform AEaH are independent by design and hence fully parallelizable, while the other two transforms have to run their two component primitives sequentially.

We prove that all basic transforms achieve both AE security and strong committing security. For committing security, in addition to the primary context-committing (CMT) notion, we also consider the context-discovery (CDY) notion proposed in [MLGR23]. CMT and CDY are very relevant notions; they can be viewed as analogous to collision-resistance and preimage-resistance for hash functions. As shown in [MLGR23], CDY can be implied by CMT when the encryption algorithm is “context-compressing”, i.e.,

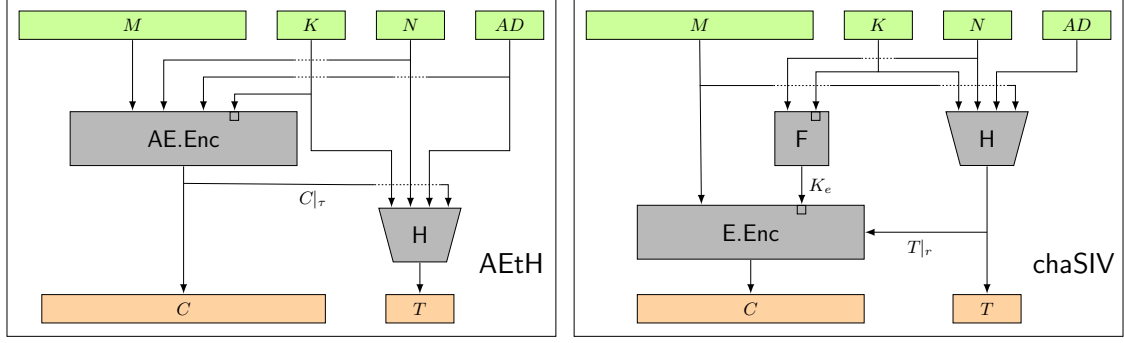


Fig. 2: Our two advanced transforms AEth and chaSIV. $X|_m$ denotes last m bits of a bitstring X . τ denotes the ciphertext expansion of the AE scheme AE and r denotes the nonce size of the encryption scheme E.

ciphertexts are decryptable under multiple contexts. However, using generic transforms to construct AE with strong committing security usually results in larger ciphertext expansion, which may lead to “non-context-compressing” encryption algorithms. Therefore, we also include CDY for our security analysis. CDY security was also analyzed in [BCC+24] for their transforms built from generic compositions. However, to the best of our knowledge, so far no AE-based transforms have been analyzed for CDY security.

For performance, we compare our two AE-based transforms with other similar existing transforms, and show that HtAE and AEaH are both more efficient than their comparable counterparts. Our comparison also implies that AEaH achieves the highest efficiency. To understand the practical performance of our basic transforms and some relevant existing ones, we implement these transforms using OpenSSL and wolfSSL libraries,⁷ and measure their overhead relative to the widely used (non-committing) AE scheme, AES-GCM [Dwo07]. Our results show that, for all tested message lengths (16 to 2048 bytes), the total cost of the most efficient transform, AEaH, is at most three times that of AES-GCM.

Advanced transforms. Next, to achieve MR security, we propose two advanced hash-based transforms.

Our first transform is designed to add committing security to AE schemes that already provide MR security. Such a so-called MRAE-preserving transform follows the AE-then-Hash (AEth) style, and hence we name it AEth. Its encryption process is illustrated in Figure 2 on the left. As shown in the figure, instead of hashing the entire AE ciphertext, AEth hashes only the last block (or any block) of it to improve efficiency while preserving security.

Next, we construct the *first* generic transform that can directly “lift” a basic (unique-nonce) AE scheme to one with both committing and MR security. Our design of such a MRAE-lifting transform is very simple. It can be viewed as a composition of HtAE and the SIV mode [RS06]: just replace the MAC in SIV with a hash function and add a PRF to re-key the underlying encryption scheme per distinct nonce. We therefore name this transform chaSIV, short for *committing hash-based SIV*, with its encryption process illustrated in Figure 2 on the right. As shown in the figure, chaSIV can be built from a basic privacy-only encryption scheme, so like EtH it can be applied to both AE and privacy-only encryption.

We prove that both of our advanced transforms achieve the desired strong committing misuse-resistant security. Then, we compare their concrete security when instantiated with consistent standardized primitives. The results show that our chaSIV, despite its simple design and MRAE-lifting power, achieves security comparable to that of our MRAE-preserving transform AEth.

Finally, we compare the performance of our advanced transforms with other similar existing MRAE-preserving transforms. Both the theoretical and implementation results show that CTX [CR22] and our AEth (essentially a black-box generalization of CTX) exhibit the highest efficiency. In OpenSSL, our MRAE-lifting transform chaSIV demonstrates similar performance to CTX and AEth and surpasses them for messages longer than about 360 bytes; surprisingly, for messages longer than 1024 bytes, our committing AE chaSIV even outperforms the benchmark, non-committing standardized misuse-resistant AE AES-GCM-SIV [GLL19]. Then, in wolfSSL, chaSIV outperforms all other transforms across all message sizes, including the non-committing benchmark AES-SIV [Har08], the only misuse-resistant AE available

⁷ While OpenSSL is arguably the most widely used cryptographic library, wolfSSL, despite its lower conventional popularity, still powers cryptography in billions of devices [wInc], making it a valuable second choice for our experiments.

in wolfSSL; the result is mainly due to the slow implementation of AES-SIV, which is used by all MRAE-preserving transforms.

2 Notation and Definitions

In the section, we introduce some notation, recall preliminary definitions of the building blocks, and define certain notions that we will use in this work.

Notation. Let ε denote the empty string and $\{0,1\}^*$ denote the set of all finite binary strings. For a binary string X , let $|X|$ denote its bit length. For a binary string X , where $|X| \geq n$, the last (i.e., rightmost) n bits of X is denoted by $X|_n$. Let the special symbol $\perp$ (which is outside $\{0,1\}^*$) denote uninitialized state or failure. Let $x \leftarrow y$ denote assigning y to x . For a randomized algorithm A , let $x \xleftarrow{\$} A$ denote running A and assigning the output to x ; if A is deterministic, we write $x \leftarrow A$ instead. For a finite set $\mathcal{S}$, let $|\mathcal{S}|$ denote its size and $S \xleftarrow{\$} \mathcal{S}$ denote sampling S from $\mathcal{S}$ uniformly at random. For two sets $\mathcal{X}$ and $\mathcal{Y}$, let $\mathcal{X} \leftarrow^{\cup} \mathcal{Y}$ denote $\mathcal{X} \leftarrow \mathcal{X} \cup \mathcal{Y}$. For a real number r , let $\lceil r \rceil$ denote the smallest integer that is larger than or equal to r . For a function H that takes only one input, $H(x_1, x_2, \dots, x_n)$ indicates that the inputs $x_1, x_2, \dots, x_n$ are injectively encoded to a single input before being fed into H ; note that they can be simply concatenated when the first $n-1$ inputs are of fixed lengths. For an adversary $\mathcal{A}$ (an algorithm), the notation $\mathcal{A}^O$ denotes that $\mathcal{A}$ has oracle access to $O(\cdot)$.

2.1 Pseudorandom Functions and Hash Functions

Pseudorandom functions (PRFs). For a function $F : \{0,1\}^\kappa \times \{0,1\}^* \rightarrow \{0,1\}^n$, consider two PRF games associated with an adversary $\mathcal{A}$. In the “real” world, $\mathcal{A}$ has oracle access to the function $F_K(\cdot) = F(K, \cdot)$, where K is randomly sampled from $\{0,1\}^\kappa$; in the “ideal” world, it has oracle access to a random function $f(\cdot)$, where f is uniformly sampled from $\mathcal{F}$, the family of all functions with domain $\{0,1\}^*$ and range $\{0,1\}^n$. In the end, $\mathcal{A}$ outputs a bit b' guessing which world it was in. Its PRF advantage measure is defined as

$$\text{Adv}_F^{\text{prf}}(\mathcal{A}) = \Pr\left[K \xleftarrow{\$} \{0,1\}^\kappa : \mathcal{A}^{F_K} \Rightarrow 1\right] - \Pr\left[f \xleftarrow{\$} \mathcal{F} : \mathcal{A}^f \Rightarrow 1\right].$$

F is called a PRF if the above advantage is sufficiently small for any efficient adversary $\mathcal{A}$.

Hash functions. An (unkeyed) hash function $H : \{0,1\}^* \rightarrow \mathcal{Y}$ maps a binary string $X \in \{0,1\}^*$ to an element Y from the message digest space $\mathcal{Y}$. In this work, we consider two types of digest spaces $\mathcal{Y} = \{0,1\}^u$ and $\mathcal{Y} = \{0,1\}^v \times \{0,1\}^w = \{0,1\}^{v+w}$, where u, v, w are integers. That is, the second type is a special case of the first type, with the digest space explicitly divided into two parts.

As shown in Figure 3, we consider two collision resistance notions for hash functions. The first security game CR_H^A defines the “regular” collision resistance for a hash function H with digest space $\mathcal{Y} = \{0,1\}^u$, where the adversary wins by finding a collision, i.e., two distinct messages hashing to the same digest. Then, the second security game $\text{rCR}_{H'}^A$ defines a so-called *right collision resistance* notion for a hash function H' with digest space $\mathcal{Y} = \{0,1\}^v \times \{0,1\}^w$, where the adversary wins by finding a collision only on the right part of the digest space $\{0,1\}^w$. Their advantage measures are defined respectively as

$$\text{Adv}_H^{\text{cr}}(\mathcal{A}) = \Pr\left[\text{CR}_H^A \Rightarrow 1\right] \quad \text{and} \quad \text{Adv}_{H'}^{\text{rct}}(\mathcal{A}) = \Pr\left[\text{rCR}_{H'}^A \Rightarrow 1\right].$$

From the above definitions, it is clear that right collision resistance implies collision resistance. It is generally believed that the practical SHA-2/3 hash families satisfy both collision resistance notions, but the security of right collision resistance is restricted by the right-part digest length w .

Game CR_H^A	Game $\text{rCR}_{H'}^A$
$(X_1, X_2) \xleftarrow{\$} \mathcal{A}$	$(X_1, X_2) \xleftarrow{\$} \mathcal{A} ; (Y_{1,L}, Y_{1,R}) \leftarrow H'(X_1) ; (Y_{2,L}, Y_{2,R}) \leftarrow H'(X_2)$
return $H(X_1) = H(X_2)$	return $Y_{1,R} = Y_{2,R}$

Fig. 3: The collision-resistance game (**left**) for a hash function $H : \{0,1\}^* \rightarrow \{0,1\}^v$, and the right-collision-resistance game (**right**) for a hash function $H' : \{0,1\}^* \rightarrow \{0,1\}^v \times \{0,1\}^w$.

Game $\text{REAL}_{\text{SE}}^{\mathcal{A}}$ $K \xleftarrow{\$} \{0, 1\}^{\kappa}$ $b' \xleftarrow{\$} \mathcal{A}^{\text{Enc}}$ return b'	Oracle $\text{Enc}(N, AD, M)$ $C \leftarrow \text{SE.Enc}(K, N, AD, M)$ return C	Game $\text{RAND}_{\text{SE}}^{\mathcal{A}}$ $b' \xleftarrow{\$} \mathcal{A}^{\text{Enc}}$ return b'	Oracle $\text{Enc}(N, AD, M)$ $C \xleftarrow{\$} \{0, 1\}^{\text{SE.clen}(M)}$ return C
---	--	--	---

Game $\text{INT-CTXT}_{\text{SE}}^{\mathcal{A}}$ $K \xleftarrow{\$} \{0, 1\}^{\kappa}$; $\mathcal{Q} \leftarrow \emptyset$ $\text{win} \leftarrow 0$ $\mathcal{A}^{\text{Enc, Ver}}$ return win	Oracle $\text{Enc}(N, AD, M)$ $C \leftarrow \text{SE.Enc}(K, N, AD, M)$ $\mathcal{Q} \leftarrow \{(N, AD, C)\}$ return C	Oracle $\text{Ver}(N, AD, C)$ $M \leftarrow \text{SE.Dec}(K, N, AD, C)$ if $M \neq \perp \wedge (N, AD, C) \notin \mathcal{Q}$: win $\leftarrow 1$ return $M \neq \perp$
--	---	--

Fig. 4: The privacy games, real (**top left**) and random (**top right**), and the authenticity game (**bottom**) for a symmetric encryption scheme SE.

2.2 Symmetric Encryption

A (nonce-based) *symmetric encryption (SE)* scheme SE specifies a pair of deterministic algorithms (Enc, Dec) and is associated with a key space $\{0,1\}^\kappa$, nonce space $\mathcal{N} \subseteq \{0,1\}^*$, associated data space $\mathcal{AD} \subseteq \{0,1\}^*$, and message space $\mathcal{M} \subseteq \{0,1\}^*$. Its encryption algorithm SE.Enc takes as input a key $K \in \{0,1\}^\kappa$, nonce $N \in \mathcal{N}$, associated data $AD \in \mathcal{AD}$, message $M \in \mathcal{M}$, and outputs a ciphertext $C \in \{0,1\}^*$. Its decryption algorithm SE.Dec takes as input a key K , nonce $N \in \mathcal{N}$, associated data $AD \in \mathcal{AD}$, ciphertext $C \in \{0,1\}^*$, and outputs either a message $M \in \mathcal{M}$ or the special symbol $\perp$ indicating decryption failure. Correctness requires that $\text{SE.Dec}(K, N, AD, \text{SE.Enc}(K, N, AD, M)) = M$ holds for all $K \in \{0,1\}^\kappa$, $N \in \mathcal{N}$, $AD \in \mathcal{AD}$, $M \in \mathcal{M}$.

The above SE syntax actually coincides with the syntax of an *authenticated encryption (AE)* scheme AE. For simplicity, we also abuse the same SE syntax to capture a basic nonce-based encryption scheme E, for which the associated data is ignored and correctness requires that $\text{E.Dec}(K, N, \text{E.Enc}(K, N, M)) = M$ holds for all $K \in \{0,1\}^\kappa$, $N \in \mathcal{N}$, $M \in \mathcal{M}$. A basic nonce-based encryption scheme E is said to be *tidy* [NRS14] if $M \leftarrow \text{E.Dec}(K, N, C)$ implies $\text{E.Enc}(K, N, M) = C$. Tidiness is a natural requirement for practical nonce-based encryption schemes to avoid unnecessary degeneration and mitigate security risks. Correctness and tidiness together imply that functions $\text{E.Enc}(K, N, \cdot)$ and $\text{E.Dec}(K, N, \cdot)$ are the inverse of each other.

We assume a symmetric encryption scheme SE (and also implicitly E) is further associated with a linear-time computable ciphertext-length function SE.clen such that $|\text{SE.Enc}(K, N, AD, M)| = \text{SE.clen}(|M|)$ holds for all K, N, AD, M . In this work, we consider only “natural” SE schemes that satisfy $\min_{M \in \mathcal{M}} \{\text{SE.clen}(|M|) - |M|\} = \tau \geq 0$, where the constant τ is called *ciphertext expansion* of SE. We note that, in order to achieve practical security, ciphertext expansions of real-world deployed AE schemes are usually of sufficient length, e.g., 96 or 128 bits.

For security of an SE scheme, in addition to AE security notions (i.e., privacy and authenticity), we also consider two committing security notions: context committing (CMT) and context discovery (CDY). Here “context” refers to the encryption input (K, N, AD, M) . These committing security notions can be viewed as analogous to collision-resistance and preimage-resistance for hash functions. If a symmetric encryption scheme is expected to ensure both AE security and committing security, it is called *committing authenticated encryption*.

AE security. We recall the basic privacy and authenticity notions as defined in [Rog02], for which the adversary is required to be *nonce-respecting*, i.e., it never repeats a nonce for its encryption queries. Besides, we also consider the stronger nonce-misuse resistant notions and a weaker privacy notion.

For privacy, consider the two privacy games shown in Figure 4 that define the notion of IND\$-CPA: indistinguishability from random bits under chosen-plaintext attack. In both games the adversary $\mathcal{A}$ has access to an encryption oracle Enc, which returns the encrypted ciphertext in the “real” world $\text{REAL}_{\text{SE}}^{\mathcal{A}}$ and returns random bits in the “ideal” world $\text{RAND}_{\text{SE}}^{\mathcal{A}}$. In the end, $\mathcal{A}$ outputs a bit b' guessing which world it was in. Its IND\$-CPA advantage measure is defined as

$$\text{Adv}_{\text{SE}}^{\text{ind\$-cpa}}(\mathcal{A}) = \Pr[\text{REAL}_{\text{SE}}^{\mathcal{A}} \Rightarrow 1] - \Pr[\text{RAND}_{\text{SE}}^{\mathcal{A}} \Rightarrow 1].$$

Game CMT_{SE}^A
$((K_1, N_1, AD_1, M_1), (K_2, N_2, AD_2, M_2)) \xleftarrow{\$} \mathcal{A}$
$C_1 \leftarrow \text{SE.Enc}(K_1, N_1, AD_1, M_1) ; C_2 \leftarrow \text{SE.Enc}(K_2, N_2, AD_2, M_2)$
return $C_1 = C_2 \wedge (K_1, N_1, AD_1) \neq (K_2, N_2, AD_2)$

Fig. 5: The context-committing game for a symmetric encryption scheme SE.

Game $\text{CDY}_{\text{SE}, S}^A$	Context selector $S_{\S}$
$C \xleftarrow{\$} S ; (K, N, AD, M) \xleftarrow{\$} \mathcal{A}(C)$	$K \xleftarrow{\$} \{0, 1\}^{\kappa} ; N \xleftarrow{\$} \mathcal{N} ; AD \xleftarrow{\$} \mathcal{AD} ; M \xleftarrow{\$} \mathcal{M}$
return $\text{SE.Enc}(K, N, AD, M) = C$	return $\text{SE.Enc}(K, N, AD, M)$

Fig. 6: The context-discovery game parameterized by a context selector S for a symmetric encryption scheme SE (left) and the context selector $S_{\S}$ (right).

If the adversary is restricted to make *at most one* query to its encryption oracle, the resulting weaker privacy notion is named one-time IND $\S$ -CPA (or simply OT-IND $\S$ -CPA), with its advantage measure denoted by $\text{Adv}_{\text{SE}}^{\text{ot-ind}\S\text{-cpa}}(\mathcal{A})$.

For authenticity, consider the authenticity game $\text{INT-CTXT}_{\text{SE}}^A$ also shown in Figure 4 that defines ciphertext integrity (INT-CTXT). Here, the adversary $\mathcal{A}$ wins by forging a new tuple (N, AD, C) that can be successfully decrypted. Its INT-CTXT advantage measure is defined as

$$\text{Adv}_{\text{SE}}^{\text{int-ctxt}}(\mathcal{A}) = \Pr[\text{INT-CTXT}_{\text{SE}}^A \Rightarrow 1].$$

Note that the above also defines privacy and authenticity security for a nonce-based encryption scheme E when associated data is ignored [Rog04]. In this work, when such encryption schemes are concerned, we implicitly refer to those that ensure only privacy but not authenticity. These basic encryption schemes usually serve as a component of AE schemes and hence have simpler and more efficient instantiations than AE schemes.

MRAE security. We also consider the stronger (nonce-)misuse-resistant (MR) AE security, for which the adversary is no longer required to be nonce-respecting. However, to avoid trivial wins, the adversary is not allowed to repeat a query to its encryption oracle. The misuse-resistant AE security notions are named MR-IND $\S$ -CPA and MR-INT-CTXT respectively, with advantage measures defined in the same way and denoted by $\text{Adv}_{\text{SE}}^{\text{mr-ind}\S\text{-cpa}}(\mathcal{A})$ and $\text{Adv}_{\text{SE}}^{\text{mr-int-ctxt}}(\mathcal{A})$.

Context-committing security. The context-committing game shown in Figure 5 defines the CMT-3 notion proposed in [BH22], which we refer to simply as CMT; as proved in [BH22], this notion is conceptually simpler than but security-wise equivalent to their strongest CMT-4 notion. The adversary wins by finding two different contexts encrypting to the same ciphertext. Its advantage measure is defined as

$$\text{Adv}_{\text{SE}}^{\text{cmt}}(\mathcal{A}) = \Pr[\text{CMT}_{\text{SE}}^A \Rightarrow 1].$$

Context-discovery security. We follow [BCC+24] to consider a simplified version of the context-discovery security notion proposed by [MLGR23], which we call CDY. As shown in Figure 6, the CDY game is parameterized by a so-called context selector S . The adversary is challenged with a ciphertext C selected by S and wins by finding a context (K, N, AD, M) that encrypts to C . We define its advantage measure as:

$$\text{Adv}_{\text{SE}, S}^{\text{cdy}}(\mathcal{A}) = \Pr[\text{CDY}_{\text{SE}, S}^A \Rightarrow 1].$$

In Figure 6 we also specify a context selector $S_{\S}$ that we think is of particular interest. $S_{\S}$ just samples the challenge context uniformly at random, with this context selector $S_{\S}$ the CDY notion can be viewed as *one-wayness* of the encryption algorithm. For simplicity, we write CDY $\S$ to denote this notion and its advantage measure is therefore defined as:

$$\text{Adv}_{\text{SE}}^{\text{cdy}\S}(\mathcal{A}) = \text{Adv}_{\text{SE}, S_{\S}}^{\text{cdy}}(\mathcal{A}) = \Pr[\text{CDY}_{\text{SE}, S_{\S}}^A \Rightarrow 1].$$

The above context selector $S_\$$ also occurred in the proof of Theorem 1 in [MLGR23]. This theorem shows that CMT implies CDY\$ for symmetric encryption schemes that are “context-compressing”, i.e., ciphertexts are decryptable under multiple contexts. This property is satisfied by many standardized AE schemes (e.g., AES-GCM) and their unstandardized CMT-secure variants (e.g., those proposed in [BH22]). Nevertheless, it is still interesting to explore both CMT and CDY\$ security for committing AE schemes that are generically transformed from AE schemes, e.g., $HtE \circ UtC$ [BH22], CTX [CR22], CTY [BH24], and the hash-based constructions that we will present in this work. Such schemes may involve larger ciphertext expansion due to the added committing security, and hence may not be context-compressing, especially when the $\mathcal{AD}$ size is small.

We note that Bhaumik *et al.* [BCC+24] proved CDY security for some SE schemes constructed from generic compositions, but their proofs can only work with context selectors that have *no* access to the underlying MAC component. As a result, the context selectors considered in [BCC+24] are actually quite restrictive, e.g., they cannot even capture the above $S_\$$ that encrypts a random context; also, such selectors may not produce a valid challenge ciphertext, for which CDY security trivially holds and is hence meaningless. Therefore, we focus on context selector $S_\$$ when analyzing CDY security of our constructions, but one can easily adapt our CDY\$ security proofs to handle arbitrary context selectors that have *no* access to the encryption algorithm.

3 Basic Transforms

We begin by exploring generic transforms that combine an AE scheme with a hash function, aiming for the resulting schemes to achieve both AE security and committing security. We refer to constructions in this section as *basic transforms*, as the more advanced MRAE security is not considered here.

Similar to the classical generic composition paradigms for the design of authenticated encryption schemes [BN00], we also consider three transform methods: Hash-then-AE (HtAE), AE-and-Hash (AEaH) and Encrypt-then-Hash (EtH). The latter transform is named Encrypt-then-Hash because it can be applied to both AE and basic privacy-only encryption schemes.

In the following, we first present our basic transforms and their security results individually, comparing each to similar existing transforms. Then, we compare the security guarantees of our basic transforms, deferring their performance comparison to a later section, where we also compare the performance of advanced transforms.

3.1 Hash-then-AE (HtAE)

For the Hash-then-AE (HtAE) transform, we present the HtAE construction. HtAE is built from a hash function $H : \{0,1\}^* \rightarrow \{0,1\}^\kappa \times \{0,1\}^t$ and an authenticated encryption scheme AE with key space $\{0,1\}^\kappa$. For simplicity, we consider HtAE with the same key space $\{0,1\}^\kappa$. Its pseudocode and the graphical illustration of its encryption algorithm HtAE.Enc are shown in Figure 7.

To encrypt, HtAE first computes the hash function H with the input (K, N, AD) to derive an symmetric key K_e and a committing tag T , then initializes the underlying AE scheme AE with key K_e and uses it to encrypt the input message M with nonce N and empty associated data, and finally outputs the derived AE ciphertext C together with T . To decrypt, HtAE also computes the hash function H to derive an symmetric key K_e and a committing tag T' , then checks if this tag T' is equal to the input committing tag T and outputs $\perp$ on failure; if the check passes, it uses AE to decrypt the input ciphertext C under key K_e with nonce N and empty associated data, and finally outputs the decrypted message M .

We present the security guarantees of HtAE in the following theorems, with proofs in Appendix B.1~B.4.

Theorem 1. *Let the underlying hash function of HtAE be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_e and q_h queries respectively to oracles Enc and H , there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{HtAE}}^{\text{ind}\$-\text{cpa}}(\mathcal{A}) \leq q_e \text{Adv}_{\text{AE}}^{\text{ot-ind}\$-\text{cpa}}(\mathcal{B}) + \frac{q_h}{2^\kappa}.$$

Theorem 2. *Let the underlying hash function of HtAE be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_e , q_v , q_h queries respectively to oracles Enc , Ver , H , there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{HtAE}}^{\text{int-ctxt}}(\mathcal{A}) \leq (q_e + q_v) \text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{B}) + \frac{q_h}{2^\kappa}.$$

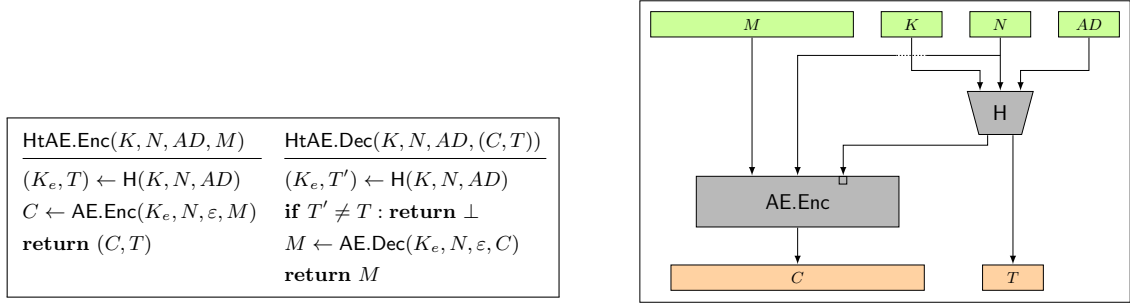


Fig. 7: The pseudocode (**left**) of HtAE transform and the graphical illustration (**right**) of its encryption algorithm.

Theorem 3. *For any efficient adversary $\mathcal{A}$, there exists an efficient adversary $\mathcal{B}$ (that can be constructed from $\mathcal{A}$) such that*

$$\text{Adv}_{\text{HtAE}}^{\text{cmt}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{rcr}}(\mathcal{B}).$$

Remark 1. The above theorem reduces CMT security of HtAE to right collision resistance of H to avoid idealized assumptions, but one can also easily prove it in the random oracle model. We follow the same style when analyzing CMT security of our other transforms.

Theorem 4. *Let the underlying hash function of HtAE be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_h random oracle queries, we have*

$$\text{Adv}_{\text{HtAE}}^{\text{cdy\$}}(\mathcal{A}) \leq \frac{1}{|\mathcal{AD}|} \cdot \frac{q_h + 1}{2^\kappa} + \frac{q_h + 1}{2^t}.$$

Remark 2. As implied in the proof, if there is no associated data (i.e., $\mathcal{AD} = \emptyset$), the above security bound still holds without the $|\mathcal{AD}|$ term.

Comparison with CommitKeyII and $\text{HtE} \circ \text{UtC}$. Among the existing transforms, CommitKeyII [ADG+22] and $\text{HtE} \circ \text{UtC}$ [BH22] follow a similar paradigm to our HtAE. They are also designed to first derive a *fresh* encryption key K_e for each distinct nonce and then feed this K_e to the underlying AE. As discussed below, our HtAE can be viewed as a simpler hash-based counterpart.

- Instead of computing a single hash, CommitKeyII uses two separate collision-resistant PRFs to derive the encryption key K_e and the committing tag T , respectively. It is quite similar to our HtAE, as in practice collision-resistant PRFs can be instantiated with hash functions. However, CommitKeyII incurs one more primitive call (and hence is slower than HtAE) and does not ensure CMT security.
- $\text{HtE} \circ \text{UtC}$ improved CommitKeyII by first computing a hash and then calling a more efficient primitive called committing PRF. Nevertheless, this transform still makes two primitive calls to derive K_e and T . As we will show later in Section 5, our HtAE outperforms $\text{HtE} \circ \text{UtC}$ even when the committing PRF is instantiated with the practical block-cipher-based construction proposed in [BH22]. More importantly, unlike hash functions, their practical instantiation is not yet standardized.

3.2 AE-and-Hash (AEaH)

Then, for the AE-and-Hash (AEaH) transform, we present the AEaH construction. AEaH is built from a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^t$ and an authenticated encryption scheme AE with key space $\{0, 1\}^\kappa$. Again, for simplicity, we consider AEaH with the same key space $\{0, 1\}^\kappa$. Its pseudocode and the graphical illustration of its encryption algorithm AEaH.Enc are shown in Figure 8.

To encrypt, AEaH performs two separate computations: (i) using the underlying AE scheme AE to encrypt the input message M under the input key K with the input nonce N and empty associated data, to derive an AE ciphertext C , (ii) computing the hash function $H(K, N, AD)$, to derive a committing tag T . Finally, it outputs (C, T) as its ciphertext. To decrypt, AEaH performs two similar separate computations: (iii) using AE to decrypt the input ciphertext C under the input key K with the input nonce N and empty associated data, to derive a decrypted message M , (iv) computing $H(K, N, AD)$, to derive a committing tag T' . Then, it checks if AE decryption succeeds (i.e., $M \neq \perp$) and if T' is equal to the input committing tag T , and finally it outputs M if the check passes or outputs $\perp$ otherwise.

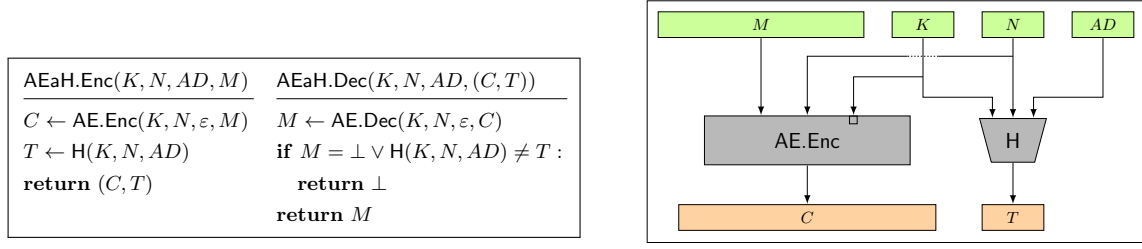


Fig. 8: The pseudocode (**left**) of AEaH transform and the graphical illustration (**right**) of its encryption algorithm.

We note that computations (i) and (ii) are independent of each other, as are (iii) and (iv), so in practice they can be executed *in parallel*.

The security guarantees of AEaH are presented in the following theorems, with proofs in Appendix B.5~B.8.

Theorem 5. *Let the underlying hash function of AEaH be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_e and q_h queries respectively to oracles Enc and H , there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{AEaH}}^{\text{ind\$-cpa}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{AE}}^{\text{ind\$-cpa}}(\mathcal{B}) + \frac{q_h}{2^\kappa},$$

where $\mathcal{B}$ makes at most $q_e + 1$ encryption queries and its one extra encryption query (if any) is on a κ -bit message.

Theorem 6. *Let the underlying hash function of AEaH be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_e , q_v , q_h queries respectively to oracles Enc , Ver , H , there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{AEaH}}^{\text{int-ctxt}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{B}) + \frac{q_v}{2^t},$$

where $\mathcal{B}$ makes q_e encryption queries and at most $q_v + q_h$ verification queries.

Theorem 7. *For any efficient adversary $\mathcal{A}$, there exists an efficient adversary $\mathcal{B}$ (that can be constructed from $\mathcal{A}$) such that*

$$\text{Adv}_{\text{AEaH}}^{\text{cmt}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{cr}}(\mathcal{B}).$$

Theorem 8. *Let the underlying hash function of AEaH be modeled as a random oracle H and let τ be the ciphertext expansion of the underlying AE scheme AE. For any efficient adversary $\mathcal{A}$ making q_h random oracle queries, there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{AEaH}}^{\text{cdy\$}}(\mathcal{A}) \leq \text{Adv}_{\text{AE}}^{\text{ot-ind\$-cpa}}(\mathcal{B}) + \frac{q_h + 1}{2^\tau} + \frac{q_h + 1}{2^t}.$$

Comparison with comPACT. The recently proposed comPACT [BBD24] is the most similar existing transform to AEaH. In fact, it computes the AE ciphertext C and H digest T in the same way as in AEaH. However, comPACT uses the H digest T as an encryption key rather than a committing tag. More precisely, it applies a block cipher under key T to encrypt the authentication tag (or at most one last block) of the derived AE ciphertext C , denoted by T_C , to produce T_C^* ; it then outputs C with T_C replaced by the modified tag T_C^* .

The extra tag encryption in comPACT is adopted to reduce ciphertext expansion; however, as mentioned in the Introduction, the maximum committing security of the resulting scheme is thus restricted to half the block size. In contrast, our AEaH can provide stronger committing security by choosing a hash function with large enough digest size. By construction, it is obvious that AEaH has higher efficiency than comPACT, and our AEaH is fully parallelizable.

3.3 Encrypt-then-Hash (EtH)

Finally, we present the EtH construction for the Encrypt-then-Hash transform. Here, we use “Encrypt” to emphasize that our EtH transform can be applied to both AE schemes and basic privacy-only encryption schemes. In this section, we adopt the syntax and security of a basic encryption scheme $\bar{\text{E}}$ to describe our

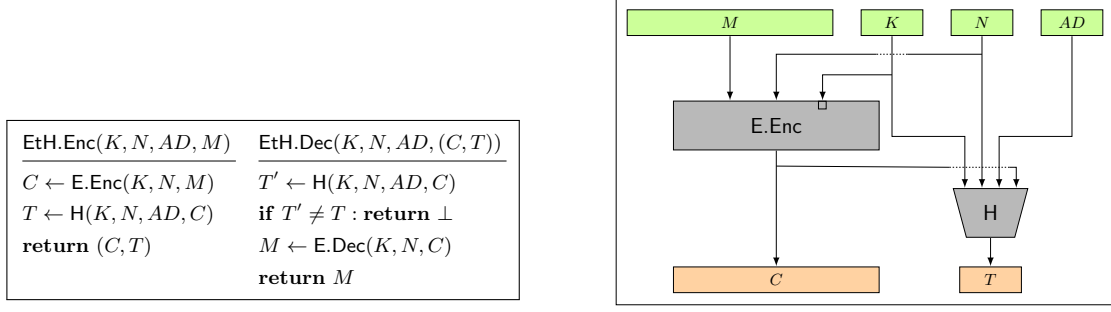


Fig. 9: The pseudocode (**left**) of EtH transform and the graphical illustration (**right**) of its encryption algorithm.

construction and analyze the security results, but one can also easily adapt the construction and results to capture arbitrary AE schemes.

EtH is built from a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^t$ and a symmetric encryption scheme E with key space $\{0, 1\}^\kappa$. Again, for simplicity, we consider EtH with the same key space $\{0, 1\}^\kappa$. Its pseudocode and the graphical illustration of its encryption algorithm EtH.Enc are shown in Figure 9.

To encrypt, EtH first uses the underlying AE scheme E to encrypt the input message M under the input key K with the input nonce N , to derive an E ciphertext C ; then it computes the hash function H with input (K, N, AD, C) , to derive a committing tag T ; finally it outputs (C, T) as its ciphertext. To decrypt, EtH first computes the hash function H to derive a committing tag T' , then checks if this tag T' is equal to the input committing tag T and outputs $\perp$ on failure; if the check passes, it uses E to decrypt the input ciphertext C under key K with nonce N and outputs the decrypted message M .

We present the security guarantees of EtH in the following theorems, with proofs in Appendix B.9~B.12. We note that our security results for EtH do *not* require the underlying basic encryption scheme E to be tidy.

Theorem 9. *Let the underlying hash function of EtH be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_e and q_h queries respectively to oracles Enc and H , there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{EtH}}^{\text{ind\$-cpa}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{E}}^{\text{ind\$-cpa}}(\mathcal{B}) + \frac{q_h}{2^\kappa},$$

where $\mathcal{B}$ makes at most $q_e + 1$ encryption queries and its one extra encryption query (if any) is on a κ -bit message.

Theorem 10. *Let the underlying hash function of EtH be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_e, q_v, q_h queries respectively to oracles $\text{Enc}, \text{Ver}, H$, there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{EtH}}^{\text{int-ctxt}}(\mathcal{A}) \leq \text{Adv}_{\text{E}}^{\text{ind\$-cpa}}(\mathcal{B}) + \frac{q_h}{2^\kappa} + \frac{q_v}{2^t},$$

where $\mathcal{B}$ makes $q_e + 1$ encryption queries and its one extra encryption query is on a κ -bit message.

Remark 3. The above theorem reduces the INT-CTXT security of EtH to the IND\\$-CPA security of E , because the underlying basic encryption scheme E may not have authenticity security. However, if our Encrypt-then-Hash transform is built from an authenticated encryption scheme AE, one can perform a proof (omitted here) very similar to that of Theorem 6 to reduce the INT-CTXT security of our transform to the INT-CTXT security of the underlying AE.

Theorem 11. *For any efficient adversary $\mathcal{A}$, there exists an efficient adversary $\mathcal{B}$ (that can be constructed from $\mathcal{A}$) such that*

$$\text{Adv}_{\text{EtH}}^{\text{cmt}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{cr}}(\mathcal{B}).$$

Theorem 12. *Let the underlying hash function of EtH be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_h random oracle queries, there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{EtH}}^{\text{cdy\$}}(\mathcal{A}) \leq \text{Adv}_{\text{E}}^{\text{ind\$-cpa}}(\mathcal{B}) + \frac{q_h + 1}{2^\kappa} + \frac{q_h + 1}{2^t},$$

where $\mathcal{B}$ makes two encryption queries and one of them is on a κ -bit message.

Comparison with CTY and the libsodium transform. Among the existing transforms, CTY [BH24] and the libsodium transform [Aut] are the closest to EtH. Both follow the Encrypt-then-Hash paradigm, but they are constructed from AE schemes that produce a separate authentication tag and hash only this tag, rather than the entire AE ciphertext. The two transforms differ in how they handle the authentication tag in the output: CTY omits it, whereas the libsodium transform retains it, similar to our EtH when the underlying encryption scheme is an AE scheme.

With practical instantiations, CTY and the libsodium transform usually have better performance than EtH, even when the underlying encryption scheme of EtH is instantiated with basic privacy-only encryption. For example, as demonstrated by our implementation results in Section 5, CTY instantiated with AES-GCM runs faster than EtH instantiated with AES-CTR (i.e., AES in counter mode) for common message sizes, because hashing the entire ciphertext (as in EtH) is typically slower than MACing it within AE (as in CTY).

However, CTY and the libsodium transform are not black-box transforms and cannot be applied to basic privacy-only encryption schemes; our EtH, by contrast, can work with arbitrary AE and basic privacy-only encryption schemes, making it a more powerful generic transform. Additionally, when instantiated with basic encryption schemes that do not incur ciphertext expansion (e.g., with AES-CTR), our EtH exhibits the same compact ciphertext expansion as CTY.

3.4 Security Comparison

Now, we compare the security guarantees of our three basic transforms: HtAE, AEaH, EtH, when instantiated with the same or consistent component primitives.

AE security. All of our basic transforms achieve AE security.

To achieve IND\$-CPA security, HtAE requires only *one-time* IND\$-CPA security for its underlying AE, while AEaH and EtH require the regular IND\$-CPA security. This is because, for each HtAE encryption with a distinct nonce, a fresh AE key K_e is derived and used solely for encrypting that one message.

For INT-CTXT security, the guarantees for our basic transforms are similarly aligned, except for EtH with basic privacy-only encryption instantiation. In this case, as remarked for Theorem 10, the INT-CTXT security of EtH has to rely on the IND\$-CPA security of the underlying basic encryption scheme E.

Committing security. All of our basic transforms achieve committing security.

To achieve CMT security, AEaH and EtH rely on collision resistance of the underlying hash function, while HtAE requires the stronger right collision resistance. As a result, to ensure the same level of CMT security, the hash function used in HtAE must have a larger digest size compared to the other two transforms, leading to more constraint choices for hash instantiations.

For CDY\$ security, however, HtAE has a better security bound than the other two transforms. As shown in Theorem 4, its CDY\$ security does not rely on the security of its underlying AE, which means HtAE can still ensure context-discovery security even if AE is broken. This is because revealing the AE key K_e used in HtAE does not help with determining the input key K . In contrast, AEaH and EtH directly use the same input key K for encryption, so breaking the underlying encryption scheme would reveal the input message, key and nonce. As discussed in the end of Section 2.2, CDY\$ security is of particular interest when the associated data space $\mathcal{AD}$ has small size, and in this case breaking the encryption scheme compromises CDY\$ security. Therefore, AEaH and EtH have to rely on the privacy security of the underlying encryption scheme to ensure context-discovery security; however, they require only very minimal one-time or two-time IND\$-CPA security for privacy.

4 Advanced Transforms

In this section, we present our two *advanced* generic transforms. The first simpler transform adds committing security to an AE scheme while preserving its misuse-resistance (MR) security, while the second transform directly “lifts” a basic privacy-only encryption scheme to a committing AE that is secure against nonce misuse.

In the following, we first present our advanced transforms and their security results individually, comparing the first with similar existing MRAE-preserving transforms. Then, we instantiate our transforms with consistent standardized component primitives and compare their security guarantees.

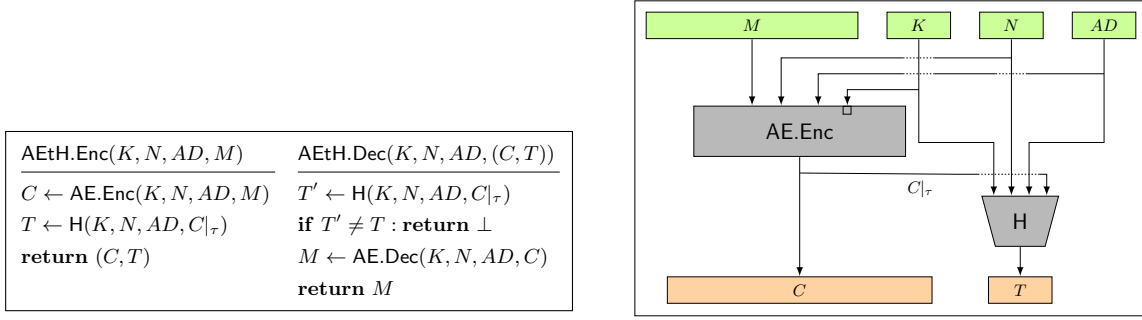


Fig. 10: The pseudocode (**left**) of AEtH transform and the graphical illustration (**right**) of its encryption algorithm.

4.1 MRAE-Preserving Transform AEtH

For our MRAE-preserving transform, we present the AEtH construction, which follows the AE-then-Hash paradigm. AEtH is built from a hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^t$ and an authenticated encryption scheme AE with key space $\{0, 1\}^\kappa$. For simplicity, we consider AEtH with the same key space $\{0, 1\}^\kappa$. Its pseudocode and the graphical illustration of its encryption algorithm AEtH.Enc are shown in Figure 10.

As shown in the figure, AEtH is very similar to our basic EtH transform (see Figure 9), except that the basic encryption scheme E in EtH is replaced with AE in AEtH and only the last τ bits of the AE ciphertext is fed into the hash function. Here τ refers to the ciphertext expansion of the underlying AE, but it can be adjusted for stronger security when the length of the AE ciphertext is guaranteed to have more than τ bits. We also note that our AEtH transform can be seen as a black-box generalization of the libsodium approach [LAut], but their latest transform, like CTY, does not achieve MR security because the associated data is omitted from the underlying AE.

To encrypt, AEtH first uses the underlying AE scheme AE to encrypt the input message M under the input key K with the input nonce N and associated data AD , to derive an AE ciphertext C ; then it computes the hash function H with input $(K, N, AD, C|_\tau)$, to derive a committing tag T ; finally it outputs (C, T) as its ciphertext. To decrypt, AEtH first computes the hash function H to derive a committing tag T' , then checks if this tag T' is equal to the input committing tag T and outputs $\perp$ on failure; if the check passes, it uses AE to decrypt the input ciphertext C under key K with the input nonce N and associated data AD , and outputs the decrypted message M .

The security guarantees of AEtH are presented in the following theorems, with proofs in Appendix B.13~B.16.

Theorem 13. *Let the underlying hash function of AEtH be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_e and q_h queries respectively to oracles Enc and H , there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{AEtH}}^{\text{mr-ind\$-cpa}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{AE}}^{\text{mr-ind\$-cpa}}(\mathcal{B}) + \frac{q_e^2}{2^{\tau+1}} + \frac{q_h}{2^\kappa},$$

where $\mathcal{B}$ makes at most $q_e + 1$ encryption queries and its one extra encryption query (if any) is on a κ -bit message.

Remark 4. We remark that the $\frac{q_e^2}{2^{\tau+1}}$ term is suboptimal with respect to the allowed number of encryptions, e.g., it restricts $q_e \leq 2^{48.5}$ for $\tau = 128$ (the typical expansion size) with a 2^{-32} advantage. This, however, could be circumvented by (i) padding all messages with l bits before encryption and then feeding $\tau + l$ bits of the derived AE ciphertext into H , or (ii) excluding short messages from the message space, which ensures sufficiently long ciphertexts. We note that some other advanced transforms such as CTX [CR22] and PACT [BBD24] also suffer from the above suboptimal term.⁸ The above circumvention ideas can be applied to CTX, but fail for PACT, as the expansion size τ of PACT is fixed to the block size by design due to the added block cipher encryption.

⁸ The term did not appear explicitly in the original work that presents CTX [CR22], since the authors proved only AE security in the basic nonce-respecting setting. However, due to the very similar design of CTX and our AEtH, our security proof can be directly applied to CTX to derive almost identical bounds.

Theorem 14. *Let the underlying hash function of AEtH be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_e , q_v , q_h queries respectively to oracles Enc , Ver , H , there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{AEtH}}^{\text{mr-int-ctxt}}(\mathcal{A}) \leq 2 \cdot \text{Adv}_{\text{AE}}^{\text{mr-int-ctxt}}(\mathcal{B}),$$

where $\mathcal{B}$ makes q_e encryption queries and at most $q_v + q_h$ verification queries.

Theorem 15. *For any efficient adversary $\mathcal{A}$, there exists an efficient adversary $\mathcal{B}$ (that can be constructed from $\mathcal{A}$) such that*

$$\text{Adv}_{\text{AEtH}}^{\text{cmt}}(\mathcal{A}) \leq \text{Adv}_{\text{H}}^{\text{cr}}(\mathcal{B}).$$

Theorem 16. *Let the underlying hash function of AEtH be modeled as a random oracle H and let τ be the ciphertext expansion of the underlying AE scheme AE. For any efficient adversary $\mathcal{A}$ making q_h random oracle queries, there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{AEtH}}^{\text{cdy\$}}(\mathcal{A}) \leq \text{Adv}_{\text{AE}}^{\text{ot-ind\$-cpa}}(\mathcal{B}) + \frac{q_h + 1}{2^\tau} + \frac{q_h + 1}{2^t}.$$

Comparison with existing MRAE-preserving transforms. There are three existing transforms that also preserve MRAE security: HtE $\circ$ RtC [BH22], CTX [CR22] and PACT [BBD24]. We compare them to our AEtH as follows:

- The HtE $\circ$ RtC transform follows the same approach as HtE $\circ$ UtC, with RtC applied to a misuse-resistant AE scheme rather than a basic one. As a result, HtE $\circ$ RtC still suffers from the expensive cost to re-key the underlying AE per distinct nonce, resulting in worse performance than our AEtH. Furthermore, like HtE $\circ$ UtC, the HtE $\circ$ RtC transform also makes use of the non-standard committing PRF primitive.
- Our AEtH can be viewed as a black-box generalization of the CTX transform, where CTX is restricted to hash a separate authentication tag of the underlying AE ciphertext rather than any part of it. The performance of CTX and AEtH are hence almost the same, with CTX saving τ -bit ciphertext output.
- The black-box transform PACT [BBD24] is the heavier variant of comPACT, aiming to preserve misuse-resistance security. The only difference between PACT and comPACT is that, in PACT, the underlying AE inputs AD rather than an empty string as associated data, where the same difference also applies to CTX and CTY. As with comPACT, PACT applies an additional block cipher encryption after hashing, leading to slower performance than our AEtH.

4.2 MRAE-Lifting Transform chaSIV

Finally, we present our MRAE-lifting transform named chaSIV, which stands for *committing hash-based SIV*. Like our EtH transform, this advanced chaSIV can also be applied to both AE schemes and basic privacy-only encryption schemes. In this section, we adopt the syntax and security of a basic encryption scheme E to describe our construction and analyze the security results, but one can easily adapt the construction and results to capture arbitrary AE schemes.

As its name indicates, chaSIV follows the synthetic IV (SIV) paradigm [RS06]: it just replaces the MAC in SIV with a hash function and adds a PRF to re-key the underlying encryption scheme per distinct nonce. As a result, in addition to hash function $H : \{0, 1\}^* \rightarrow \{0, 1\}^t$ and an encryption scheme E with key space $\{0, 1\}^\kappa$, chaSIV also makes use of a PRF $F : \{0, 1\}^\kappa \times \{0, 1\}^r \rightarrow \{0, 1\}^\kappa$. For chaSIV to be properly defined, we require $t \geq r$, i.e., the committing tag size must be at least the nonce size, which holds for all natural and practical choices of t and r . As with our previous transforms, we consider chaSIV with the same key space $\{0, 1\}^\kappa$. Its pseudocode and the graphical illustration of its encryption algorithm chaSIV.Enc are shown in Figure 11.

To encrypt, chaSIV first uses PRF F under the input key K to derive the encryption key K_e for the underlying E . Then, it hashes the whole context with H to derive a committing tag T . The last r bits of T are then used as a nonce for E to encrypt the input message M and get ciphertext C . Finally, it outputs (C, T) as its ciphertext. To decrypt, chaSIV derives K_e as is done in encryption and takes the last r bits of the input committing tag T as nonce N_e . Then, it uses E to decrypt the input ciphertext C under the key K_e with nonce N_e . If decryption fails, it outputs $\perp$; otherwise, chaSIV gets the decrypted message M and computes the hash function H over (K, N, AD, M) to derive T' . Finally, it checks if $T' = T$ and outputs $\perp$ on failure; otherwise, it outputs M .

The security guarantees of chaSIV are presented in the following theorems, with proofs in Appendix B.17~B.20.

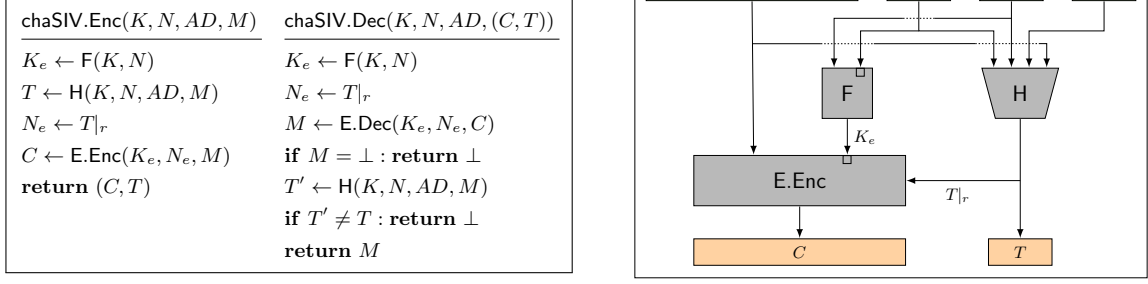


Fig. 11: The pseudocode (**left**) of chaSIV[F, H, E] transform and the graphical illustration (**right**) of its encryption algorithm.

Theorem 17. *Let the underlying hash function of chaSIV be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_h queries to oracle H and querying oracle Enc with Q_e distinct nonces, where the i -th nonce is repeated R_i times for $i \in \{1, \dots, Q_e\}$, there exist efficient adversaries $\mathcal{B}$ and $\mathcal{C}$ such that*

$$\text{Adv}_{\text{chaSIV}}^{\text{mr-ind\$-cpa}}(\mathcal{A}) \leq Q_e \cdot \text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}) + 2 \cdot \text{Adv}_F^{\text{prf}}(\mathcal{C}) + \frac{\sum_{i=1}^{Q_e} R_i^2}{2^r} + \frac{q_h}{2^\kappa},$$

where $\mathcal{B}$ makes at most $\max_{i \in \{1, \dots, Q_e\}} \{R_i\}$ encryption queries and $\mathcal{C}$ makes at most $Q_e + 1$ PRF oracle queries.

Remark 5. Note that here we do not have the suboptimal quadratic term that appeared in Theorem 13. Instead, we have $\frac{\sum_{i=1}^{Q_e} R_i^2}{2^r}$, which typically allows for more encryptions for a common nonce size, e.g., $r = 96$, since in practice the number of repetitions is usually not high, e.g., hundreds for random nonces.

Theorem 18. *Let the underlying hash function of chaSIV be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_h, q_v queries respectively to oracles H, Ver and querying oracles Enc, Ver with a total of Q distinct nonces, there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{chaSIV}}^{\text{mr-int-ctxt}}(\mathcal{A}) \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}) + \frac{q_h}{2^\kappa} + \frac{q_v}{2^t},$$

where $\mathcal{B}$ makes at most $Q + 1$ PRF oracle queries.

Remark 6. The above theorem requires the underlying encryption scheme E to be *tidy*. In our entire work, this is the only theorem that relies on tidiness.

Theorem 19. *For any efficient adversary $\mathcal{A}$, there exists an efficient adversary $\mathcal{B}$ (that can be constructed from $\mathcal{A}$) such that*

$$\text{Adv}_{\text{chaSIV}}^{\text{cmt}}(\mathcal{A}) \leq \text{Adv}_H^{\text{cr}}(\mathcal{B}).$$

Theorem 20. *Let the underlying hash function of chaSIV be modeled as a random oracle H . For any efficient adversary $\mathcal{A}$ making q_h random oracle queries, there exists an efficient adversary $\mathcal{B}$ such that*

$$\text{Adv}_{\text{chaSIV}}^{\text{cdy\$}}(\mathcal{A}) \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}) + \frac{q_h + 1}{2^\kappa} + \frac{q_h + 1}{2^t},$$

where $\mathcal{B}$ makes only one PRF oracle query.

4.3 Security Comparison

We now compare the security guarantees of our advanced transforms, AEtH and chaSIV, when instantiated with consistent component primitives. Since they respectively employ misuse-resistant AE and basic encryption schemes, which typically have quite different concrete security bounds, we focus on specific consistent primitives when comparing their security. In particular, we consider the misuse-resistant AE instantiated with AES-GCM-SIV, basic encryption scheme E instantiated with AES-CTR, and the PRF F of chaSIV instantiated with AES.

MRAE security. Both of our advanced transforms achieve MRAE security. We next compare their concrete security under the aforementioned instantiations.

As proved in [GL17], the dominating terms of the MRAE security bound for AES-GCM-SIV are $\frac{3Q}{2^{96}}, \frac{QB_{\max}^2}{2^{128}}, \frac{l_{\max} \sum_{i=1}^Q R_i^2}{2^{126}}$, where Q is the number of distinct nonces queried across *both* encryption and decryption/verification oracles, $l_{\max}$ is the maximum message length in blocks, and $B_{\max}$ is the maximum number of blocks queried per nonce (i.e. $B_{\max} \leq \max_i \{R_i\} \cdot l_{\max}$). Plugging these terms into Theorem 13 (with $\tau = 128$) and Theorem 14 yields the dominating terms of the MRAE security bound for our AEtH: $\frac{3Q}{2^{95}}, \frac{QB_{\max}^2}{2^{128}}, \frac{l_{\max} \sum_{i=1}^Q R_i^2}{2^{125}}, \frac{q_e^2}{2^{129}}$.

Then, for B queried blocks, the dominating term of both the IND\$-CPA security bound for AES-CTR and the PRF security bound for AES is $\frac{B^2}{2^{128}}$, as shown in [IR89; Rog04]. Plugging these terms into Theorem 17 (with $r = 96$) and Theorem 18 yields the dominating terms of the MRAE security bound for our chaSIV: $\frac{Q_e B_{\max}^2}{2^{128}}, \frac{\sum_{i=1}^{Q_e} R_i^2}{2^{96}}, \frac{3(Q+1)^2}{2^{128}}$.

By comparing the above dominating terms, we observe that AEtH and chaSIV achieve very similar concrete security.⁹ When nonce repetitions are considered, $\frac{3(Q+1)^2}{2^{128}}$ is better than $\frac{q_e^2}{2^{129}}$, e.g., with at least 3 repetitions per nonce on average; this aligns with our remarks for Theorem 13 and Theorem 17. Therefore, chaSIV has only one slightly worse term compared to AEtH: for $l_{\max} < 2^{29}$ we could have $\frac{\sum_{i=1}^{Q_e} R_i^2}{2^{96}} > \frac{l_{\max} \sum_{i=1}^Q R_i^2}{2^{125}}$. However, for common nonce repetitions in practice, e.g., at most 2^{10} on average, $\frac{\sum_{i=1}^{Q_e} R_i^2}{2^{96}}$ is typically good enough, allowing $Q_e = 2^{44}$ with a 2^{-32} advantage, very close to the Q limit imposed by the $\frac{3(Q+1)^2}{2^{128}}$ term.

Committing security. Both of our advanced transforms achieve committing security. Their CMT security bounds are identical, both relying on the collision resistance of the underlying hash function. They offer very similar CDY\$ security, with AEtH relying on one-time IND\$-CPA security of its underlying AE, while chaSIV relies on PRF security (with a single query) of its underlying F; in concrete instantiations, these IND\$-CPA and PRF bounds become minor. The dominating terms for CDY\$ security of AEtH is determined by the sizes of the committing tag and ciphertext expansion, while those for chaSIV are similarly determined by the sizes of the committing tag and the key.

5 Performance Evaluation

Finally, we evaluate the performance of our basic and advanced transforms, along with relevant existing transforms, both in theory and through implementations.

Theoretical evaluation. For theoretical performance evaluation, we focus on the following factors of the considered transforms: ciphertext expansion, number of cryptographic passes over specific targets, key consistency across encryptions/decryptions, and parallelizability. We also include the minimal assumptions made by the transforms on the component encryption primitive to capture their primitive applicability. The results are summarized in Table 2.

As shown in the table, HtE $\circ$ UtC exhibits the worst efficiency among the AE-based basic transforms because it involves more computation passes and, more crucially, re-keys its AE component for each encryption/decryption. Similarly, HtE $\circ$ RtC runs the slowest among the MRAE-preserving advanced transforms.

To reduce ciphertext expansion, CTY and comPACT introduce an additional cryptographic pass over T_C , the separate authentication tag (or the last block) of the derived AE ciphertext. This extra pass, besides requiring more computation, renders such transforms non-parallelizable. Furthermore, to support black-box transforms, comPACT encrypts T_C rather than hashing it, using a non-fixed key. Similar operations are performed by their advanced transform variants, CTX and PACT, but the cryptographic pass over T_C is now employed by all MRAE-preserving transforms to achieve misuse-resistant security. This makes all such MRAE-preserving transforms non-parallelizable.

As motivated in the Introduction, our transforms prioritize black-boxness and design simplicity over minimizing ciphertext expansion. As shown in Table 2, when ciphertext expansion is not a primary concern, our AEaH exhibits the best performance among the AE-based basic transforms; additionally, AEaH

⁹ We remark that actually the existing MRAE-preserving transforms HtE $\circ$ RtC [BH22], CTX [CR22], PACT [BBD24] all show very similar MRAE concrete security, as their security bounds are mostly dominated by the MRAE security of the underlying AE.

Construction	Min. Assm. on Enc. Primitive	Ciphertext Expansion	Passes over (K, N, AD, M, C, T_C)	Fixed Key	Parallel.
HtE $\circ$ UtC [BH22]	AE-secure	$\tau + t$	(2, 2, 1, 1, 0, 0)	✗	✗
CTY [BH24]	AE-secure	t	(1, 1, 1, 1, 0, 1)	✓	✗
comPACT [BBD24]	AE-secure	τ	(1, 1, 1, 1, 0, 1)	(✗)	(✗)
HtAE (this work)	AE-secure	$\tau + t$	(1, 1, 1, 1, 0, 0)	✗	✗
AEaH (this work)	AE-secure	$\tau + t$	(1, 1, 1, 1, 0, 0)	✓	✓
EtH (this work)	IND\$-CPA-secure	t	(1, 1, 1, 1, 1, 0)	✓	✗
HtE $\circ$ RtC [BH22]	MRAE-secure	$\tau + t$	(2, 2, 1, 1, 0, 1)	✗	✗
CTX [BH24]	MRAE-secure	t	(1, 1, 2, 1, 0, 1)	✓	✗
PACT [BBD24]	MRAE-secure	τ	(1, 1, 2, 1, 0, 1)	(✗)	(✗)
AEtH (this work)	MRAE-secure	$\tau + t$	(1, 1, 2, 1, 0, 1)	✓	✗
chaSIV (this work)	IND\$-CPA-secure	t	(2, 2, 1, 2, 0, 0)	✗	✗

Table 2: Theoretical performance evaluation of our generic transforms and relevant others. Ciphertext expansion is measured in terms of bit length, where τ is the ciphertext expansion of the component encryption scheme and t is the size of the committing tag. For the ciphertext C derived from the component encryption scheme, let T_C denote its separate authentication tag (if any) or its last block. The (✗) symbol indicates partial satisfaction of the key consistency (as shown in the “Fixed Key” column) or parallelizability properties.

is fully parallelizable, making it even more attractive. Similarly, our AEtH, along with CTX, outperforms other MRAE-preserving advanced transforms.

Among the transforms presented in the table, our EtH and chaSIV are the only ones that support basic privacy-only encryption schemes, and using these schemes results in compact ciphertext expansion. We observe that, when instantiated with AE, EtH guarantees the same security as AEaH but exhibits strictly worse performance. Therefore, EtH is primarily designed to accommodate basic encryption schemes. However, it is useful to apply chaSIV to both basic and authenticated encryption schemes, as AE schemes may not be misuse-resistant and hence cannot be accommodated by MRAE-preserving transforms. The performance of EtH and chaSIV is compared to other transforms through implementations, as we will show shortly.

Finally, we note that while the primary standardized misuse-resistant AE, AES-GCM-SIV, requires re-keying across distinct nonces, some schemes do support key consistency. For example, Deoxys-II [JNPS21], the winner of the CAESAR competition for the *defense-in-depth* category, is one such scheme. This suggests that advanced fixed-key transforms, e.g., CTX and our AEtH, may offer potential performance benefits when used with suitable AE schemes.

Implementation results. To assess the practical efficiency of our transforms and relevant others, we implemented their encryption algorithms using both OpenSSL (version 3.3.2) and wolfSSL (version 5.7.0).¹⁰ For basic transforms we used AES-GCM speed (CPU cycles per byte) as a benchmark, and for advanced transforms we used AES-GCM-SIV (resp. AES-SIV) in OpenSSL (resp. wolfSSL). Performance is tested with message lengths ranging from 16 to 2048 bytes, using a 5-byte associated data that matches the length of the authenticated but unencrypted part of the TLS 1.3 header. Further testing details are presented in Appendix A. The results are presented in Figure 12 for basic transforms and Figure 13 for advanced ones.

For all AE-based transforms, we instantiate the basic (unique-nonce) AE with AES-GCM; the misuse-resistant AE is instantiated with AES-GCM-SIV in OpenSSL and AES-SIV in wolfSSL, as the latter is the only misuse-resistant AE available in wolfSSL. The basic privacy-only scheme E in EtH and chaSIV is instantiated with AES-CTR in OpenSSL and AES-GCM in wolfSSL, as AES-CTR is significantly slower than AES-GCM in wolfSSL. All encryption schemes use a 128-bit key, i.e., $\kappa = 128$. For HtE $\circ$ UtC/HtE $\circ$ RtC, we instantiate their underlying committing PRF with the CX scheme proposed and suggested by the authors in [BH22]. The block ciphers used in CX and comPACT/PACT are instantiated with AES under a 128-bit key. The PRF F in chaSIV is also instantiated with AES under a 128-bit key; if the input nonce is shorter than the block size, it is automatically padded by the libraries. The hash functions

¹⁰ The implementation artifact can be accessed at <https://artifacts.iacr.org/eurocrypt/2025/a6/>.

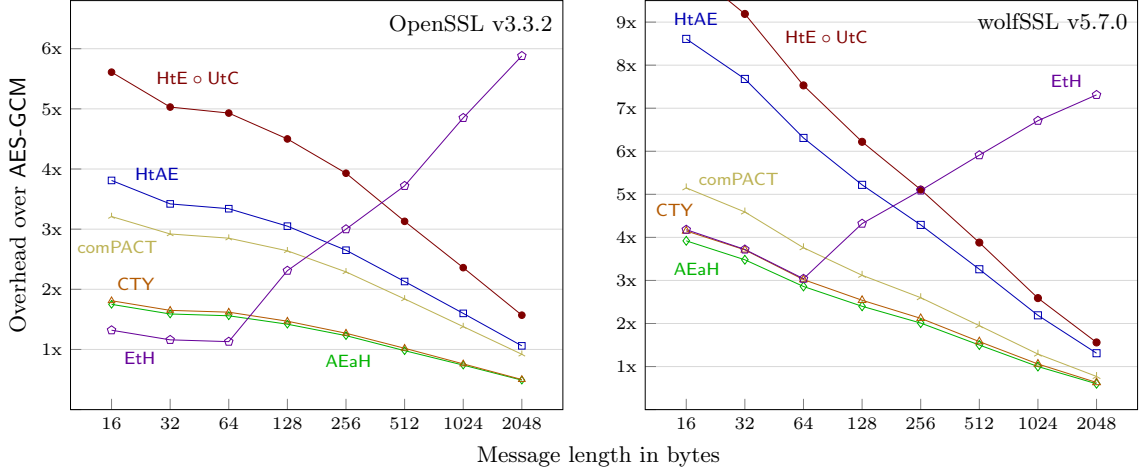


Fig. 12: Encryption speed overhead of basic transforms compared to AES-GCM. Smaller is better.

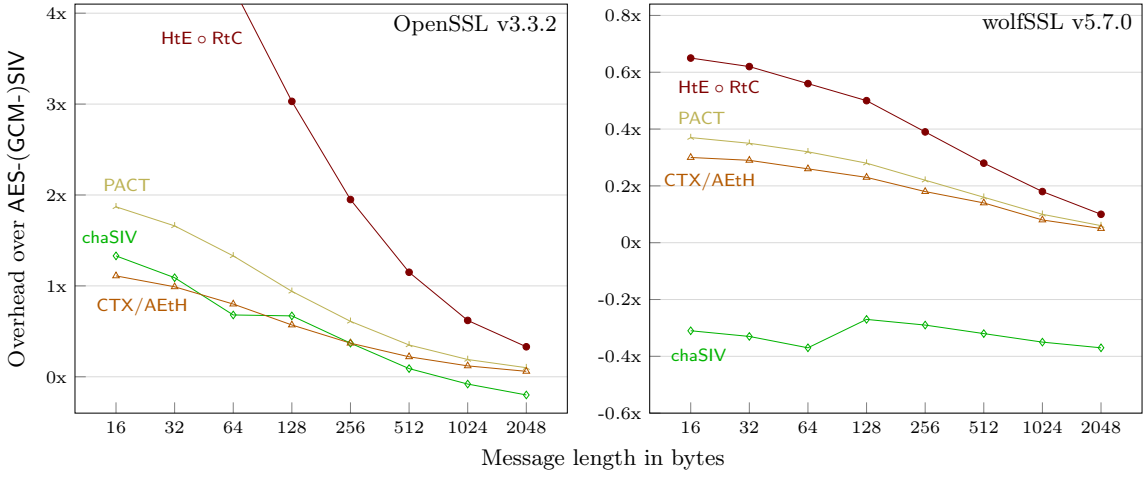


Fig. 13: Encryption speed overhead of advanced transforms compared to AES-GCM-SIV in OpenSSL and AES-SIV in wolfSSL. Smaller is better.

are instantiated with truncated SHA-512 in all implemented transforms, with a 128- or 256-bit output depending on the transform.

As shown in Figure 12, our implementation results demonstrate that AEaH has the highest efficiency among basic transforms, which matches our theoretical discussions. For all tested message lengths, the overhead of AEaH is no more than twice the benchmark. Furthermore, we note that the AEaH transform is implemented *sequentially*, so we expect that an implementation fully utilizing its parallelizability could perform even better. For EtH, one may notice the sudden increase in its efficiency overhead. This is because for message length of 79 bytes the hash function input reaches 112 bytes,¹¹ which triggers a doubling of the padded input processed by SHA-512, causing the hashing cost to surpass the underlying MACing cost of the AE component in other transforms. However, for short messages (e.g., less than 64 bytes), EtH shows the highest efficiency.

For advanced transforms, as shown in Figure 13, our results demonstrate that AEtH/CTX and chaSIV achieve the best performance across all tested message sizes. Their overhead is within 1.5 times the benchmark in both libraries. In wolfSSL, due to the slow AES-SIV implementation, we notice that chaSIV, which does not rely on AES-SIV, outperforms all other transforms by a large margin. In OpenSSL, chaSIV outperforms AEtH/CTX for messages longer than about 360 bytes, and even surpasses the benchmark AES-GCM-SIV for messages exceeding 1024 bytes. The faster performance of our chaSIV can be explained by the suboptimal OpenSSL implementation of AES-GCM-SIV. In particular, the CTR mode is imple-

¹¹ Apart from the 79 message bytes, there is a 16-byte key, 12-byte nonce and 5-byte associated data, all of which sums up to 112 bytes.

mented in plain C using ECB mode implicitly, which prevents full utilization of the CPU pipeline.¹² In addition, although the polynomial hash algorithm POLYVAL of AES-GCM-SIV leverages hardware acceleration, it is not implemented directly; instead, the implementation relies on GHASH, introducing extra computational overhead [GLL17].¹³

The results show that our `chaSIV` not only is the first committing MRAE-lifting transform but also has great practical potential.

Implementation benefits of our transforms. During our implementation process, we found that our transforms are very easy to implement. If the practitioners are familiar with the cryptographic library they use, implementing our transforms would take just a couple of lines of code per functionality (e.g., encryption algorithm). Unfortunately, this is not the case for some of the other transforms. For example, implementing the decryption algorithm of CTY is tricky and inefficient (e.g., in OpenSSL) or even impossible (e.g., in wolfSSL) using the standard API functions in commonly used cryptographic libraries. This is because the API functions for AES-GCM decryption are not designed for (or do not allow at all) decrypting only the core part of the ciphertext (i.e., without the authentication tag) and recalculating the original authentication tag, which is needed for a final verification check in CTY.

6 Conclusion

Our work explored hash-based black-box generic transforms to add strong committing security to authenticated encryption schemes. To this end, we proposed three basic transforms and two advanced transforms, all exhibiting a simple design and strong security. As part of this effort, we developed the first generic transform that directly elevates a basic encryption scheme to a committing and misuse-resistant authenticated encryption scheme. By focusing on a straightforward design and relying crucially on efficient and well-standardized hash functions, we hope that our transforms will promote the adoption and deployment of committing authenticated encryption.

Acknowledgments. Shan Chen was supported by the research start-up grant from the Southern University of Science and Technology. Vukašin Karadžić was supported by the German Federal Ministry of Education and Research and the Hessen State Ministry for Higher Education, Research and the Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE and by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – SFB 1119 – 236615297.

References

- [ADG+22] Ange Albertini, Thai Duong, Shay Gueron, Stefan Kölbl, Atul Luykx, and Sophie Schmieg. “How to Abuse and Fix Authenticated Encryption Without Key Commitment”. In: *USENIX Security 2022: 31st USENIX Security Symposium*. Ed. by Kevin R. B. Butler and Kurt Thomas. Boston, MA, USA: USENIX Association, Aug. 2022, pp. 3291–3308 (cit. on pp. 3, 4, 12).
- [BBD24] Arghya Bhattacharjee, Ritam Bhaumik, and Chandranan Dhar. *Universal Context Commitment without Ciphertext Expansion*. Cryptology ePrint Archive, Report 2024/1382. 2024. URL: <https://eprint.iacr.org/2024/1382> (cit. on pp. 4, 5, 13, 16, 17, 19, 20).
- [BCC+24] Ritam Bhaumik, Bishwajit Chakraborty, Wonseok Choi, Avijit Dutta, Jérôme Govinden, and Yaobin Shen. “The Committing Security of MACs with Applications to Generic Composition”. In: *Advances in Cryptology – CRYPTO 2024, Part IV*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14923. Lecture Notes in Computer Science. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 2024, pp. 425–462 (cit. on pp. 4, 5, 7, 10, 11).
- [BDJR97] Mihir Bellare, Anand Desai, Eron Jokipii, and Phillip Rogaway. “A concrete security treatment of symmetric encryption”. In: *Proceedings 38th Annual Symposium on Foundations of Computer Science*. IEEE. 1997, pp. 394–403 (cit. on p. 3).
- [Ber05] Daniel J Bernstein. “The Poly1305-AES message-authentication code”. In: *International workshop on fast software encryption*. Springer. 2005, pp. 32–49 (cit. on p. 3).

¹² https://github.com/openssl/openssl/blob/openssl-3.3/providers/implementations/ciphers/cipher_aes_gcm_siv_hw.c#L335

¹³ https://github.com/openssl/openssl/blob/openssl-3.3/providers/implementations/ciphers/cipher_aes_gcm_siv_polyval.c#L78

- [Ber08] Daniel J Bernstein. “ChaCha, a variant of Salsa20”. In: *Workshop record of SASC*. Vol. 8. 1. Citeseer. 2008, pp. 3–5 (cit. on p. 3).
- [BH22] Mihir Bellare and Viet Tung Hoang. “Efficient Schemes for Committing Authenticated Encryption”. In: *Advances in Cryptology – EUROCRYPT 2022, Part II*. Ed. by Orr Dunkelman and Stefan Dziembowski. Vol. 13276. Lecture Notes in Computer Science. Trondheim, Norway: Springer, Cham, Switzerland, May 2022, pp. 845–875 (cit. on pp. 3–5, 10–12, 17, 19, 20).
- [BH24] Mihir Bellare and Viet Tung Hoang. “Succinctly-Committing Authenticated Encryption”. In: *Advances in Cryptology – CRYPTO 2024, Part IV*. Ed. by Leonid Reyzin and Douglas Stebila. Vol. 14923. Lecture Notes in Computer Science. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 2024, pp. 305–339 (cit. on pp. 4, 6, 11, 15, 20).
- [BN00] Mihir Bellare and Chanathip Namprempre. “Authenticated encryption: Relations among notions and analysis of the generic composition paradigm”. In: *ASIACRYPT 2000*. Springer. 2000, pp. 531–545 (cit. on pp. 3, 5, 6, 11).
- [BNT19] Mihir Bellare, Ruth Ng, and Björn Tackmann. “Nonces Are Noticed: AEAD Revisited”. In: *CRYPTO 2019, Part I*. Ed. by Alexandra Boldyreva and Daniele Micciancio. Vol. 11692. LCNS. Springer, Heidelberg, Aug. 2019, pp. 235–265 (cit. on p. 6).
- [BRSA26] Mihir Bellare, Rishabh Ranjan, Nujud Senan, and Basel Alomair. *UCX is All You Need: A Universal Transform for Committing Authenticated Encryption*. Cryptology ePrint Archive, Paper 2026/1222. 2026. URL: <https://eprint.iacr.org/2026/1222> (cit. on p. 5).
- [CDD+24] Yu Long Chen, Michael Davidson, Morris Dworkin, Jinkeon Kang, John Kelsey, Yu Sasaki, Meltem Sönmez Turan, Donghoon Chang, Nicky Mouha, and Alyssa Thompson. *Proposal of Requirements for an Accordion Mode: Discussion Draft for the NIST Accordion Mode Workshop 2024*. 2024 (cit. on p. 4).
- [CR22] John Chan and Phillip Rogaway. “On Committing Authenticated-Encryption”. In: *ESORICS 2022: 27th European Symposium on Research in Computer Security, Part II*. Ed. by Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng. Vol. 13555. Lecture Notes in Computer Science. Copenhagen, Denmark: Springer, Cham, Switzerland, Sept. 2022, pp. 275–294 (cit. on pp. 3, 4, 6, 7, 11, 16, 17, 19).
- [DGRW18] Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. “Fast Message Franking: From Invisible Salamanders to Encryptment”. In: *Advances in Cryptology – CRYPTO 2018, Part I*. Ed. by Hovav Shacham and Alexandra Boldyreva. Vol. 10991. Lecture Notes in Computer Science. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 2018, pp. 155–186 (cit. on pp. 3, 4).
- [Dwo07] Morris J Dworkin. “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC”. In: (2007) (cit. on pp. 3, 5, 7).
- [FM21] Marc Fischlin and Arno Mittelbach. *An Overview of the Hybrid Argument*. Cryptology ePrint Archive, Report 2021/088. <https://eprint.iacr.org/2021/088>. 2021 (cit. on p. 25).
- [FOR17] Pooya Farshim, Claudio Orlandi, and Răzvan Roşie. “Security of Symmetric Primitives under Incorrect Usage of Keys”. In: *IACR Transactions on Symmetric Cryptology* 2017.1 (2017), pp. 449–473 (cit. on pp. 3–5).
- [GL17] Shay Gueron and Yehuda Lindell. “Better bounds for block cipher modes of operation via nonce-based key derivation”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2017, pp. 1019–1036 (cit. on p. 19).
- [GLL17] Shay Gueron, Adam Langley, and Yehuda Lindell. “AES-GCM-SIV: specification and analysis”. In: (2017). <https://eprint.iacr.org/2017/168> (cit. on p. 22).
- [GLL19] Shay Gueron, Adam Langley, and Yehuda Lindell. *AES-GCM-SIV: Nonce Misuse-Resistant Authenticated Encryption*. RFC 8452. Apr. 2019. URL: <https://www.rfc-editor.org/info/rfc8452> (cit. on pp. 3, 7).
- [GLR17] Paul Grubbs, Jiahui Lu, and Thomas Ristenpart. “Message Franking via Committing Authenticated Encryption”. In: *Advances in Cryptology – CRYPTO 2017, Part III*. Ed. by Jonathan Katz and Hovav Shacham. Vol. 10403. Lecture Notes in Computer Science. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 2017, pp. 66–97 (cit. on pp. 3, 4).
- [Har08] Dan Harkins. *Synthetic Initialization Vector (SIV) Authenticated Encryption Using the Advanced Encryption Standard (AES)*. RFC 5297. Oct. 2008. URL: <https://www.rfc-editor.org/info/rfc5297> (cit. on p. 7).
- [HKR15] Viet Tung Hoang, Ted Krovetz, and Phillip Rogaway. “Robust Authenticated-Encryption AEZ and the Problem That It Solves”. In: *Advances in Cryptology – EUROCRYPT 2015*,

- Part I*. Ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9056. Lecture Notes in Computer Science. Sofia, Bulgaria: Springer Berlin Heidelberg, Germany, Apr. 2015, pp. 15–44 (cit. on p. 4).
- [HM24] Viet Tung Hoang and Sanketh Menda. “Robust AE With Committing Security”. In: *Advances in Cryptology – ASIACRYPT 2024, Part IX*. Ed. by Kai-Min Chung and Yu Sasaki. Vol. 15492. Lecture Notes in Computer Science. Kolkata, India: Springer, Singapore, Singapore, Dec. 2024, pp. 312–342 (cit. on p. 4).
- [IR89] Russell Impagliazzo and Steven Rudich. “Limits on the provable consequences of one-way permutations”. In: *Proceedings of the twenty-first annual ACM symposium on theory of computing*. 1989, pp. 44–61 (cit. on p. 19).
- [JNPS21] Jérémy Jean, Ivica Nikolic, Thomas Peyrin, and Yannick Seurin. “The Deoxys AEAD Family”. In: *Journal of Cryptology* 34.3 (July 2021), p. 31 (cit. on p. 20).
- [lAut] libsodium Authors. *libsodium documentation, Secret-key cryptography, AEAD constructions, Robustness*. https://doc.libsodium.org/secret-key_cryptography/aead#robustness. Accessed on 11.18.2025 (cit. on pp. 4, 15, 16).
- [LGR21] Julia Len, Paul Grubbs, and Thomas Ristenpart. “Partitioning oracle attacks”. In: *USENIX Security 2021*. USENIX Association. 2021, pp. 195–212 (cit. on p. 3).
- [MLGR23] Sanketh Menda, Julia Len, Paul Grubbs, and Thomas Ristenpart. “Context Discovery and Commitment Attacks - How to Break CCM, EAX, SIV, and More”. In: *Advances in Cryptology – EUROCRYPT 2023, Part IV*. Ed. by Carmit Hazay and Martijn Stam. Vol. 14007. Lecture Notes in Computer Science. Lyon, France: Springer, Cham, Switzerland, Apr. 2023, pp. 379–407 (cit. on pp. 3, 6, 10, 11).
- [NRS14] Chanathip Namprempre, Phillip Rogaway, and Thomas Shrimpton. “Reconsidering Generic Composition”. In: *Advances in Cryptology – EUROCRYPT 2014*. Ed. by Phong Q. Nguyen and Elisabeth Oswald. Vol. 8441. Lecture Notes in Computer Science. Copenhagen, Denmark: Springer Berlin Heidelberg, Germany, May 2014, pp. 257–274 (cit. on pp. 4, 9).
- [NSS24] Yusuke Naito, Yu Sasaki, and Takeshi Sugawara. “KIVR: Committing Authenticated Encryption Using Redundancy and Application to GCM, CCM, and More”. In: *International Conference on Applied Cryptography and Network Security*. Springer. 2024, pp. 318–347 (cit. on p. 5).
- [NST25] Yusuke Naito, Yu Sasaki, and Takeshi Takeshi. “Committing Wide Encryption Mode with Minimum Ciphertext Expansion”. In: *IACR Transactions on Symmetric Cryptology* 2025.1 (Mar. 2025), pp. 44–69. URL: <https://tosc.iacr.org/index.php/ToSC/article/view/12071> (cit. on p. 4).
- [RBB03] Phillip Rogaway, Mihir Bellare, and John Black. “OCB: A block-cipher mode of operation for efficient authenticated encryption”. In: *ACM Transactions on Information and System Security (TISSEC)* 6.3 (2003), pp. 365–403 (cit. on p. 3).
- [Rog02] Phillip Rogaway. “Authenticated-Encryption With Associated-Data”. In: *ACM CCS 2002: 9th Conference on Computer and Communications Security*. Ed. by Vijayalakshmi Atluri. Washington, DC, USA: ACM Press, Nov. 2002, pp. 98–107 (cit. on pp. 3, 9).
- [Rog04] Phillip Rogaway. “Nonce-based symmetric encryption”. In: *International Workshop on Fast Software Encryption*. Springer. 2004, pp. 348–358 (cit. on pp. 10, 19).
- [RS06] Phillip Rogaway and Thomas Shrimpton. “A provable-security treatment of the key-wrap problem”. In: *Annual international conference on the theory and applications of cryptographic techniques*. Springer. 2006, pp. 373–390 (cit. on pp. 7, 17).
- [Sho04] Victor Shoup. *Sequences of games: a tool for taming complexity in security proofs*. Cryptology ePrint Archive, Report 2004/332. <https://eprint.iacr.org/2004/332>. 2004 (cit. on pp. 25, 26, 28, 29, 31, 32, 35, 36, 38–40, 42, 44, 46).
- [SMC+24] Meltem Sönmez Turan, Kerry McKay, Donghoon Chang, Jinkeon Kang, and John Kelsey. *Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions*. Tech. rep. National Institute of Standards and Technology, 2024 (cit. on p. 5).
- [SW24] Patrick Struck and Maximiliane Weishäupl. “Constructing Committing and Leakage-Resilient Authenticated Encryption”. In: *IACR Transactions on Symmetric Cryptology* 2024.1 (2024), pp. 497–528 (cit. on pp. 4, 5).
- [WHF03] Doug Whiting, Russ Housley, and Niels Ferguson. *Counter with CBC-MAC (CCM)*. Tech. rep. 2003 (cit. on p. 3).

[wInc] wolfSSL Inc. *Case Studies*. <https://www.wolfssl.com/docs/case-studies/>. Accessed on 28.01.2025 (cit. on p. 7).

A Performance Testing Details

For each tested message length (ranging from 16 bytes to 2048 bytes) and every tested transform, we start the experiment by running the encryption 500 times to “warm-up” the cache, and then run 1000 encryptions and measure the average number of cycles per byte needed for one encryption call. We ran the experiment thirty times, discarded the five best and worst results and then calculated the average number of cycles needed over the remaining twenty runs. The resulting average serves as the final performance cost. As a benchmark for comparison, we measure the performance of AES-GCM for basic transforms, and AES-GCM-SIV/AES-SIV for advanced transforms in the same way described above. The tests are run on a notebook using Intel Core i5-8265U CPU (Skylake microarchitecture), with the base frequency of 1.6GHz and the hyper-threading, frequency scaling and turbo mode functionalities disabled.

B Security Proofs

Our proofs follow the *game-based technique* (see [Sho04] for a tutorial) and some proofs require a *hybrid argument* (see [FM21] for a tutorial). In the proofs, when the hash function H is modeled as a random oracle H , the random oracle H is instantiated via *lazy sampling*, i.e., an independent random output is sampled for each distinct H query and the same output is returned for the same query.

B.1 Proof of Theorem 1 (IND\$-CPA Security of HtAE)

Proof. We define three games $\mathbf{G}_0 \sim \mathbf{G}_2$ as shown in Figure 14.

Game $\boxed{\mathbf{G}_0}, \mathbf{G}_1$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H(X) \wedge X = (K, *)$:	$(K_e, T) \xleftarrow{\$} \{0, 1\}^\kappa \times \{0, 1\}^t$
$T_H[\cdot] \leftarrow \perp$	bad $\leftarrow \text{true}$	$(K_e, T) \leftarrow H(K, N, AD)$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$C \leftarrow \text{AE.Enc}(K_e, N, \varepsilon, M)$
return b'	return $T_H[X]$	return (C, T)

Game $\mathbf{G}_2$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \xleftarrow{\$} \{0, 1\}^t$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	return $T_H[X]$	$C \xleftarrow{\$} \{0, 1\}^{\text{AE.clen}(M)}$
return b'		return (C, T)

Fig. 14: Games $\mathbf{G}_0 \sim \mathbf{G}_2$ in the proof of Theorem 1, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

Game $\mathbf{G}_0$ is the same as $\text{REAL}_{\text{HtAE}}^A$ (see Figure 4) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^A \Rightarrow 1] = \Pr[\text{REAL}_{\text{HtAE}}^A \Rightarrow 1]$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that the AE key K_e and committing tag T are now sampled uniformly at random from the set $\{0, 1\}^\kappa \times \{0, 1\}^t$ in the encryption oracle Enc . We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically until the **bad** event happens, where the adversary $\mathcal{A}$ ever queries H with an input that is prefixed with the key K sampled in the beginning of the games. The only syntactical difference between the two games is the generation of K_e and T . Recall that $\mathcal{A}$ is nonce-respecting, i.e., it never repeats a nonce for its encryption queries. Therefore, if **bad** does not occur, then in both games the AE key K_e and committing tag T are random strings that are independent of $\mathcal{A}$'s view, and hence games $\mathbf{G}_0$ and $\mathbf{G}_1$ are identical-until-bad. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^A \Rightarrow 1] - \Pr[\mathbf{G}_1^A \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (1)$$

Then, note that K is independent of $\mathcal{A}$'s view, because K_e and T do not reveal any information about the random oracle input K . Since K is sampled from $\{0, 1\}^\kappa$ uniformly at random and $\mathcal{A}$ makes at most q_h random oracle queries, we have

$$\Pr[\text{bad}] \leq \frac{q_h}{2^\kappa}. \quad (2)$$

Game $\mathbf{G}_2$ is the same as game $\mathbf{G}_1$, except that in the Enc oracle both the committing tag T and the AE ciphertext C are sampled uniformly at random. To bound the gap between $\mathbf{G}_1$ and $\mathbf{G}_2$, we apply a hybrid argument as follows. Consider a sequence of hybrid games $\mathbf{H}_0, \mathbf{H}_1, \dots, \mathbf{H}_{q_e}$ such that $\mathbf{G}_1 = \mathbf{H}_0$ and $\mathbf{G}_2 = \mathbf{H}_{q_e}$. Game $\mathbf{H}_i$ is the same as game $\mathbf{G}_1$, except that the output of the first i Enc queries are replaced by independent random strings as in $\mathbf{G}_2$. By construction, consecutive games $\mathbf{H}_{i-1}$ and $\mathbf{H}_i$ are the same except for the i -th Enc query. We can construct an adversary $\mathcal{B}_i$ against the IND $\mathbb{S}$ -CPA security of the underlying AE such that it simulates $\mathcal{A}$'s view in $\mathbf{H}_{i-1}$ in the real world and $\mathbf{H}_i$ in the ideal world. To do so, $\mathcal{B}_i$ simulates $\mathcal{A}$'s views in $\mathbf{H}_{i-1}$ and $\mathbf{H}_i$ in the same way following their game procedures, except that, for the i -th Enc query $\mathcal{B}_i$ samples a random T and calls its own encryption oracle Enc' to simulate the AE ciphertext C . Finally, $\mathcal{B}_i$ outputs the bit that $\mathcal{A}$ outputs. By construction, we have $\Pr[\mathbf{H}_{i-1}^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{H}_i^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{AE}}^{\text{ot-ind}\mathbb{S}\text{-cpa}}(\mathcal{B}_i)$, because $\mathcal{B}_i$ makes only one Enc' query. With a hybrid argument, there exists an efficient adversary $\mathcal{B}$ such that

$$\begin{aligned} \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] &= \Pr[\mathbf{H}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{H}_{q_e}^{\mathcal{A}} \Rightarrow 1] \\ &= \sum_{i=1}^{q_e} (\Pr[\mathbf{H}_{i-1}^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{H}_i^{\mathcal{A}} \Rightarrow 1]) \leq q_e \text{Adv}_{\text{AE}}^{\text{ot-ind}\mathbb{S}\text{-cpa}}(\mathcal{B}). \end{aligned} \quad (3)$$

Finally, we observe that game $\mathbf{G}_2$ is identical to $\text{RAND}_{\text{HtAE}}^{\mathcal{A}}$ (see Figure 4) equipped with a random oracle H . This is because, in game $\mathbf{G}_2$, for each Enc query with a distinct nonce, its output is an independent uniform random string sampled from $\{0, 1\}^{\text{HtAE.clen}(|M|)} = \{0, 1\}^{\text{AE.clen}(|M|)+t}$. Therefore, we have

$$\text{Adv}_{\text{HtAE}}^{\text{ind}\mathbb{S}\text{-cpa}}(\mathcal{A}) = \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1]. \quad (4)$$

The proof is concluded by combining equations (1)~(4). $\square$

B.2 Proof of Theorem 2 (INT-CTXT Security of HtAE)

Proof. We define two games $\mathbf{G}_0$ and $\mathbf{G}_1$ as shown in Figure 15.

Game $\mathbf{G}_0$ is the same as $\text{INT-CTXT}_{\text{HtAE}}^{\mathcal{A}}$ (see Figure 4) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{HtAE}}^{\text{int-ctxt}}(\mathcal{A})$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that it aborts when the bad event **bad** occurs, which is triggered when adversary $\mathcal{A}$ queries H with an input that is prefixed with the key K sampled in the beginning of the games. Since K is sampled from $\{0, 1\}^\kappa$ uniformly at random and it is independent of $\mathcal{A}$'s view, the probability that **bad** occurs is at most $q_h/2^\kappa$. By construction, $\mathbf{G}_0$ and $\mathbf{G}_1$ are identical-until-bad. Therefore, by the Difference Lemma of the game-based proof technique [Sho04], we have

$$|\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1]| \leq \Pr[\text{bad}] \leq \frac{q_h}{2^\kappa}. \quad (5)$$

Finally, we bound $\mathcal{A}$'s winning probability $\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{B}$ against the INT-CTXT security of the underlying AE. To simulate $\mathcal{A}$'s view in $\mathbf{G}_1$, adversary $\mathcal{B}$ first samples a random key $K \xleftarrow{\$} \{0, 1\}^\kappa$. Then, for each random oracle query made by $\mathcal{A}$ on an input X not prefixed with K , adversary $\mathcal{B}$ simulates $H(X)$ via lazy sampling. For encryption and verification queries on input $(N, AD, \cdot)$ made by $\mathcal{A}$, we note that each distinct (N, AD) pair determines an independent key K_e for the underlying AE. Therefore, to simulate responses to such queries, $\mathcal{B}$ first randomly picks one of such distinct (N, AD) pairs, say the i -th distinct pair denoted by $(\hat{N}, \hat{AD})$, and then samples an independent random committing tag $\hat{T} \xleftarrow{\$} \{0, 1\}^t$. For each encryption or verification query on input $(N, AD, \cdot) \neq (\hat{N}, \hat{AD}, \cdot)$, $\mathcal{B}$ can easily simulate its response: it first derives an AE key K_e and a committing tag T by simulating the output of $H(K, N, AD)$ via lazy sampling, then it uses (K_e, T) to simulate the rest of the oracle. For a $(\hat{N}, \hat{AD}, \cdot)$ query, $\mathcal{B}$ does not simulate the random oracle output but instead calls its own INT-CTXT Enc'/Ver' oracle to simulate the answer. A code-based description of $\mathcal{B}$ is shown in Figure 15.

Note that, if **bad** does not occur, then $\mathcal{A}$'s random oracle queries do not contain any information about the AE keys. Therefore, by construction of $\mathcal{B}$, it simulates $\mathcal{A}$'s view in $\mathbf{G}_1$ perfectly. Then, suppose $\mathcal{A}$ wins by querying $\text{Ver}(N^*, AD^*, (C^*, T^*))$ and $\mathcal{B}$ happens to pick the correct (N, AD) pair (i.e., $(\hat{N}, \hat{AD}) =$

Game $\mathbf{G}_0, \mathbf{G}_1$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H(X)$	$(K_e, T) \leftarrow H(K, N, AD)$	$(K_e, T') \leftarrow H(K, N, AD)$
$T_H[\cdot] \leftarrow \perp$	$\wedge X = (K, *) :$	$C \leftarrow \text{AE.Enc}(K_e, N, \varepsilon, M)$	if $T' \neq T : \text{return false}$
$\mathcal{Q} \leftarrow \emptyset$	bad $\leftarrow \text{true}$	$\mathcal{Q} \leftarrow \{(N, AD, (C, T))\}$	$M \leftarrow \text{AE.Dec}(K_e, N, \varepsilon, C)$
$\text{win} \leftarrow 0$	abort	return (C, T)	if $M \neq \perp \wedge (N, AD, (C, T)) \notin \mathcal{Q} :$
$\mathcal{A}^{\text{H,Enc,Ver}}$	if $T_H[X] = \perp :$		$\text{win} \leftarrow 1$
return win	$T_H[X] \xleftarrow{\$} \{0, 1\}^{\kappa+t}$		return $M \neq \perp$
	return $T_H[X]$		

Adv. $\mathcal{B}^{\text{Enc}', \text{Ver}'}$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queried $H(X)$	if (N, AD) is the $\hat{i}$ -th	if (N, AD) is the $\hat{i}$ -th
$\hat{i} \xleftarrow{\$} [q_e + q_v]$	$\wedge X = (K, *) :$	distinct pair :	distinct pair :
$\hat{T} \xleftarrow{\$} \{0, 1\}^t$	abort	$T \leftarrow \hat{T}$	if $T \neq \hat{T} : \text{return false}$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp :$	$C \leftarrow \text{Enc}'(N, \varepsilon, M)$	return $\text{Ver}'(N, \varepsilon, C)$
$\mathcal{A}^{\text{H,Enc,Ver}}$	$T_H[X] \xleftarrow{\$} \{0, 1\}^{\kappa+t}$	else	else
	return $T_H[X]$	$(K_e, T) \leftarrow H(K, N, AD)$	$(K_e, T') \leftarrow H(K, N, AD)$
		$C \leftarrow \text{AE.Enc}(K_e, N, \varepsilon, M)$	if $T \neq T' : \text{return false}$
		return (C, T)	$M \leftarrow \text{AE.Dec}(K_e, N, \varepsilon, C)$
			return $M \neq \perp$

Fig. 15: Games $\mathbf{G}_0, \mathbf{G}_1$ and the adversary $\mathcal{B}$ in the proof of Theorem 2, where $\mathbf{G}_1$ aborts when **bad** occurs while $\mathbf{G}_0$ does not.

(N^*, AD^*)), we show that $\mathcal{B}$ also wins in its $\text{INT-CTXT}_{\text{AE}}^{\mathcal{B}}$ game. By definition of the winning condition of $\mathcal{A}$, we know (C^*, T^*) was not output by any $\text{Enc}(N^*, AD^*, \cdot)$ query and $\text{Ver}(N^*, AD^*, (C^*, T^*)) = 1$. By construction of $\mathcal{B}$, the latter implies that $\hat{T} = T^*$ and $\text{Ver}'(N^*, \varepsilon, C^*) = 1$, so it suffices to show that C^* was not output by any $\text{Enc}'(N^*, \varepsilon, \cdot)$ query made by $\mathcal{B}$. This is true by observing that the committing tag for an $\text{Enc}(N^*, AD^*, \cdot)$ query (i.e., an $\text{Enc}(\hat{N}, \hat{AD}, \cdot)$ query) is T^* (i.e., $\hat{T}$). That is, if C^* was output by an $\text{Enc}'(N^*, \varepsilon, \cdot)$ query, then $\mathcal{A}$ must have queried $\text{Enc}(N^*, AD^*, \cdot)$ that output (C^*, T^*) , which is a contradiction. Therefore, $\mathcal{B}$ also wins. Since there are at most $q_e + q_v$ distinct (N, AD) pairs, we have

$$\frac{1}{q_e + q_v} \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{B}). \quad (6)$$

The proof is concluded by combining equations (5) and (6). $\square$

B.3 Proof of Theorem 3 (CMT Security of HtAE)

Proof. In order to win the $\text{CMT}_{\text{HtAE}}^{\mathcal{A}}$ game (see Figure 5), $\mathcal{A}$ needs to output two tuples (K_1, N_1, AD_1, M_1) , (K_2, N_2, AD_2, M_2) such that they are encrypted to the same ciphertext and $(K_1, N_1, AD_1) \neq (K_2, N_2, AD_2)$. By construction of HtAE, this implies that the hashes of (K_1, N_1, AD_1) and (K_2, N_2, AD_2) collide in the right part (i.e. the committing tag T), which breaks right collision resistance of H . Therefore, we can construct $\mathcal{B}$ as follows: it first invokes $\mathcal{A}$, then extracts (K_1, N_1, AD_1) and (K_2, N_2, AD_2) from $\mathcal{A}$'s output and outputs this pair. $\square$

B.4 Proof of Theorem 4 (CDY\$ Security of HtAE)

Proof. In the $\text{CDY}_{\text{HtAE}, S_{\$}}^{\mathcal{A}}$ game (see Figure 6), $\mathcal{A}$ gets a challenge ciphertext (C, T) output by the context selector $S_{\$}$ and wins by outputting a context tuple (K', N', AD', M') such that $\text{HtAE.Enc}(K', N', AD', M') = (C, T)$. Therefore, if $\mathcal{A}$ wins, then the committing tag T' derived from $(C', T') \leftarrow \text{HtAE.Enc}(K', N', AD', M')$ must be equal to T . This can only happen in two cases: (1) when a random oracle query was made on the exact tuple (K, N, AD) sampled by $S_{\$}$ (together with a random M) to derive the challenge ciphertext (C, T) , or (2) when one of the random oracle queries (not on input (K, N, AD)) outputs a committing tag equal to T . The former occurs with probability at most

$(q_h + 1)/2^\kappa |\mathcal{AD}|$, because by construction of HtAE the randomly sampled K and AD are independent of $\mathcal{A}$'s view; the latter occurs with probability at most $(q_h + 1)/2^t$, because the random oracle H outputs independent random committing tags for distinct inputs. Note that the denominators are $q_h + 1$ because $\mathcal{A}$ made q_h random oracle queries and the challenger made one last H query to compute the encryption of $\mathcal{A}$'s output. Therefore, we have

$$\text{Adv}_{\text{HtAE}}^{\text{cdy}\$}(\mathcal{A}) \leq \frac{1}{|\mathcal{AD}|} \cdot \frac{q_h + 1}{2^\kappa} + \frac{q_h + 1}{2^t}. \quad (7)$$

□

B.5 Proof of Theorem 5 (IND\$-CPA Security of AEaH)

Proof. We define three games $\mathbf{G}_0 \sim \mathbf{G}_2$ as shown in Figure 16.

Game $\boxed{\mathbf{G}_0}, \mathbf{G}_1$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H(X) \wedge X = (K, *)$:	$C \leftarrow \text{AE.Enc}(K, N, \varepsilon, M)$
$T_H[\cdot] \leftarrow \perp$	bad $\leftarrow$ true	$T \xleftarrow{\$} \{0, 1\}^t$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow H(K, N, AD)$
return b'	return $T_H[X]$	return (C, T)

Game $\mathbf{G}_*$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	$(K', *) \leftarrow X$	$C \leftarrow \text{AE.Enc}(K, N, \varepsilon, M)$
$\mathcal{Q}^* \leftarrow \emptyset$	$\mathcal{Q}^* \leftarrow \mathcal{Q}^* \cup \{K'\}$	$T \xleftarrow{\$} \{0, 1\}^t$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	return (C, T)
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	return $T_H[X]$	
return $K \in \mathcal{Q}^*$		

Game $\mathbf{G}_2$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$C \xleftarrow{\$} \{0, 1\}^{\text{AE.clen}(M)}$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	return $T_H[X]$	$T \xleftarrow{\$} \{0, 1\}^t$
return b'		return (C, T)

Fig. 16: Games $\mathbf{G}_0 \sim \mathbf{G}_2$ and $\mathbf{G}_*$ in the proof of Theorem 5, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

Game $\mathbf{G}_0$ is the same as $\text{REAL}_{\text{AEaH}}^{\mathcal{A}}$ (see Figure 4) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \Pr[\text{REAL}_{\text{AEaH}}^{\mathcal{A}} \Rightarrow 1]$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that the committing tag T is now sampled randomly from the set $\{0, 1\}^t$ in the encryption oracle Enc . We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the **bad** event happens, where the adversary $\mathcal{A}$ ever queries H with an input that is prefixed with the key K used in Enc . The only syntactical difference between the two games is the generation of T . Recall that $\mathcal{A}$ is nonce-respecting, i.e., it never repeats a nonce for its encryption queries. Therefore, if **bad** does not occur, then in both games the committing tag T is a random string that is independent of $\mathcal{A}$'s view, and hence games $\mathbf{G}_0$ and $\mathbf{G}_1$ are identical-until-**bad**. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (8)$$

Next, we bound $\Pr[\text{bad}]$ by considering a game $\mathbf{G}_*$ also shown in Figure 16. Here $\mathbf{G}_*$ is just defined to clarify the **bad** event, such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_0/\mathbf{G}_1$. Formally, we define a set $\mathcal{Q}^*$ that records all the keys contained in the random oracle H queries made by $\mathcal{A}$, then let $\mathbf{G}_*$ return 1 if and only if $\mathcal{Q}^*$ contains the key K used in Enc . Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (9)$$

Then, we bound $\Pr[\mathbf{G}_*^A \Rightarrow 1]$ by constructing an adversary $\mathcal{B}_1$ against the IND\$-CPA security of the underlying AE. $\mathcal{B}_1$ starts by initializing the set $\mathcal{Q}^*$ and the table T_H (to an empty set and empty table, respectively), and then it runs $\mathcal{A}$. To answer $\mathcal{A}$'s encryption query on input (N, AD, M) , $\mathcal{B}_1$ calls its own encryption oracle with input (N, ε, M) and gets back C , then samples a random committing tag T from $\{0, 1\}^t$ and returns (C, T) back to $\mathcal{A}$. For $\mathcal{A}$'s random oracle query $X = (K', *)$, $\mathcal{B}_1$ records K' in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates, $\mathcal{B}_1$ ignores its output and queries its own encryption oracle with input $(N^*, \varepsilon, 0^\kappa)$ and gets back C^* , where N^* is a nonce different from those used in previous encryption queries. Then, for each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}_1$ computes $C_i \leftarrow \text{AE.Enc}(K_i, N^*, \varepsilon, 0^\kappa)$ and checks if $C_i = C^*$ holds. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}_1$ guesses that it was in the real world $\text{REAL}_{\text{AE}}^{\mathcal{B}_1}$ and outputs 1; otherwise, it outputs 0, guessing that it was in the ideal world $\text{RAND}_{\text{AE}}^{\mathcal{B}_1}$.

If $\mathcal{B}_1$ was in the real world, then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^A \Rightarrow 1] \leq \Pr[\text{REAL}_{\text{AE}}^{\mathcal{B}_1} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $C_i = C^*$ check. On the other hand, if $\mathcal{B}_1$ was in the ideal world, the probability that a random ciphertext C^* equals $\text{AE.Enc}(K_i, N^*, \varepsilon, 0^\kappa)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^{\text{AE.clen}(\kappa)}$. Since $|\mathcal{Q}^*| \leq q_h$, we have $\Pr[\text{RAND}_{\text{AE}}^{\mathcal{B}_1} \Rightarrow 1] \leq q_h/2^{\text{AE.clen}(\kappa)}$. Recalling that $\text{Adv}_{\text{AE}}^{\text{ind\$-cpa}}(\mathcal{B}_1) = \Pr[\text{REAL}_{\text{AE}}^{\mathcal{B}_1} \Rightarrow 1] - \Pr[\text{RAND}_{\text{AE}}^{\mathcal{B}_1} \Rightarrow 1]$ and $\text{AE.clen}(\kappa) \geq \kappa$, we have

$$\text{Adv}_{\text{AE}}^{\text{ind\$-cpa}}(\mathcal{B}_1) \geq \Pr[\mathbf{G}_*^A \Rightarrow 1] - \frac{q_h}{2^{\text{AE.clen}(\kappa)}} \geq \Pr[\mathbf{G}_*^A \Rightarrow 1] - \frac{q_h}{2^\kappa}, \quad (10)$$

and it follows from equations (8)~(10) that

$$\Pr[\mathbf{G}_0^A \Rightarrow 1] - \Pr[\mathbf{G}_1^A \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{ind\$-cpa}}(\mathcal{B}_1) + \frac{q_h}{2^\kappa}. \quad (11)$$

We continue by making a game hop from $\mathbf{G}_1$ to $\mathbf{G}_2$. As shown in Figure 16, game $\mathbf{G}_2$ is the same as game $\mathbf{G}_1$, except that now in the encryption oracle Enc the ciphertext C is also sampled uniformly at random. We bound the gap between $\mathbf{G}_1$ and $\mathbf{G}_2$ by constructing an adversary $\mathcal{B}_2$ against the IND\$-CPA security of the underlying AE. $\mathcal{B}_2$ just simulates $\mathcal{A}$'s view as the challenger in $\mathbf{G}_1/\mathbf{G}_2$ does, but for $\mathcal{A}$'s encryption query, $\mathcal{B}_2$ calls its own encryption oracle to get C and then finish the rest of the simulation. It is easy to see that $\mathcal{B}_2$ simulates $\mathcal{A}$'s view in $\mathbf{G}_1$ in the real world and $\mathbf{G}_2$ in the ideal world. In the end, adversary $\mathcal{B}_2$ outputs the bit that $\mathcal{A}$ outputs. Therefore, we have

$$\Pr[\mathbf{G}_1^A \Rightarrow 1] - \Pr[\mathbf{G}_2^A \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{ind\$-cpa}}(\mathcal{B}_2). \quad (12)$$

Finally, we observe that game $\mathbf{G}_2$ is identical to $\text{RAND}_{\text{AEaH}}^A$ (see Figure 4) equipped with a random oracle H . This is because, in game $\mathbf{G}_2$, for each Enc query with a distinct nonce, its output is an independent uniform random string sampled from $\{0, 1\}^{\text{HtAE.clen}(|M|)} = \{0, 1\}^{\text{AE.clen}(|M|)+t}$.

The proof is concluded by combining equations (11) and (12). $\square$

B.6 Proof of Theorem 6 (INT-CTXT Security of AEaH)

Proof. We define two games $\mathbf{G}_0$ and $\mathbf{G}_1$ as shown in Figure 17.

Game $\mathbf{G}_0$ is the same as $\text{INT-CTXT}_{\text{AEaH}}^A$ (see Figure 4) equipped with a random oracle H . (Note that the T_T operations can be ignored in $\mathbf{G}_0$.) Therefore, we have $\Pr[\mathbf{G}_0^A \Rightarrow 1] = \text{Adv}_{\text{AEaH}}^{\text{int-ctxt}}(\mathcal{A})$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that the committing tag T is now lazily sampled from the table T_T in both oracles, where the table is indexed by the queried (N, AD) pair. We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the bad event happens, which occurs if the adversary $\mathcal{A}$ ever queries H with an input that is prefixed with the key K . The only syntactical difference between the two games are in the generation of T . If bad does not occur, in both games the committing tag T is either randomly sampled for a new (N, AD) pair, or gets the same previously sampled value if (N, AD) is repeated; hence the two games are indeed identical-until-bad. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^A \Rightarrow 1] - \Pr[\mathbf{G}_1^A \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (13)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by considering a game $\mathbf{G}_*$ also shown in Figure 17. $\mathbf{G}_*$ is defined such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers bad in $\mathbf{G}_0/\mathbf{G}_1$. Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^A \Rightarrow 1]. \quad (14)$$

Game $\mathbf{G}_0, \mathbf{G}_1$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H(X)$	$C \leftarrow \text{AE.Enc}(K, N, \varepsilon, M)$	if $T_T[(N, AD)] = \perp :$
$T_H[\cdot] \leftarrow \perp$	$\wedge X = (K, *) :$	if $T_T[(N, AD)] = \perp :$	$T_T[(N, AD)] \xleftarrow{\$} \{0, 1\}^t$
$T_T[\cdot] \leftarrow \perp$	bad $\leftarrow \text{true}$	$T_T[(N, AD)] \xleftarrow{\$} \{0, 1\}^t$	$T' \leftarrow T_T[(N, AD)]$
$\mathcal{Q} \leftarrow \emptyset$	if $T_H[X] = \perp :$	$T \leftarrow T_T[(N, AD)]$	$T' \leftarrow H(K, N, AD)$
$\text{win} \leftarrow 0$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow H(K, N, AD)$	if $T \neq T' : \text{return false}$
$\mathcal{A}^{\text{H,Enc,Ver}}$	return $T_H[X]$	$\mathcal{Q} \leftarrow \{(N, AD, (C, T))\}$	$M \leftarrow \text{AE.Dec}(K, N, \varepsilon, C)$
return win		return (C, T)	if $M \neq \perp \wedge (N, AD, (C, T)) \notin \mathcal{Q} :$
			$\text{win} \leftarrow 1$
			return $M \neq \perp$

Game $\mathbf{G}_*$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	$(K', *) \leftarrow X$	$C \leftarrow \text{AE.Enc}(K, N, \varepsilon, M)$	if $T_T[(N, AD)] = \perp :$
$T_H[\cdot] \leftarrow \perp$	$\mathcal{Q}^* \leftarrow \{K'\}$	if $T_T[(N, AD)] = \perp :$	$T_T[(N, AD)] \xleftarrow{\$} \{0, 1\}^t$
$T_T[\cdot] \leftarrow \perp$	if $T_H[X] = \perp :$	$T_T[(N, AD)] \xleftarrow{\$} \{0, 1\}^t$	$T' \leftarrow T_T[(N, AD)]$
$\mathcal{Q}^* \leftarrow \emptyset$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow T_T[(N, AD)]$	if $T \neq T' : \text{return false}$
$\mathcal{A}^{\text{H,Enc,Ver}}$	return $T_H[X]$	return (C, T)	$M \leftarrow \text{AE.Dec}(K, N, \varepsilon, C)$
return $K \in \mathcal{Q}^*$			return $M \neq \perp$

Fig. 17: Games $\mathbf{G}_0$, $\mathbf{G}_1$ and $\mathbf{G}_*$ in the proof of Theorem 6, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

Then, we bound $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{B}_1$ against the INT-CTXT security of the underlying AE. $\mathcal{B}_1$ starts by initializing the set $\mathcal{Q}^*$ and tables T_H, T_T (to an empty set and empty tables, respectively), and then it runs $\mathcal{A}$. To answer $\mathcal{A}$'s encryption query on input (N, AD, M) , $\mathcal{B}_1$ calls its own encryption oracle Enc' with input (N, ε, M) and gets back C , then lazily samples T with the help of table T_T . At the end, it returns (C, T) back to $\mathcal{A}$. To answer $\mathcal{A}$'s verification query on input $(N, AD, (C, T))$, $\mathcal{B}_1$ lazily samples T' with the help of the table T_T and checks if T' equals T . If not, $\mathcal{B}_1$ returns **false**, otherwise it calls its own verification oracle Ver' with input (N, ε, C) and forwards the answer back to $\mathcal{A}$. For $\mathcal{A}$'s random oracle query $X = (K', *)$, $\mathcal{B}_1$ records K' in $\mathcal{Q}^*$ and answers the query via lazy sampling using the table T_H . After $\mathcal{A}$ terminates, $\mathcal{B}_1$ picks a nonce N^* that is different from those used by $\mathcal{A}$ in encryption oracle queries; then, for each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}$ computes $C_i \leftarrow \text{AE.Enc}(K_i, N^*, \varepsilon, 0^\kappa)$ and queries its own verification oracle with (N^*, ε, C_i) .

The above adversary $\mathcal{B}_1$ correctly simulates the $\mathcal{A}$'s view in $\mathbf{G}_*$, and we claim that, if $\mathcal{A}$ wins in $\mathbf{G}_*$, then $\mathcal{B}_1$ wins in its $\text{INT-CTXT}_{\text{AE}}^{\mathcal{B}_1}$ game. If $\mathcal{A}$ wins, then the AE key K (sampled by $\mathcal{B}_1$'s challenger) must be included in the set $\mathcal{Q}^*$. Since $\mathcal{B}_1$ enumerated all keys in $\mathcal{Q}^*$ to compute ciphertexts with a new nonce, the ciphertext derived from the correct key K must be a valid forgery for $\mathcal{B}_1$ to win in its game. Therefore, we have

$$\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{B}_1). \quad (15)$$

and it follows from equations (13)~(15) that

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{B}_1). \quad (16)$$

Finally, we bound $\mathcal{A}$'s winning probability $\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1]$ by constructing another INT-CTXT adversary $\mathcal{B}_2$ against the underlying AE. Adversary $\mathcal{B}_2$ is the same as adversary $\mathcal{B}_1$, except that $\mathcal{B}_2$ terminates immediately after $\mathcal{A}$ terminates, i.e., $\mathcal{B}_2$ does not record keys in $\mathcal{Q}^*$ or make the final verification queries after $\mathcal{A}$ terminates. Clearly, $\mathcal{B}_2$ correctly simulates $\mathcal{A}$'s view in $\mathbf{G}_1$. Now, suppose $\mathcal{A}$ wins by querying $\text{Ver}(N^*, AD^*, (C^*, T^*))$. By definition of the winning condition of $\mathcal{A}$, we know (C^*, T^*) was not output by any $\text{Enc}(N^*, AD^*, \cdot)$ query and $\text{Ver}(N^*, AD^*, (C^*, T^*)) = 1$. By construction of $\mathcal{B}_2$, the latter implies that $T^* = T_T[(N^*, AD^*)]$ and $\text{Ver}'(N^*, \varepsilon, C^*) = 1$. There are two cases. If C^* was not output by any $\text{Enc}'(N^*, \varepsilon, \cdot)$ query made by $\mathcal{B}_2$, then $\mathcal{B}_2$ wins in its $\text{INT-CTXT}_{\text{AE}}^{\mathcal{B}_2}$ game. Otherwise, $\mathcal{A}$ must have queried $\text{Enc}(N^*, \widehat{AD}, \cdot)$ with some $\widehat{AD} \neq AD^*$ that output $(C^*, \widehat{T})$, because any $\text{Enc}(N^*, AD^*, \cdot)$ query will produce the committing tag T^* . Then, since $\mathcal{B}_2$ is nonce-respecting, it has never made an $\text{Enc}(N^*, AD^*, \cdot)$

query to learn $T_T[(N^*, AD^*)]$. Therefore, $\mathcal{B}_2$ can only guess T^* when making verification queries. Since it can make q_v verification queries and T^* is randomly sampled from $\{0, 1\}^t$, the probability that $\mathcal{B}_2$ wins is at most $q_v/2^t$. Therefore, we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{int-ctxt}}(\mathcal{B}_2) + \frac{q_v}{2^t}. \quad (17)$$

The proof is concluded by combining equations (16) and (17). $\square$

B.7 Proof of Theorem 7 (CMT Security of AEaH)

Proof. In order to win the $\text{CMT}_{\text{AEaH}}^{\mathcal{A}}$ game (see Figure 5), $\mathcal{A}$ needs to output two tuples (K_1, N_1, AD_1, M_1) , (K_2, N_2, AD_2, M_2) such that they are encrypted to the same ciphertext and $(K_1, N_1, AD_1) \neq (K_2, N_2, AD_2)$. By construction of AEaH, this implies that the hashes of (K_1, N_1, AD_1) and (K_2, N_2, AD_2) collide, which breaks the collision resistance of H . Therefore, we can construct $\mathcal{B}$ as follows: it first invokes $\mathcal{A}$, then extracts (K_1, N_1, AD_1) and (K_2, N_2, AD_2) from $\mathcal{A}$'s output and outputs this pair. $\square$

B.8 Proof of Theorem 8 (CDY\$ Security of AEaH)

Proof. We define two games $\mathbf{G}_0$ and $\mathbf{G}_1$ as shown in Figure 18.

Game $\mathbf{G}_0, \mathbf{G}_1$	Oracle $H(X)$
$T_H[\cdot] \leftarrow \perp$; $K \xleftarrow{\$} \{0, 1\}^\kappa$; $N \xleftarrow{\$} \mathcal{N}$; $AD \xleftarrow{\$} \mathcal{AD}$; $M \xleftarrow{\$} \mathcal{M}$ $C \leftarrow \text{AE.Enc}(K, N, \varepsilon, M)$; $T \xleftarrow{\$} \{0, 1\}^t$ $T_H[(K, N, AD)] \leftarrow T$ $(K', N', AD', M') \xleftarrow{\$} \mathcal{A}^H(C, T)$ $C' \leftarrow \text{AE.Enc}(K', N', \varepsilon, M')$; $T' \leftarrow H(K', N', AD')$ return $(C', T') = (C, T)$	if $X = (K, *)$: bad $\leftarrow$ true if $T_H[X] = \perp$: $T_H[X] \xleftarrow{\$} \{0, 1\}^t$ return $T_H[X]$
Game $\mathbf{G}_*$	Oracle $H(X)$
$T_H[\cdot] \leftarrow \perp$; $K \xleftarrow{\$} \{0, 1\}^\kappa$; $N \xleftarrow{\$} \mathcal{N}$; $AD \xleftarrow{\$} \mathcal{AD}$; $M \xleftarrow{\$} \mathcal{M}$ $C \leftarrow \text{AE.Enc}(K, N, \varepsilon, M)$; $T \xleftarrow{\$} \{0, 1\}^t$ $\mathcal{Q}^* \leftarrow \emptyset$ $(K', N', AD', M') \xleftarrow{\$} \mathcal{A}^H(C, T)$ $H(K', N', AD')$ return $K \in \mathcal{Q}^*$	$(K^*, *) \leftarrow X$ $\mathcal{Q}^* \leftarrow \mathcal{Q}^* \cup \{K^*\}$ if $T_H[X] = \perp$: $T_H[X] \xleftarrow{\$} \{0, 1\}^t$ return $T_H[X]$

Fig. 18: Games $\mathbf{G}_0, \mathbf{G}_1$ and $\mathbf{G}_*$ in the proof of Theorem 8, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

Game $\mathbf{G}_0$ is the same as $\text{CDY}_{\text{AEaH}, S_S}^{\mathcal{A}}$ (see Figure 6) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{AEaH}, S_S}^{\text{cdy}}(\mathcal{A}) = \text{Adv}_{\text{AEaH}}^{\text{cdy}\$}(\mathcal{A})$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that the committing tag T , calculated by the challenger, is now independent of queries to the random oracle H . We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the **bad** event happens, where the random oracle H is ever queried with an input that is prefixed with the key K sampled by the challenger. This is because, if **bad** does not occur, then T in $\mathbf{G}_0$ is also independent of random oracle queries made by $\mathcal{A}$ and by the challenger (to compute T'). Therefore, by the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (18)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by constructing a game $\mathbf{G}_*$ also shown in Figure 18. $\mathbf{G}_*$ is defined such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_0/\mathbf{G}_1$ when (1)

$\mathcal{A}$ queried the random oracle H with an input prefixed with K or (2) $\mathcal{A}$ output (K', N', AD', M') with $K' = K$ and then $H(K', N', AD')$ was queried by the challenger. Therefore, it holds that

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (19)$$

Then, we bound $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{B}$ against the one-time IND\$-CPA security of the underlying AE. $\mathcal{B}$ starts by initializing the set $\mathcal{Q}^*$ and the table T_H (to an empty set and empty table, respectively). Then it samples the tuple (N, AD, M) uniformly at random, and calls its own encryption oracle with input (N, ε, M) to derive C . It then samples T randomly and runs $\mathcal{A}$ with input (C, T) . For $\mathcal{A}$'s each random oracle query $X = (K^*, *)$, $\mathcal{B}$ records K^* in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates and outputs (K', N', AD', M') , $\mathcal{B}$ “queries” the random oracle H with the input (K', N', AD') and does the following. For each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}$ computes $C_i \leftarrow \text{AE.Enc}(K_i, N, \varepsilon, M)$ and checks if $C_i = C$. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}$ guesses that it was in the real world $\text{REAL}_{\text{AE}}^{\mathcal{B}}$ and outputs 1; otherwise, it outputs 0, guessing that it was in the ideal world $\text{RAND}_{\text{AE}}^{\mathcal{B}}$.

If $\mathcal{B}$ was in the real world, then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{REAL}_{\text{AE}}^{\mathcal{B}} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $C_i = C$ check. On the other hand, if $\mathcal{B}$ was in the ideal world, the probability that a random ciphertext C equals $\text{AE.Enc}(K_i, N, \varepsilon, M)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^{\text{AE.clen}(|M|)}$. Since $|\mathcal{Q}^*| \leq q_h + 1$, we have $\Pr[\text{RAND}_{\text{AE}}^{\mathcal{B}} \Rightarrow 1] \leq (q_h + 1)/2^{\text{AE.clen}(|M|)}$. Recall that $\text{Adv}_{\text{AE}}^{\text{ot-ind\$-cpa}}(\mathcal{B}) = \Pr[\text{REAL}_{\text{AE}}^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RAND}_{\text{AE}}^{\mathcal{B}} \Rightarrow 1]$ and $\text{AE.clen}(|M|) \geq \tau$, we have

$$\text{Adv}_{\text{AE}}^{\text{ot-ind\$-cpa}}(\mathcal{B}) \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h + 1}{2^\tau}, \quad (20)$$

and it follows from equations (18)~(20) that

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{ot-ind\$-cpa}}(\mathcal{B}) + \frac{q_h + 1}{2^\tau}. \quad (21)$$

Finally, we observe that in game $\mathbf{G}_1$ the committing tag T in the challenge ciphertext (C, T) is independent of random oracle queries and hence independent of T' , so in order for $\mathcal{A}$ to win one of the random oracle queries must output a committing tag equal to the independent random T . This probability is bounded by $(q_h + 1)/2^t$. Therefore, we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \frac{q_h + 1}{2^t}. \quad (22)$$

The proof is concluded by combining equations (21) and (22). $\square$

B.9 Proof of Theorem 9 (IND\$-CPA Security of EtH)

Proof. We define three games $\mathbf{G}_0 \sim \mathbf{G}_2$ as shown in Figure 19.

Game $\mathbf{G}_0$ is the same as $\text{REAL}_{\text{EtH}}^{\mathcal{A}}$ (see Figure 4) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \Pr[\text{REAL}_{\text{EtH}}^{\mathcal{A}} \Rightarrow 1]$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that the committing tag T is now sampled randomly from the set $\{0, 1\}^t$ in the encryption oracle Enc . We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the **bad** event happens, where the adversary $\mathcal{A}$ ever queries H with an input that is prefixed with the key K used in Enc . The only syntactical difference between the two games is the generation of T . Recall that $\mathcal{A}$ is nonce-respecting, i.e., it never repeats a nonce for its encryption queries. Therefore, if **bad** does not occur, then in both games the committing tag T is a random string that is independent of $\mathcal{A}$'s view, and hence games $\mathbf{G}_0$ and $\mathbf{G}_1$ are identical-until-**bad**. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (23)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by considering a game $\mathbf{G}_*$ also shown in Figure 19. Here $\mathbf{G}_*$ is just defined to clarify the **bad** event, such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_0/\mathbf{G}_1$. Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (24)$$

Game $\boxed{\mathbf{G}_0}, \mathbf{G}_1$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H(X) \wedge X = (K, *)$:	$C \leftarrow \mathbf{E}.\text{Enc}(K, N, M)$
$T_H[\cdot] \leftarrow \perp$	bad $\leftarrow \mathbf{true}$	$T \xleftarrow{\$} \{0, 1\}^t$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow H(K, N, AD, C)$
return b'	return $T_H[X]$	return (C, T)

Game $\mathbf{G}_*$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	$(K', *) \leftarrow X$	$C \leftarrow \mathbf{E}.\text{Enc}(K, N, M)$
$\mathcal{Q}^* \leftarrow \emptyset$	$\mathcal{Q}^* \leftarrow \cup \{K'\}$	$T \xleftarrow{\$} \{0, 1\}^t$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	return (C, T)
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	return $T_H[X]$	
return $K \in \mathcal{Q}^*$		

Game $\mathbf{G}_2$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$C \xleftarrow{\$} \{0, 1\}^{\text{E.clen}(M)}$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	return $T_H[X]$	$T \xleftarrow{\$} \{0, 1\}^t$
return b'		return (C, T)

Fig. 19: Games $\mathbf{G}_0 \sim \mathbf{G}_2$ and $\mathbf{G}_*$ in the proof of Theorem 9, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

Then, we bound $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{B}_1$ against the IND\\$-CPA security of the underlying $\mathbf{E}$. $\mathcal{B}_1$ starts by initializing the set $\mathcal{Q}^*$ and the table T_H (to an empty set and empty table, respectively), and then it runs $\mathcal{A}$. To answer $\mathcal{A}$'s encryption query on input (N, AD, M) , $\mathcal{B}_1$ calls its own encryption oracle with input (N, M) and gets back C , then samples a random committing tag T from $\{0, 1\}^t$ and returns (C, T) back to $\mathcal{A}$. For $\mathcal{A}$'s random oracle query $X = (K', *)$, $\mathcal{B}_1$ records K' in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates, $\mathcal{B}_1$ ignores its output and queries its own encryption oracle with input $(N^*, 0^\kappa)$ and gets back C^* , where N^* is a nonce different from those used in previous encryption queries. Then, for each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}_1$ computes $C_i \leftarrow \mathbf{E}.\text{Enc}(K_i, N^*, 0^\kappa)$ and checks if $C_i = C^*$ holds. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}_1$ guesses that it was in the real world $\text{REAL}_E^{\mathcal{B}_1}$ and outputs 1; otherwise, it outputs 0, guessing that it was in the ideal world $\text{RAND}_E^{\mathcal{B}_1}$.

If $\mathcal{B}_1$ was in the real world, then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{REAL}_E^{\mathcal{B}_1} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $C_i = C^*$ check. On the other hand, if $\mathcal{B}_1$ was in the ideal world, the probability that a random ciphertext C^* equals $\mathbf{E}.\text{Enc}(K_i, N^*, 0^\kappa)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^{\text{E.clen}(\kappa)}$. Since $|\mathcal{Q}^*| \leq q_h$, we have $\Pr[\text{RAND}_E^{\mathcal{B}_1} \Rightarrow 1] \leq q_h/2^{\text{E.clen}(\kappa)}$. Then, recall that $\text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}_1) = \Pr[\text{REAL}_E^{\mathcal{B}_1} \Rightarrow 1] - \Pr[\text{RAND}_E^{\mathcal{B}_1} \Rightarrow 1]$ and $\text{E.clen}(\kappa) \geq \kappa$, we have

$$\text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}_1) \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h}{2^{\text{E.clen}(\kappa)}} \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h}{2^\kappa}, \quad (25)$$

and it follows from equations (23)~(25) that

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}_1) + \frac{q_h}{2^\kappa}. \quad (26)$$

We continue by making a game hop from $\mathbf{G}_1$ to $\mathbf{G}_2$. As shown in Figure 19, game $\mathbf{G}_2$ is the same as game $\mathbf{G}_1$, except that now in the encryption oracle Enc the ciphertext C is also sampled uniformly at random. We bound the gap between $\mathbf{G}_1$ and $\mathbf{G}_2$ by constructing an adversary $\mathcal{B}_2$ against the IND\\$-CPA security of the underlying $\mathbf{E}$. $\mathcal{B}_2$ just simulates $\mathcal{A}$'s view as the challenger in $\mathbf{G}_1/\mathbf{G}_2$ does, but for $\mathcal{A}$'s encryption query, $\mathcal{B}_2$ calls its own encryption oracle to get C and then finish the rest of the simulation. It is easy to see that $\mathcal{B}_2$ simulates $\mathcal{A}$'s view in $\mathbf{G}_1$ in the real world and $\mathbf{G}_2$ in the ideal world. In the end, adversary $\mathcal{B}_2$ outputs the bit that $\mathcal{A}$ outputs. Therefore, we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}_2). \quad (27)$$

Finally, we observe that game $\mathbf{G}_2$ is identical to $\text{RAND}_{\text{EtH}}^A$ (see Figure 4) equipped with a random oracle H . This is because, in game $\mathbf{G}_2$, for each Enc query with a distinct nonce, its output is an independent uniform random string sampled from $\{0, 1\}^{\text{EtH.clen}(|M|)} = \{0, 1\}^{\text{E.clen}(|M|)+t}$.

The proof is concluded by combining equations (26) and (27). $\square$

B.10 Proof of Theorem 10 (INT-CTXT Security of EtH)

Proof. We define three games $\mathbf{G}_0 \sim \mathbf{G}_2$ as shown in Figure 20.

Game $\mathbf{G}_0$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \mathcal{K}$	if $\mathcal{A}$ queries $H(X)$	$C \leftarrow \text{E.Enc}(K, N, M)$	$T' \leftarrow H(K, N, AD, C)$
$T_H[\cdot] \leftarrow \perp$	$\wedge X = (K, *) :$	$T \leftarrow H(K, N, AD, C)$	if $T' \neq T : \text{return false}$
$\mathcal{Q} \leftarrow \emptyset$	bad $\leftarrow \text{true}$	$\mathcal{Q} \leftarrow \{(N, AD, (C, T))\}$	$M \leftarrow \text{E.Dec}(K, N, C)$
win $\leftarrow 0$	if $T_H[X] = \perp :$	return (C, T)	if $M \neq \perp \wedge (N, AD, (C, T)) \notin \mathcal{Q} :$
$\mathcal{A}^{\text{H,Enc,Ver}}$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$		win $\leftarrow 1$
return win	return $T_H[X]$		return $M \neq \perp$

Game $\mathbf{G}_1, \mathbf{G}_2$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \mathcal{K}$	if $\mathcal{A}$ queries $H(X)$	$C \leftarrow \text{E.Enc}(K, N, M)$	if $T_T[(N, AD, C)] = \perp :$
$T_H[\cdot] \leftarrow \perp$	$\wedge X = (K, *) :$	if $T_T[(N, AD, C)] = \perp :$	$T_T[(N, AD, C)] \xleftarrow{\$} \{0, 1\}^t$
$T_T[\cdot] \leftarrow \perp$	bad $\leftarrow \text{true}$	$T_T[(N, AD, C)] \xleftarrow{\$} \{0, 1\}^t$	$T' \leftarrow T_T[(N, AD, C)]$
$\mathcal{Q} \leftarrow \emptyset$	if $T_H[X] = \perp :$	$T \leftarrow T_T[(N, AD, C)]$	$T' \leftarrow H(K, N, AD, C)$
win $\leftarrow 0$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow H(K, N, AD, C)$	if $T' \neq T : \text{return false}$
$\mathcal{A}^{\text{H,Enc,Ver}}$	return $T_H[X]$	$\mathcal{Q} \leftarrow \{(N, AD, (C, T))\}$	if $(N, AD, (C, T)) \notin \mathcal{Q} :$
return win		return (C, T)	win $\leftarrow 1$
			return true

Game $\mathbf{G}_*$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^{\kappa}$	$(K', *) \leftarrow X$	$C \leftarrow \text{E.Enc}(K, N, M)$	if $T_T[(N, AD, C)] = \perp :$
$T_H[\cdot] \leftarrow \perp$	$\mathcal{Q}^* \leftarrow \{K'\}$	if $T_T[(N, AD, C)] = \perp :$	$T_T[(N, AD, C)] \xleftarrow{\$} \{0, 1\}^t$
$T_T[\cdot] \leftarrow \perp$	if $T_H[X] = \perp :$	$T_T[(N, AD, C)] \xleftarrow{\$} \{0, 1\}^t$	$T' \leftarrow T_T[(N, AD, C)]$
$\mathcal{Q}^* \leftarrow \emptyset$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow T_T[(N, AD, C)]$	if $T' \neq T : \text{return false}$
$\mathcal{A}^{\text{H,Enc,Ver}}$	return $T_H[X]$	return (C, T)	return true
return $K \in \mathcal{Q}^*$			

Fig. 20: Games $\mathbf{G}_0 \sim \mathbf{G}_2$ and $\mathbf{G}_*$ in the proof of Theorem 10, where $\mathbf{G}_1$ contains boxed content while $\mathbf{G}_2$ does not.

Game $\mathbf{G}_0$ is the same as $\text{INT-CTXT}_{\text{EtH}}^A$ (see Figure 4) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^A \Rightarrow 1] = \text{Adv}_{\text{EtH}}^{\text{int-ctxt}}(\mathcal{A})$.

Game $\mathbf{G}_1$ is the same as $\mathbf{G}_0$, except that the verification oracle does not check if the ciphertext C can be successfully decrypted (note that the T_T operations can be ignored in $\mathbf{G}_1$). As a result, the winning condition in $\mathbf{G}_1$ is relaxed, i.e., if $\mathcal{A}$ wins in $\mathbf{G}_0$ then it must also win in $\mathbf{G}_1$. To see this, suppose $\mathcal{A}$ wins in $\mathbf{G}_0$ by querying $(N^*, AD^*, (C^*, T^*))$ and before this query $\mathcal{A}$ did not already win in $\mathbf{G}_1$. In this case, $\mathcal{A}$'s views in $\mathbf{G}_0$ and $\mathbf{G}_1$ are identical. This is because before that winning query each verification query with a valid T must satisfy $(N, AD, (C, T)) \in \mathcal{Q}$ (otherwise $\mathcal{A}$ wins in $\mathbf{G}_1$), which implies that $M \neq \perp$ and hence the verification oracle returns **true** in both games. Then, by the winning condition in $\mathbf{G}_1$, the above query also allows $\mathcal{A}$ to win in $\mathbf{G}_1$. Therefore, we have

$$\Pr[\mathbf{G}_0^A \Rightarrow 1] \leq \Pr[\mathbf{G}_1^A \Rightarrow 1]. \quad (28)$$

Game $\mathbf{G}_2$ is the same as $\mathbf{G}_1$, except that the committing tag is now lazily sampled from the table T_T in both oracles, where the table is indexed by the (N, AD, C) triple. We claim that $\mathbf{G}_1$ and $\mathbf{G}_2$

behave identically unless the **bad** event happens, which occurs if the adversary $\mathcal{A}$ ever queries H with an input that is prefixed with the key K . The only syntactical difference between the two games are in the generation of T . If **bad** does not occur, in both games the committing tag T is either randomly sampled for a new (N, AD, C) triple or equal to the same previously sampled value if (N, AD, C) is not new. Hence the two games are indeed identical-until-**bad**. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\mathbf{bad}]. \quad (29)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\mathbf{bad}]$ by considering a game $\mathbf{G}_*$ also shown in Figure 20. $\mathbf{G}_*$ is defined such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_1/\mathbf{G}_2$. Therefore, we have

$$\Pr[\mathbf{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (30)$$

Then, we bound $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{B}$ against the IND\$-CPA security of the underlying $\mathbf{E}$. $\mathcal{B}$ starts by initializing the set $\mathcal{Q}^*$ and tables T_H, T_T (to an empty set and empty tables, respectively), and then it runs $\mathcal{A}$. To answer $\mathcal{A}$'s encryption query on input (N, AD, M) , $\mathcal{B}$ calls its own encryption oracle Enc' with input (N, M) and gets back C , then lazily samples T with the help of table T_T . At the end, it returns (C, T) to $\mathcal{A}$. To answer $\mathcal{A}$'s verification query on input $(N, AD, (C, T))$, $\mathcal{B}$ lazily samples T' with the help of the table T_T and checks if $T' = T$. If not, $\mathcal{B}$ returns **false**, otherwise it returns **true**. For $\mathcal{A}$'s random oracle query $X = (K', *)$, $\mathcal{B}$ records K' in $\mathcal{Q}^*$ and answers the query via lazy sampling of the table T_H . After $\mathcal{A}$ terminates, $\mathcal{B}$ picks a nonce N^* that is different from those used previous encryption queries and then calls its own encryption oracle with input $(N^*, 0^\kappa)$ to get back C^* . Then, for each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}$ computes $C_i \leftarrow \mathbf{E}.\text{Enc}(K_i, N^*, 0^\kappa)$ and checks if $C_i = C^*$ holds. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}$ guesses that it was in the real world $\text{REAL}_E^{\mathcal{B}}$ and outputs 1; otherwise, it outputs 0, guessing that it was in the ideal world $\text{RAND}_E^{\mathcal{B}}$.

If $\mathcal{B}$ was in the real world, then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{REAL}_E^{\mathcal{B}} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $C_i = C^*$ check. On the other hand, if $\mathcal{B}$ was in the ideal world, the probability that a random ciphertext C^* equals $\mathbf{E}.\text{Enc}(K_i, N^*, 0^\kappa)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^{\text{E.clen}(\kappa)}$. Since $|\mathcal{Q}^*| \leq q_h$, we have $\Pr[\text{RAND}_E^{\mathcal{B}} \Rightarrow 1] \leq q_h/2^{\text{E.clen}(\kappa)}$. Recalling that $\text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}) = \Pr[\text{REAL}_E^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RAND}_E^{\mathcal{B}} \Rightarrow 1]$ and $\text{E.clen}(\kappa) \geq \kappa$, we have

$$\text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}) \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h}{2^{\text{E.clen}(\kappa)}} \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h}{2^\kappa}, \quad (31)$$

and it follows from equations (29)~(31) that

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}) + \frac{q_h}{2^\kappa}. \quad (32)$$

Finally, it is left to bound $\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1]$. Suppose $\mathcal{A}$ wins by querying $\text{Ver}(N^*, AD^*, (C^*, T^*))$, then we know (C^*, T^*) was not output by any $\text{Enc}(N^*, AD^*, \cdot)$ query and $\text{Ver}(N^*, AD^*, (C^*, T^*)) = 1$. The latter implies that $T^* = T_T[(N^*, AD^*, C^*)]$. If C^* was output by an $\text{Enc}(N^*, AD^*, \cdot)$ query, then $T^* = T_T[(N^*, AD^*, C^*)]$ is the output committing tag and this contradicts with the above winning condition. Therefore, $\mathcal{A}$ has never learned $T_T[(N^*, AD^*, C^*)]$ by encryption queries and hence it can only guess T^* when making verification queries. Since $\mathcal{A}$ can make q_v verification queries and T^* is randomly sampled from $\{0, 1\}^t$, the probability that $\mathcal{A}$ wins is at most $q_v/2^t$. Therefore, we have

$$\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \frac{q_v}{2^t}. \quad (33)$$

The proof is concluded by combining equations (28), (32) and (33). $\square$

B.11 Proof of Theorem 11 (CMT Security of EtH)

Proof. In order to win the $\text{CMT}_{\text{EtH}}^{\mathcal{A}}$ game (see Figure 5), $\mathcal{A}$ needs to output two tuples (K_1, N_1, AD_1, M_1) , (K_2, N_2, AD_2, M_2) such that they are encrypted to the same ciphertext (C, T) and $(K_1, N_1, AD_1) \neq (K_2, N_2, AD_2)$. By construction of EtH, this implies that the hashes of (K_1, N_1, AD_1, C) and (K_2, N_2, AD_2, C) collide. Therefore, we can construct $\mathcal{B}$ as follows: it first invokes $\mathcal{A}$, then extracts (K_1, N_1, AD_1) and (K_2, N_2, AD_2) from $\mathcal{A}$'s output. It continues by computing C and outputting the tuple pair $((K_1, N_1, AD_1, C), (K_2, N_2, AD_2, C))$. $\square$

B.12 Proof of Theorem 12 (CDY\$ Security of EtH)

Proof. We define two games $\mathbf{G}_0$ and $\mathbf{G}_1$ as shown in Figure 21.

Game $\boxed{\mathbf{G}_0}, \mathbf{G}_1$	Oracle $H(X)$
$T_H[\cdot] \leftarrow \perp$; $K \xleftarrow{\$} \{0,1\}^\kappa$; $N \xleftarrow{\$} \mathcal{N}$; $AD \xleftarrow{\$} \mathcal{AD}$; $M \xleftarrow{\$} \mathcal{M}$ $C \leftarrow \mathbf{E}.\text{Enc}(K, N, M)$; $T \xleftarrow{\$} \{0,1\}^t$ $\boxed{T_H[(K, N, AD, C)] \leftarrow T}$ $(K', N', AD', M') \xleftarrow{\$} \mathcal{A}^H(C, T)$ $C' \leftarrow \mathbf{E}.\text{Enc}(K', N', M')$; $T' \leftarrow H(K', N', AD', C')$ return $(C', T') = (C, T)$	if $X = (K, *)$: bad $\leftarrow$ true if $T_H[X] = \perp$: $T_H[X] \xleftarrow{\$} \{0,1\}^t$ return $T_H[X]$
Game $\mathbf{G}_*$	Oracle $H(X)$
$T_H[\cdot] \leftarrow \perp$; $K \xleftarrow{\$} \{0,1\}^\kappa$; $N \xleftarrow{\$} \mathcal{N}$; $AD \xleftarrow{\$} \mathcal{AD}$; $M \xleftarrow{\$} \mathcal{M}$ $C \leftarrow \mathbf{E}.\text{Enc}(K, N, M)$; $T \xleftarrow{\$} \{0,1\}^t$ $\mathcal{Q}^* \leftarrow \emptyset$ $(K', N', AD', M') \xleftarrow{\$} \mathcal{A}^H(C, T)$ $C' \leftarrow \mathbf{E}.\text{Enc}(K', N', M')$; $H(K', N', AD', C')$ return $K \in \mathcal{Q}^*$	$(K^*, *) \leftarrow X$ $\mathcal{Q}^* \leftarrow \mathcal{Q}^* \cup \{K^*\}$ if $T_H[X] = \perp$: $T_H[X] \xleftarrow{\$} \{0,1\}^t$ return $T_H[X]$

Fig. 21: Games $\mathbf{G}_0, \mathbf{G}_1$ and $\mathbf{G}_*$ in the proof of Theorem 12, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

Game $\mathbf{G}_0$ is the same as $\text{CDY}_{\text{EtH}, S_g}^A$ (see Figure 6) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^A \Rightarrow 1] = \text{Adv}_{\text{EtH}, S_g}^{\text{cdy}}(\mathcal{A}) = \text{Adv}_{\text{EtH}}^{\text{cdy}\$}(\mathcal{A})$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that the committing tag T , calculated by the challenger, is now independent of queries to the random oracle H . We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the **bad** event happens, where the random oracle H is ever queried with an input that is prefixed with the key K sampled by the challenger. This is because, if **bad** does not occur, then T in $\mathbf{G}_0$ is also independent of random oracle queries made by $\mathcal{A}$ and by the challenger (to compute T'). Therefore, by the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^A \Rightarrow 1] - \Pr[\mathbf{G}_1^A \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (34)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by constructing a game $\mathbf{G}_*$ also shown in Figure 21. $\mathbf{G}_*$ is defined such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_0/\mathbf{G}_1$ when (1) $\mathcal{A}$ queried the random oracle H with an input prefixed with K or (2) $\mathcal{A}$ output (K', N', AD', M') with $K' = K$ and then $H(K', N', AD', C')$ was queried by the challenger. Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^A \Rightarrow 1]. \quad (35)$$

Then, we bound $\Pr[\mathbf{G}_*^A \Rightarrow 1]$ by constructing an adversary $\mathcal{B}$ against the IND\$-CPA security of the underlying encryption scheme $\mathbf{E}$. $\mathcal{B}$ starts by initializing the set $\mathcal{Q}^*$ and the table T_H (to an empty set and empty table, respectively). Then it samples the tuple (N, AD, M) uniformly at random, and calls its own encryption oracle with input (N, M) to derive C . It then samples T randomly and runs $\mathcal{A}$ with input (C, T) . For $\mathcal{A}$'s each random oracle query $X = (K^*, *)$, $\mathcal{B}$ records K^* in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates and outputs (K', N', AD', M') , $\mathcal{B}$ takes a nonce $N^* \neq N$, and queries its own encryption oracle with the input $(N^*, 0^\kappa)$ to get back C' . Then it “queries” the random oracle H with the input (K', N', AD', C') and continues as follows. For each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}$ computes $C_i \leftarrow \mathbf{E}.\text{Enc}(K_i, N^*, 0^\kappa)$ and checks if $C_i = C$. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}$ guesses that it was in the real world REAL_E^B and outputs 1; otherwise, it outputs 0, guessing that it was in the ideal world RAND_E^B .

If $\mathcal{B}$ was in the real world, then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^A \Rightarrow 1] \leq \Pr[\text{REAL}_E^B \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $C_i = C$ check.

On the other hand, if $\mathcal{B}$ was in the ideal world, the probability that a random ciphertext C equals $\text{E.Enc}(K_i, N^*, 0^\kappa)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^\kappa$. Since $|\mathcal{Q}^*| \leq q_h + 1$, we have $\Pr[\text{RAND}_E^{\mathcal{B}} \Rightarrow 1] \leq (q_h + 1)/2^\kappa$. Recalling that $\text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}) = \Pr[\text{REAL}_E^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RAND}_E^{\mathcal{B}} \Rightarrow 1]$, we have

$$\text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}) \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h + 1}{2^\kappa}, \quad (36)$$

and it follows from equations (34)~(36) that

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_E^{\text{ind\$-cpa}}(\mathcal{B}) + \frac{q_h + 1}{2^\kappa}. \quad (37)$$

Finally, we observe that in game $\mathbf{G}_1$ the committing tag T in the challenge ciphertext (C, T) is independent of random oracle queries and hence independent of T' , so in order for $\mathcal{A}$ to win one of the random oracle queries must output a committing tag equal to the independent random T . This probability is bounded by $(q_h + 1)/2^t$. Therefore, we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \frac{q_h + 1}{2^t}. \quad (38)$$

The proof is concluded by combining equations (37) and (38). $\square$

B.13 Proof of Theorem 13 (MR-IND\\$-CPA Security of AEth)

Proof. We define four games $\mathbf{G}_0 \sim \mathbf{G}_3$ as shown in Figure 22.

Game $\mathbf{G}_0, \mathbf{G}_1$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H \wedge X = (K, *) :$	$C \leftarrow \text{AE.Enc}(K, N, AD, M)$
$T_H[\cdot] \leftarrow \perp ; T_T[\cdot] \leftarrow \perp$	bad $\leftarrow \text{true}$	if $T_T[(N, AD, C _\tau)] = \perp :$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T_T[(N, AD, C _\tau)] \xleftarrow{\$} \{0, 1\}^t$
return b'	return $T_H[X]$	$T \leftarrow T_T[(N, AD, C _\tau)]$
		$T \leftarrow H(K, N, AD, C _\tau)$
		return (C, T)

Game $\mathbf{G}_*$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	$(K', *) \leftarrow X$	$C \leftarrow \text{AE.Enc}(K, N, AD, M)$
$\mathcal{Q}^* \leftarrow \emptyset$	$\mathcal{Q}^* \leftarrow \cup \{K'\}$	if $T_T[(N, AD, C _\tau)] = \perp :$
$T_H[\cdot] \leftarrow \perp ; T_T[\cdot] \leftarrow \perp$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T_T[(N, AD, C _\tau)] \xleftarrow{\$} \{0, 1\}^t$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	return $T_H[X]$	$T \leftarrow T_T[(N, AD, C _\tau)]$
return $K \in \mathcal{Q}^*$		return (C, T)

Game $\mathbf{G}_2, \mathbf{G}_3$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$C \xleftarrow{\$} \{0, 1\}^{\text{AE.clen}(M)}$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	return $T_H[X]$	if $T_T[(N, AD, C _\tau)] = \perp :$
return b'		$T_T[(N, AD, C _\tau)] \xleftarrow{\$} \{0, 1\}^t$
		$T \leftarrow T_T[(N, AD, C _\tau)]$
		$T \xleftarrow{\$} \{0, 1\}^t$
		return (C, T)

Fig. 22: Games $\mathbf{G}_0 \sim \mathbf{G}_3$ and $\mathbf{G}_*$ in the proof of Theorem 13, where $\mathbf{G}_0$ and $\mathbf{G}_3$ contain boxed content while $\mathbf{G}_1$ and $\mathbf{G}_2$ do not.

Game $\mathbf{G}_0$ is the same as $\text{REAL}_{\text{AEth}}^{\mathcal{A}}$ (see Figure 4) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \Pr[\text{REAL}_{\text{AEth}}^{\mathcal{A}} \Rightarrow 1]$.

Game $\mathbf{G}_1$ is the same as $\mathbf{G}_0$, except that the committing tag is now lazily sampled from the table T_T in Enc oracle, where the table is indexed by the queried $(N, AD, C|_\tau)$ triple. We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the **bad** event happens, which occurs if the adversary $\mathcal{A}$ ever queries H with an input that is prefixed with the key K . The only syntactical difference between the two games is in the generation of T . If **bad** does not occur, in both games the committing tag T is either randomly sampled for new $(N, AD, C|_\tau)$ triple, or gets the same previously sampled value if $(N, AD, C|_\tau)$ is not new. Hence the two games are indeed identical-until-bad. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (39)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by considering a game $\mathbf{G}_*$ also shown in Figure 22. Here $\mathbf{G}_*$ is just defined to clarify the **bad** event, such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_0/\mathbf{G}_1$. Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (40)$$

Then, we bound $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{B}_1$ against the MR-IND\$-CPA security of the underlying AE. $\mathcal{B}_1$ starts by initializing the set $\mathcal{Q}^*$ and tables T_H, T_T (to an empty set and empty tables, respectively), and then it runs $\mathcal{A}$. To answer $\mathcal{A}$'s encryption query on input (N, AD, M) , $\mathcal{B}_1$ calls its own encryption oracle with input (N, AD, M) and gets back C , then lazily samples T with the help of table T_T . At the end, it returns (C, T) . For $\mathcal{A}$'s random oracle query $X = (K', *)$, $\mathcal{B}_1$ records K' in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates, $\mathcal{B}_1$ ignores its output and queries its own encryption oracle with input $(N^*, AD^*, 0^\kappa)$ and gets back C^* , where N^* is a nonce different from those used in previous encryption queries and AD^* is any valid associated data. Then, for each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}_1$ computes $C_i \leftarrow \text{AE.Enc}(K_i, N^*, AD^*, 0^\kappa)$ and checks if $C_i = C^*$ holds. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}_1$ guesses that it was in the real world $\text{REAL}_{\text{AE}}^{\mathcal{B}_1}$ and outputs 1; otherwise, it outputs 0, guessing that it was in the ideal world $\text{RAND}_{\text{AE}}^{\mathcal{B}_1}$.

If $\mathcal{B}_1$ was in the real world, then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{REAL}_{\text{AE}}^{\mathcal{B}_1} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $C_i = C^*$ check. On the other hand, if $\mathcal{B}_1$ was in the ideal world, the probability that a random ciphertext C^* equals $\text{AE.Enc}(K_i, N^*, AD^*, 0^\kappa)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^{\text{AE.clen}(\kappa)}$. Since $|\mathcal{Q}^*| \leq q_h$, we have $\Pr[\text{RAND}_{\text{AE}}^{\mathcal{B}_1} \Rightarrow 1] \leq q_h/2^{\text{AE.clen}(\kappa)}$. Then, recalling that $\text{Adv}_{\text{AE}}^{\text{mr-ind\$-cpa}}(\mathcal{B}_1) = \Pr[\text{REAL}_{\text{AE}}^{\mathcal{B}_1} \Rightarrow 1] - \Pr[\text{RAND}_{\text{AE}}^{\mathcal{B}_1} \Rightarrow 1]$ and $\text{AE.clen}(\kappa) \geq \kappa$, we have

$$\text{Adv}_{\text{AE}}^{\text{mr-ind\$-cpa}}(\mathcal{B}_1) \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h}{2^{\text{AE.clen}(\kappa)}} \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h}{2^\kappa}, \quad (41)$$

and it follows from equations (39)~(41) that

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{mr-ind\$-cpa}}(\mathcal{B}_1) + \frac{q_h}{2^\kappa}. \quad (42)$$

We continue by making a game hop from $\mathbf{G}_1$ to $\mathbf{G}_2$. As shown in Figure 22, game $\mathbf{G}_2$ is the same as game $\mathbf{G}_1$, except that now in the encryption oracle Enc the ciphertext C is sampled uniformly at random. We bound the gap between $\mathbf{G}_1$ and $\mathbf{G}_2$ by constructing an adversary $\mathcal{B}_2$ against the MR-IND\$-CPA security of the underlying AE. $\mathcal{B}_2$ just simulates $\mathcal{A}$'s view as the challenger in $\mathbf{G}_1/\mathbf{G}_2$ does, but for $\mathcal{A}$'s encryption query, $\mathcal{B}_2$ calls its own encryption oracle to get C and then it finishes the rest of the simulation. It is easy to see that $\mathcal{B}_2$ simulates $\mathcal{A}$'s view in $\mathbf{G}_1$ in the real world and $\mathbf{G}_2$ in the ideal world. In the end, adversary $\mathcal{B}_2$ outputs the bit that $\mathcal{A}$ outputs. Therefore, we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{mr-ind\$-cpa}}(\mathcal{B}_2). \quad (43)$$

We make the final game hop from $\mathbf{G}_2$ to $\mathbf{G}_3$ and for this we define a bad event **bad'**. We say that **bad'** occurs if $\mathcal{A}$ queried the encryption oracle with input (N, AD, M) where the table T_T was already defined for an entry $(N, AD, C|_\tau)$. If **bad'** does not happen, games $\mathbf{G}_2$ and $\mathbf{G}_3$ behave identically, i.e. the committing tag T will be a random string for $\mathcal{A}$'s every query. The games $\mathbf{G}_2$ and $\mathbf{G}_3$ behave identically if it does not happen for some encryption query (N, AD, M) that the table T_T was already set at point $(N, AD, C|_\tau)$. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}'] = \binom{q_e}{2} \frac{1}{2^\tau} \leq \frac{q_e^2}{2^{\tau+1}}, \quad (44)$$

Game $\boxed{\mathbf{G}_0}, \mathbf{G}_1$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \mathcal{K}$	if $\mathcal{A}$ queries $H(X)$	$C \leftarrow \text{AE.Enc}(K, N, AD, M)$	if $T_T[(N, AD, C _\tau)] = \perp :$
$T_H[\cdot] \leftarrow \perp$	$\wedge X = (K, *) :$	if $T_T[(N, AD, C _\tau)] = \perp :$	$T_T[(N, AD, C _\tau)] \xleftarrow{\$} \{0, 1\}^t$
$T_T[\cdot] \leftarrow \perp$	bad $\leftarrow \text{true}$	$T_T[(N, AD, C _\tau)] \xleftarrow{\$} \{0, 1\}^t$	$T' \leftarrow T_T[(N, AD, C _\tau)]$
$\mathcal{Q} \leftarrow \emptyset$	if $T_H[X] = \perp :$	$T \leftarrow T_T[(N, AD, C _\tau)]$	$\boxed{T' \leftarrow H(K, N, AD, C _t)}$
$\text{win} \leftarrow 0$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$\boxed{T \leftarrow H(K, N, AD, C _\tau)}$	if $T' \neq T : \text{return false}$
$\mathcal{A}^{\text{H, Enc, Ver}}$	return $T_H[X]$	$\mathcal{Q} \leftarrow \{(N, AD, (C, T))\}$	$M \leftarrow \text{AE.Dec}(N, AD, C)$
return win		return (C, T)	if $M \neq \perp \wedge (N, AD, (C, T)) \notin \mathcal{Q} :$
			win $\leftarrow 1$
			return $M \neq \perp$

Game $\mathbf{G}_*$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	$(K', *) \leftarrow X$	$C \leftarrow \text{AE.Enc}(K, AD, \varepsilon, M)$	if $T_T[(N, AD, C _\tau)] = \perp :$
$T_H[\cdot] \leftarrow \perp$	$\mathcal{Q}^* \leftarrow \{K'\}$	if $T_T[(N, AD, C _\tau)] = \perp :$	$T_T[(N, AD, C _\tau)] \xleftarrow{\$} \{0, 1\}^t$
$T_T[\cdot] \leftarrow \perp$	if $T_H[X] = \perp :$	$T_T[(N, AD, C _\tau)] \xleftarrow{\$} \{0, 1\}^t$	$T' \leftarrow T_T[(N, AD, C _\tau)]$
$\mathcal{Q}^* \leftarrow \emptyset$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow T_T[(N, AD, C _\tau)]$	if $T \neq T' : \text{return false}$
$\mathcal{A}^{\text{H, Enc, Ver}}$	return $T_H[X]$	return (C, T)	$M \leftarrow \text{AE.Dec}(N, AD, C)$
return $K \in \mathcal{Q}^*$			return $M \neq \perp$

Fig. 23: Games $\mathbf{G}_0$, $\mathbf{G}_1$ and $\mathbf{G}_*$ in the proof of Theorem 10, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

where we bound the probability of bad' happening by the probability that a collision in $C|_\tau$ value for two different queries occurs.

Finally, we observe that game $\mathbf{G}_3$ is identical to $\text{RAND}_{\text{AEtH}}^A$ (see Figure 4) equipped with a random oracle H . This is because, in game $\mathbf{G}_3$, for each Enc query, its output is an independent uniform random string sampled from $\{0, 1\}^{\text{AEtH.clen}(|M|)} = \{0, 1\}^{\text{AE.clen}(|M|)+t}$.

The proof is concluded by combining equations (42)~(44). $\square$

B.14 Proof of Theorem 14 (MR-INT-CTXT Security of AEtH)

Proof. We define two games $\mathbf{G}_0$ and $\mathbf{G}_1$ as shown in Figure 23.

Game $\mathbf{G}_0$ is the same as $\text{INT-CTXT}_{\text{AEtH}}$ (see Figure 4) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^A \Rightarrow 1] = \text{Adv}_{\text{AEtH}}^{\text{int-ctxt}}(\mathcal{A})$.

Game $\mathbf{G}_1$ is the same as $\mathbf{G}_0$, except that the committing tag is now lazily sampled from the table T_T in both oracles, where the table is indexed by the $(N, AD, C|_\tau)$ triple. We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the bad event happens, which occurs if the adversary $\mathcal{A}$ ever queries H with an input that is prefixed with the key K . The two syntactical difference between the two games are in the generation of T . If bad does not occur, in both games the committing tag T is either randomly sampled for new $(N, AD, C|_\tau)$ triple, or gets the same previously sampled value if $(N, AD, C|_\tau)$ is not new. Hence the two games are indeed identical-until- bad . By the Difference Lemma of the game-based proof technique [Sho04], we have

$$|\Pr[\mathbf{G}_0^A \Rightarrow 1] - \Pr[\mathbf{G}_1^A \Rightarrow 1]| \leq \Pr[\text{bad}]. \quad (45)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by considering a game $\mathbf{G}_*$ also shown in Figure 23. $\mathbf{G}_*$ is defined such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers bad in $\mathbf{G}_0/\mathbf{G}_1$. Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^A \Rightarrow 1]. \quad (46)$$

Then, we bound $\Pr[\mathbf{G}_*^A \Rightarrow 1]$ by constructing an adversary $\mathcal{B}_1$ against the INT-CTXT security of the underlying AE. $\mathcal{B}_1$ starts by initializing the set $\mathcal{Q}^*$ and tables T_H, T_T (to an empty set and empty tables, respectively), and then it runs $\mathcal{A}$. To answer $\mathcal{A}$'s encryption query on input (N, AD, M) , $\mathcal{B}_1$ calls its own encryption oracle with input (N, AD, M) and gets back C , then lazily samples T with the help of table

T_T . At the end, it returns (C, T) back to $\mathcal{A}$. To answer $\mathcal{A}$'s verification query on input $(N, AD, (C, T))$, $\mathcal{B}_1$ lazily samples T' with the help of the table T_T and checks if T' equals T . If not, $\mathcal{B}_1$ returns **false**, otherwise it calls its own verification oracle with input (N, AD, C) and forwards the answer back to $\mathcal{A}$. For $\mathcal{A}$'s random oracle query $X = (K', *)$, $\mathcal{B}_1$ records K' in $\mathcal{Q}^*$ and answers the query via lazy sampling using the table T_H . After $\mathcal{A}$ terminates, $\mathcal{B}_1$ picks a nonce N^* that is different from those used by $\mathcal{A}$ in encryption oracle queries; then, for each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}$ computes $C_i \leftarrow \text{AE.Enc}(K_i, N^*, AD, 0^\kappa)$ and queries its own verification oracle with (N^*, AD, C_i) .

The above adversary $\mathcal{B}_1$ correctly simulates the $\mathcal{A}$'s view in $\mathbf{G}_*$, and we claim that, if $\mathcal{A}$ wins in $\mathbf{G}_*$, then $\mathcal{B}_1$ wins in its $\text{INT-CTXT}_{\text{AE}}^{\mathcal{B}_1}$ game. If $\mathcal{A}$ wins, then the AE key K (sampled by $\mathcal{B}_1$'s challenger) must be included in the set $\mathcal{Q}^*$. Since $\mathcal{B}_1$ enumerated all keys in $\mathcal{Q}^*$ to compute ciphertexts with a new nonce, the ciphertext derived from the correct key K must be a valid forgery for $\mathcal{B}_1$ to win in its game. Therefore, it holds that

$$\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{mr-int-ctxt}}(\mathcal{B}_1). \quad (47)$$

Now we bound $\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1]$ by constructing another MR-INT-CTXT adversary $\mathcal{B}_2$ against the underlying AE. Adversary $\mathcal{B}_2$ is the same as adversary $\mathcal{B}_1$, except that $\mathcal{B}_2$ terminates immediately after $\mathcal{A}$ terminates, i.e., $\mathcal{B}_2$ does not record keys in $\mathcal{Q}^*$ or make the final verification queries after $\mathcal{A}$ terminates. Clearly, $\mathcal{B}_2$ correctly simulates $\mathcal{A}$'s view in $\mathbf{G}_1$. We continue by showing that if $\mathcal{A}$ wins, then adversary $\mathcal{B}_2$ also wins in its $\text{INT-CTXT}_{\text{AE}}^{\mathcal{B}_2}$ game. Suppose $\mathcal{A}$ wins by querying $\text{Ver}(N^*, AD^*, (C^*, T^*))$. By definition of the winning condition of $\mathcal{A}$, we know (C^*, T^*) was not output by any $\text{Enc}(N^*, AD^*, \cdot)$ query and $\text{Ver}(N^*, AD^*, (C^*, T^*)) = 1$. By construction of $\mathcal{B}_2$, the latter implies that $T^* = T_T[(N^*, AD^*, C^*|_\tau)]$ and $\text{Ver}'(N^*, AD^*, C^*) = 1$, so it suffices to show that C^* was not output by any $\text{Enc}'(N^*, AD^*, \cdot)$ query made by $\mathcal{B}_2$. Suppose otherwise, that C^* was output by some query to $\text{Enc}'(N^*, AD^*, \cdot)$. In that case, (C^*, T^*) would have been returned by Enc oracle to $\mathcal{A}$ since we know $T^* = T_T[(N^*, AD^*, C^*|_\tau)]$. That would be a contradiction. Therefore, $\mathcal{B}_2$ also wins and it holds that

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{mr-int-ctxt}}(\mathcal{B}_2). \quad (48)$$

The proof is concluded by combining equations (45)~(48). $\square$

B.15 Proof of Theorem 15 (CMT Security of AEtH)

Proof. In order to win the $\text{CMT}_{\text{AEtH}}^{\mathcal{A}}$ game (see Figure 5), $\mathcal{A}$ needs to output two tuples (K_1, N_1, AD_1, M_1) , (K_2, N_2, AD_2, M_2) such that they are encrypted to the same ciphertext and $(K_1, N_1, AD_1) \neq (K_2, N_2, AD_2)$. By construction of AEtH, this implies that the hashes of $(K_1, N_1, AD_1, C|_\tau)$ and $(K_2, N_2, AD_2, C|_\tau)$ collide, which breaks collision resistance of H . Therefore, we can construct $\mathcal{B}$ as follows: it first invokes $\mathcal{A}$, then extracts (K_1, N_1, AD_1) and (K_2, N_2, AD_2) from $\mathcal{A}$'s output. It continues by computing C and outputting the tuple pair $((K_1, N_1, AD_1, C|_\tau), (K_2, N_2, AD_2, C|_\tau))$. $\square$

B.16 Proof of Theorem 16 (CDY\$ Security of AEtH)

Proof. We define two games $\mathbf{G}_0$ and $\mathbf{G}_1$ as shown in Figure 24.

Game $\mathbf{G}_0$ is the same as $\text{CDY}_{\text{AEtH}, S_s}^{\mathcal{A}}$ (see Figure 6) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{AEtH}, S_s}^{\text{cdy}}(\mathcal{A}) = \text{Adv}_{\text{AEtH}}^{\text{cdy}\$}(\mathcal{A})$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that the committing tag T , calculated by the challenger, is now independent of queries to the random oracle H . We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the **bad** event happens, where the random oracle H is ever queried with an input that is prefixed with the key K sampled by the challenger. This is because, if **bad** does not occur, then T in $\mathbf{G}_0$ is also independent of random oracle queries made by $\mathcal{A}$ and by the challenger (to compute T'). Therefore, by the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (49)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by constructing a game $\mathbf{G}_*$ also shown in Figure 24. $\mathbf{G}_*$ is defined such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_0/\mathbf{G}_1$ when (1) $\mathcal{A}$ queried the random oracle H with an input prefixed with K or (2) $\mathcal{A}$ output (K', N', AD', M') with $K' = K$ and then $H(K', N', AD', C'|_\tau)$ was queried by the challenger, where $C' \leftarrow \text{AE.Enc}(K', N', AD', M')$. Therefore, it holds that

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (50)$$

Game $\mathbf{G}_0, \mathbf{G}_1$	Oracle $H(X)$
$T_H[\cdot] \leftarrow \perp$; $K \xleftarrow{\$} \{0,1\}^\kappa$; $N \xleftarrow{\$} \mathcal{N}$; $AD \xleftarrow{\$} \mathcal{AD}$; $M \xleftarrow{\$} \mathcal{M}$ $C \leftarrow \text{AE.Enc}(K, N, AD, M)$; $T \xleftarrow{\$} \{0,1\}^t$ $T_H[(K, N, AD, C _\tau)] \leftarrow T$ $(K', N', AD', M') \xleftarrow{\$} \mathcal{A}^H(C, T)$ $C' \leftarrow \text{AE.Enc}(K', N', AD', M')$; $T' \leftarrow H(K', N', AD', C' _\tau)$ return $(C', T') = (C, T)$	if $X = (K, *)$: bad $\leftarrow$ true if $T_H[X] = \perp$: $T_H[X] \xleftarrow{\$} \{0,1\}^t$ return $T_H[X]$
Game $\mathbf{G}_*$	Oracle $H(X)$
$T_H[\cdot] \leftarrow \perp$; $K \xleftarrow{\$} \{0,1\}^\kappa$; $N \xleftarrow{\$} \mathcal{N}$; $AD \xleftarrow{\$} \mathcal{AD}$; $M \xleftarrow{\$} \mathcal{M}$ $C \leftarrow \text{AE.Enc}(K, N, AD, M)$; $T \xleftarrow{\$} \{0,1\}^t$ $\mathcal{Q}^* \leftarrow \emptyset$ $(K', N', AD', M') \xleftarrow{\$} \mathcal{A}^H(C, T)$ $C' \leftarrow \text{AE.Enc}(K', N', AD', M')$; $H(K', N', AD', C' _\tau)$ return $K \in \mathcal{Q}^*$	$(K^*, *) \leftarrow X$ $\mathcal{Q}^* \leftarrow \mathcal{Q}^* \cup \{K^*\}$ if $T_H[X] = \perp$: $T_H[X] \xleftarrow{\$} \{0,1\}^t$ return $T_H[X]$

Fig. 24: Games $\mathbf{G}_0, \mathbf{G}_1$ and $\mathbf{G}_*$ in the proof of Theorem 16, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

Then, we bound $\Pr[\mathbf{G}_*^A \Rightarrow 1]$ by constructing an adversary $\mathcal{B}$ against the one-time IND $\$$ -CPA security of the underlying AE. $\mathcal{B}$ starts by initializing the set $\mathcal{Q}^*$ and the table T_H (to an empty set and empty table, respectively). Then it samples the tuple (N, AD, M) uniformly at random, and calls its own encryption oracle with input (N, AD, M) to derive C . It then samples T randomly and runs $\mathcal{A}$ with input (C, T) . For $\mathcal{A}$'s each random oracle query $X = (K^*, *)$, $\mathcal{B}$ records K^* in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates and outputs (K', N', AD', M') , $\mathcal{B}$ “queries” the random oracle H with the input $(K', N', AD', C'|_\tau)$, where $C' \leftarrow \text{AE.Enc}(K', N', AD', M')$, and does the following. For each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}$ computes $C_i \leftarrow \text{AE.Enc}(K_i, N, AD, M)$ and checks if $C_i = C$. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}$ guesses that it was in the real world $\text{REAL}_{\text{AE}}^{\mathcal{B}}$ and outputs 1; otherwise, it outputs 0, guessing that it was in the ideal world $\text{RAND}_{\text{AE}}^{\mathcal{B}}$.

If $\mathcal{B}$ was in the real world, then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^A \Rightarrow 1] \leq \Pr[\text{REAL}_{\text{AE}}^{\mathcal{B}} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $C_i = C$ check. On the other hand, if $\mathcal{B}$ was in the ideal world, the probability that a random ciphertext C equals $\text{AE.Enc}(K_i, N, AD, M)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^{\text{AE.clen}(|M|)}$. Since $|\mathcal{Q}^*| \leq q_h + 1$, we have $\Pr[\text{RAND}_{\text{AE}}^{\mathcal{B}} \Rightarrow 1] \leq (q_h + 1)/2^{\text{AE.clen}(|M|)}$. Recalling that $\text{Adv}_{\text{AE}}^{\text{ot-ind}\$-\text{cpa}}(\mathcal{B}) = \Pr[\text{REAL}_{\text{AE}}^{\mathcal{B}} \Rightarrow 1] - \Pr[\text{RAND}_{\text{AE}}^{\mathcal{B}} \Rightarrow 1]$ and $\text{AE.clen}(|M|) \geq \tau$, we have

$$\text{Adv}_{\text{AE}}^{\text{ot-ind}\$-\text{cpa}}(\mathcal{B}) \geq \Pr[\mathbf{G}_*^A \Rightarrow 1] - \frac{q_h + 1}{2^\tau}, \quad (51)$$

and it follows from equations (49)~(51) that

$$\Pr[\mathbf{G}_0^A \Rightarrow 1] - \Pr[\mathbf{G}_1^A \Rightarrow 1] \leq \text{Adv}_{\text{AE}}^{\text{ot-ind}\$-\text{cpa}}(\mathcal{B}) + \frac{q_h + 1}{2^\tau}. \quad (52)$$

Finally, we observe that in game $\mathbf{G}_1$ the committing tag T in the challenge ciphertext (C, T) is independent of random oracle queries and hence independent of T' , so in order for $\mathcal{A}$ to win one of the random oracle queries must output a committing tag equal to the independent random T . This probability is bounded by $(q_h + 1)/2^t$. Therefore, we have

$$\Pr[\mathbf{G}_1^A \Rightarrow 1] \leq \frac{q_h + 1}{2^t}. \quad (53)$$

The proof is concluded by combining equations (52) and (53). $\square$

B.17 Proof of Theorem 17 (MR-IND\$-CPA Security of chaSIV)

Proof. We start by defining three games $\mathbf{G}_0 \sim \mathbf{G}_2$ as shown in Figure 25.

Game $\mathbf{G}_0$ is the same as $\text{REAL}_{\text{chaSIV}}^{\mathcal{A}}$ (see Figure 4) equipped with a random oracle H . Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \Pr[\text{REAL}_{\text{chaSIV}}^{\mathcal{A}} \Rightarrow 1]$.

Game $\mathbf{G}_0, \boxed{\mathbf{G}_1}$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H(X) \wedge X = (K, *)$:	$K_e \leftarrow F(K, N)$
$T_H[\cdot] \leftarrow \perp$	bad $\leftarrow \text{true}$	$T \leftarrow H(K, N, AD, M)$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \xleftarrow{\$} \{0, 1\}^t$
return b'	return $T_H[X]$	$N_e \leftarrow T _r$
		$C \leftarrow E.\text{Enc}(K_e, N_e, M)$
		return (C, T)

Game $\mathbf{G}_*$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	$(K', *) \leftarrow X$	$K_e \leftarrow F(K, N)$
$\mathcal{Q}^* \leftarrow \emptyset$	$\mathcal{Q}^* \leftarrow \{K'\}$	$T \xleftarrow{\$} \{0, 1\}^t$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$N_e \leftarrow T _r$
$b' \xleftarrow{\$} \mathcal{A}^{H, \text{Enc}}$	return $T_H[X]$	$C \leftarrow E.\text{Enc}(K_e, AD, M)$
return $K \in \mathcal{Q}^*$		return (C, T)

Fig. 25: Games $\mathbf{G}_0, \mathbf{G}_1$ and $\mathbf{G}_*$ in the proof of Theorem 17, where $\mathbf{G}_1$ contains boxed content while $\mathbf{G}_0$ does not.

Game $\mathbf{G}_1$ is the same as $\mathbf{G}_0$, except that the committing tag is now sampled randomly from the set $\{0, 1\}^t$ in the encryption oracle Enc . We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the **bad** event happens, which occurs if the adversary $\mathcal{A}$ ever queries H with an input that is prefixed with the key K . The only syntactical difference between the two games is in the generation of T . Recall that $\mathcal{A}$ will never repeat (N, AD, M) input, therefore, if **bad** does not occur, in both games the committing tag T is random string that is independent of $\mathcal{A}$'s view. Hence, games $\mathbf{G}_0$ and $\mathbf{G}_1$ are indeed identical-until-**bad**. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (54)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by considering a game $\mathbf{G}_*$ also shown in Figure 25. Here $\mathbf{G}_*$ is just defined to clarify the **bad** event, such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_0/\mathbf{G}_1$. Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (55)$$

Then, we bound $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{C}_1$ against the PRF security of the underlying F . $\mathcal{C}_1$ starts by initializing the set $\mathcal{Q}^*$ and table T_H (to an empty set and empty table, respectively), and then it runs $\mathcal{A}$. To answer $\mathcal{A}$'s encryption query on input (N, AD, M) , $\mathcal{C}_1$ calls its own PRF oracle with input N to get K_e back. Then it samples T at random and calculates $C \leftarrow E.\text{Enc}(K_e, T|_r, M)$, returning (C, T) back to $\mathcal{A}$. For $\mathcal{A}$'s random oracle query $X = (K', *)$, $\mathcal{C}_1$ records K' in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates, $\mathcal{C}_1$ ignores its output and queries its own PRF oracle with input N^* and gets back K_e^* , where N^* is a nonce different from those used in previous PRF queries. Then, for each key $K_i \in \mathcal{Q}^*$, $\mathcal{C}_1$ computes $K_{e,i} \leftarrow F(K_i, N^*)$ and checks if $K_{e,i} = K_e^*$ holds. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{C}_1$ guesses that it was interacting with the real PRF and outputs 1; otherwise, it outputs 0, guessing that it was interacting with a truly random function.

If $\mathcal{C}_1$ had access to real F_K , then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\mathcal{C}_1^{F_K} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $K_{e,i} = K_e^*$ check. On the other hand, if $\mathcal{C}_1$ was in the ideal world, the probability that a random output

K_e^* equals $F(K_i, N^*)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^\kappa$. Since $|\mathcal{Q}^*| \leq q_h$, we have $\Pr[\mathcal{C}_1^f \Rightarrow 1] \leq q_h/2^\kappa$. Then, recalling that $\mathbf{Adv}_F^{\text{prf}}(\mathcal{C}_1) = \Pr[\mathcal{C}_1^{F_K} \Rightarrow 1] - \Pr[\mathcal{C}_1^f \Rightarrow 1]$ we have

$$\mathbf{Adv}_F^{\text{prf}}(\mathcal{C}_1) \geq \Pr[\mathbf{G}_*^A \Rightarrow 1] - \frac{q_h}{2^\kappa}, \quad (56)$$

and it follows from equations (54)~(56) that

$$\Pr[\mathbf{G}_0^A \Rightarrow 1] - \Pr[\mathbf{G}_1^A \Rightarrow 1] \leq \mathbf{Adv}_F^{\text{prf}}(\mathcal{C}_1) + \frac{q_h}{2^\kappa}. \quad (57)$$

Game $\mathbf{G}_2$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$T_H[\cdot] \leftarrow \perp$ $b' \xleftarrow{\$} \mathcal{A}^{\text{H,Enc}}$ return b'	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$ return $T_H[X]$	$K_e \leftarrow f(N)$ $T \xleftarrow{\$} \{0, 1\}^t$ $N_e \leftarrow T _r$ $C \leftarrow \text{E.Enc}(K_e, N_e, M)$ return (C, T)
Game $\mathbf{G}_3$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$T_H[\cdot] \leftarrow \perp$ $T_N[\cdot] \leftarrow \perp$ $b' \xleftarrow{\$} \mathcal{A}^{\text{H,Enc}}$ return b'	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$ return $T_H[X]$	$K_e \leftarrow f(N)$ $T \xleftarrow{\$} \{0, 1\}^t$ $N_e \leftarrow T _r$ $C \leftarrow \text{E.Enc}(K_e, N_e, M)$ if $N_e \in T_N[N] :$ bad' $\leftarrow \text{true}$ $C \xleftarrow{\$} \{0, 1\}^{\text{E.clen}(M)}$ $T_N[N] \leftarrow \cup \{N_e\}$ return (C, T)
Game $\mathbf{G}_4$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$
$T_H[\cdot] \leftarrow \perp$ $b' \xleftarrow{\$} \mathcal{A}^{\text{H,Enc}}$ return b'	if $T_H[X] = \perp : T_H[X] \xleftarrow{\$} \{0, 1\}^t$ return $T_H[X]$	$T \xleftarrow{\$} \{0, 1\}^t$ $C \xleftarrow{\$} \{0, 1\}^{\text{E.clen}(M)}$ return (C, T)

Fig. 26: Games $\mathbf{G}_2 \sim \mathbf{G}_4$ in the proof of Theorem 17.

We continue by making a game hop from $\mathbf{G}_1$ to $\mathbf{G}_2$ by replacing F_K invocations with invocations to a random function f . This is a standard game hop where a PRF adversary $\mathcal{C}_2$ simulates $\mathbf{G}_1/\mathbf{G}_2$ while having access to its own PRF oracle. Therefore, it holds

$$\Pr[\mathbf{G}_1^A \Rightarrow 1] - \Pr[\mathbf{G}_2^A \Rightarrow 1] \leq \mathbf{Adv}_F^{\text{prf}}(\mathcal{C}_2). \quad (58)$$

Next, we define two more games $\mathbf{G}_3$ and $\mathbf{G}_4$ as shown in Figure 26. In game $\mathbf{G}_3$ we define a bad event bad' which captures the event that there is a collision in derived nonce N_e for some nonce N queried by $\mathcal{A}$. Since the underlying E is secure only in a nonce-respecting case, we need to make sure there will be no collisions in derived nonce for different instances of E , where these instances of E directly correspond to nonces that are queried by $\mathcal{A}$. In order to define the bad event bad' , we also introduce the table T_N in game $\mathbf{G}_3$, where the elements of the table will be sets containing derived nonces of an E instance indexed by nonce that was queried by $\mathcal{A}$.

Games $\mathbf{G}_2$ and $\mathbf{G}_3$ are identical-until-bad and so it holds

$$\Pr[\mathbf{G}_2^A \Rightarrow 1] - \Pr[\mathbf{G}_3^A \Rightarrow 1] \leq \Pr[\text{bad}'] \leq \frac{R_1^2}{2^r} + \dots + \frac{R_{Q_e}^2}{2^r}, \quad (59)$$

where the individual summands represent the standard collision birthday bound.

Game $\mathbf{G}_4$ is the same as game $\mathbf{G}_3$, except that now C in encryption oracle is always sampled uniformly at random. To bound the gap between $\mathbf{G}_3$ and $\mathbf{G}_4$, we apply a hybrid argument as follows. Consider a sequence of hybrid games $\mathbf{H}_0, \mathbf{H}_1, \dots, \mathbf{H}_{Q_e}$ such that $\mathbf{G}_3 = \mathbf{H}_0$ and $\mathbf{G}_4 = \mathbf{H}_{Q_e}$. Now, let $\mathcal{Q}_N = \{N_1, \dots, N_{Q_e}\}$ be the list of nonces queried by $\mathcal{A}$. Game $\mathbf{H}_i$ is the same as game $\mathbf{G}_3$, except that the output of all queries that have been made (to $\mathbf{E}$) with a nonce N_j , with $j \leq i$, are replaced by independent random strings as in $\mathbf{G}_4$. By construction, consecutive games $\mathbf{H}_{i-1}$ and $\mathbf{H}_i$ are the same except for the queries made to $\mathbf{E}$ with nonce N_i . We can construct an adversary $\mathcal{B}_i$ against the IND $\mathbb{S}$ -CPA security of the underlying $\mathbf{E}$ such that it simulates $\mathcal{A}$'s view in $\mathbf{H}_{i-1}$ in the real world and $\mathbf{H}_i$ in the ideal world. To do so, $\mathcal{B}_i$ simulates $\mathcal{A}$'s views in $\mathbf{H}_{i-1}$ and $\mathbf{H}_i$ in the same way following their game procedures, except that, for Enc queries made with nonce N_i , $\mathcal{B}_i$ samples a random T and calls its own encryption oracle Enc' to simulate the $\mathbf{E}$ ciphertext C . Finally, $\mathcal{B}_i$ outputs the bit that $\mathcal{A}$ outputs. By construction, we have $\Pr[\mathbf{H}_{i-1}^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{H}_i^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\mathbf{E}}^{\text{ind}\mathbb{S}\text{-cpa}}(\mathcal{B}_i)$. With a hybrid argument, there exists an efficient adversary $\mathcal{B}$ such that

$$\begin{aligned} \Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] &= \Pr[\mathbf{H}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{H}_{Q_e}^{\mathcal{A}} \Rightarrow 1] \\ &= \sum_{i=1}^{Q_e} (\Pr[\mathbf{H}_{i-1}^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{H}_i^{\mathcal{A}} \Rightarrow 1]) \\ &\leq Q_e \text{Adv}_{\mathbf{E}}^{\text{ind}\mathbb{S}\text{-cpa}}(\mathcal{B}). \end{aligned} \quad (60)$$

Finally, we observe that game $\mathbf{G}_4$ is identical to $\text{RAND}_{\text{chaSIV}}^{\mathcal{A}}$ (see Figure 4) equipped with a random oracle $\mathbf{H}$. This is because, in game $\mathbf{G}_4$, for each Enc query, its output is an independent uniform random string sampled from $\{0, 1\}^{\text{E.clen}(|M|)+t}$. Therefore, we have

$$\text{Adv}_{\text{chaSIV}}^{\text{mr-ind}\mathbb{S}\text{-cpa}}(\mathcal{A}) = \Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1]. \quad (61)$$

The proof is concluded by combining equations (57)~(61). $\square$

B.18 Proof of Theorem 18 (MR-INT-CTXT Security of chaSIV)

Proof. We start by defining three games $\mathbf{G}_0 \sim \mathbf{G}_2$ as shown in Figure 27.

Game $\mathbf{G}_0$ is the same as $\text{INT-CTXT}_{\text{chaSIV}}^{\mathcal{A}}$ (see Figure 4) equipped with a random oracle $\mathbf{H}$. Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \Pr[\text{INT-CTXT}_{\text{chaSIV}}^{\mathcal{A}} \Rightarrow 1]$.

Game $\mathbf{G}_1$ is the same as $\mathbf{G}_0$, except that the verification oracle does not check if the ciphertext C can be successfully decrypted (note that the T_T operations can be ignored in $\mathbf{G}_1$). As a result, the winning condition in $\mathbf{G}_1$ is relaxed, i.e., if $\mathcal{A}$ wins in $\mathbf{G}_0$ then it must also win in $\mathbf{G}_1$. To see this, suppose $\mathcal{A}$ wins in $\mathbf{G}_0$ by querying $(N^*, AD^*, (C^*, T^*))$ and before this query $\mathcal{A}$ did not already win in $\mathbf{G}_1$. In this case, $\mathcal{A}$'s views in $\mathbf{G}_0$ and $\mathbf{G}_1$ are identical. This is because before that winning query each verification query with a valid T must satisfy $(N, AD, (C, T)) \in \mathcal{Q}$ (otherwise $\mathcal{A}$ wins in $\mathbf{G}_1$), which implies that $M \neq \perp$ and hence the verification oracle returns **true** in both games. Then, by the winning condition in $\mathbf{G}_1$, the above query also allows $\mathcal{A}$ to win in $\mathbf{G}_1$. Therefore, we have

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1]. \quad (62)$$

Game $\mathbf{G}_2$ is the same as $\mathbf{G}_1$, except that the committing tag is now lazily sampled from the table T_T in both oracles, where the table is indexed by the (N, AD, M) triple. We claim that $\mathbf{G}_1$ and $\mathbf{G}_2$ behave identically unless the **bad** event happens, which occurs if the adversary $\mathcal{A}$ ever queries $\mathbf{H}$ with an input that is prefixed with the key K . The only syntactical difference between the two games is in the generation of T . If **bad** does not occur, in both games the committing tag T is either randomly sampled for a new (N, AD, M) triple or equal to the same previously sampled value if (N, AD, M) is not new. Hence the two games are indeed identical-until-**bad**. By the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (63)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by considering a game $\mathbf{G}_*$ also shown in Figure 27. Here $\mathbf{G}_*$ is just defined to clarify the **bad** event, such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_1/\mathbf{G}_2$. Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (64)$$

Game $\mathbf{G}_0$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H(X)$	$K_e \leftarrow F(K, N)$	$K_e \leftarrow F(K, N) ; N_e \leftarrow T _r$
$T_H[\cdot] \leftarrow \perp$	$\wedge X = (K, *) :$	$T \leftarrow H(K, N, AD, M)$	$M \leftarrow \text{E.Dec}(K_e, N_e, C)$
$\mathcal{Q} \leftarrow \emptyset$	bad $\leftarrow \text{true}$	$N_e \leftarrow T _r$	$T' \leftarrow H(K, N, AD, M)$
$\text{win} \leftarrow 0$	if $T_H[X] = \perp :$	$C \leftarrow \text{E.Enc}(K_e, N_e, M)$	if $T \neq T' : \text{return false}$
$\mathcal{A}^{\text{H,Enc,Ver}}$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$\mathcal{Q} \leftarrow \{(N, AD, (C, T))\}$	if $M \neq \perp \wedge (N, AD, (C, T)) \notin \mathcal{Q} :$
return win	return $T_H[X]$	return (C, T)	$\text{win} \leftarrow 1$
			return $M \neq \perp$

Game $\mathbf{G}_1, \mathbf{G}_2$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	if $\mathcal{A}$ queries $H(X)$	$K_e \leftarrow F(K, N)$	$K_e \leftarrow F(K, N) ; N_e \leftarrow T _r$
$T_H[\cdot] \leftarrow \perp$	$\wedge X = (K, *) :$	if $T_T[(N, AD, M)] = \perp :$	$M \leftarrow \text{E.Dec}(K_e, N_e, C)$
$T_T[\cdot] \leftarrow \perp$	bad $\leftarrow \text{true}$	$T_T[(N, AD, M)] \xleftarrow{\$} \{0, 1\}^t$	if $T_T[(N, AD, M)] = \perp :$
$\mathcal{Q} \leftarrow \emptyset$	if $T_H[X] = \perp :$	$T \leftarrow T_T[(N, AD, M)]$	$T_T[(N, AD, M)] \xleftarrow{\$} \{0, 1\}^t$
$\text{win} \leftarrow 0$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow H(K, N, AD, M)$	$T' \leftarrow T_T[(N, AD, M)]$
$\mathcal{A}^{\text{H,Enc,Ver}}$	return $T_H[X]$	$N_e \leftarrow T _r$	$T' \leftarrow H(K, N, AD, M)$
return win		$C \leftarrow \text{E.Enc}(K_e, N_e, M)$	if $T \neq T' : \text{return false}$
		$\mathcal{Q} \leftarrow \{(N, AD, (C, T))\}$	if $(N, AD, (C, T)) \notin \mathcal{Q} :$
		return (C, T)	$\text{win} \leftarrow 1$
			return true

Game $\mathbf{G}_*$	Oracle $H(X)$	Oracle $\text{Enc}(N, AD, M)$	Oracle $\text{Ver}(N, AD, (C, T))$
$K \xleftarrow{\$} \{0, 1\}^\kappa$	$(K', *) \leftarrow X$	$K_e \leftarrow F(K, N)$	$K_e \leftarrow F(K, N) ; N_e \leftarrow T _r$
$\mathcal{Q}^* \leftarrow \emptyset$	$\mathcal{Q}^* \leftarrow \{K'\}$	if $T_T[(N, AD, M)] = \perp :$	$M \leftarrow \text{E.Dec}(K_e, N_e, C)$
$T_H[\cdot] \leftarrow \perp$	if $T_H[X] = \perp :$	$T_T[(N, AD, M)] \xleftarrow{\$} \{0, 1\}^t$	if $T_T[(N, AD, M)] = \perp :$
$T_T[\cdot] \leftarrow \perp$	$T_H[X] \xleftarrow{\$} \{0, 1\}^t$	$T \leftarrow T_T[(N, AD, M)]$	$T_T[(N, AD, M)] \xleftarrow{\$} \{0, 1\}^t$
$b' \xleftarrow{\$} \mathcal{A}^{\text{H,Enc}}$	return $T_H[X]$	$N_e \leftarrow T _r$	$T' \leftarrow T_T[(N, AD, M)]$
return $K \in \mathcal{Q}^*$		$C \leftarrow \text{E.Enc}(K_e, AD, M)$	if $T \neq T' : \text{return false}$
		return (C, T)	return true

Fig. 27: Games $\mathbf{G}_0 \sim \mathbf{G}_2$ and $\mathbf{G}_*$ in the proof of [Theorem 18](#), where $\mathbf{G}_1$ contains boxed content while $\mathbf{G}_2$ does not.

Then, we bound $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{B}$ against the PRF security of the underlying F . $\mathcal{B}$ starts by initializing the set $\mathcal{Q}^*$ and table T_H (to an empty set and empty table, respectively), and then it runs $\mathcal{A}$. During the simulation of encryption and verification oracles for $\mathcal{A}$, the adversary $\mathcal{B}$ will keep track of nonces queried by $\mathcal{A}$. If $\mathcal{A}$ makes a query with a new nonce N , $\mathcal{B}$ will query its own PRF oracle to get K_e back and save the result (N, K_e) internally. Then, if $\mathcal{A}$ makes a query with a nonce that already “appeared” before, $\mathcal{B}$ will just fetch the previous result K_e that corresponds to the queried nonce.

To answer $\mathcal{A}$ ’s encryption query on input (N, AD, M) , $\mathcal{B}$ calls its own PRF oracle with input N , or fetches a previous result if the nonce is not new, to get K_e . Then it lazily samples T with the help of table T_T and calculates $C \leftarrow \text{E.Enc}(K_e, T|_r, M)$, returning (C, T) back to $\mathcal{A}$. To answer $\mathcal{A}$ ’s verification query on input $(N, AD, (C, T))$, $\mathcal{B}$ calls its own PRF oracle with input N , or fetches a previous result if the nonce is not new, to get K_e . Then it calculates $M \leftarrow \text{E.Dec}(K_e, T'|_r, C)$. It continues by lazily sampling T' with the help of table T_T and checking if $T' = T$. If not, $\mathcal{B}$ returns **false** to $\mathcal{A}$; otherwise, it returns **true** to $\mathcal{A}$. For $\mathcal{A}$ ’s random oracle query $X = (K', *)$, $\mathcal{B}$ records K' in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates, $\mathcal{B}$ ignores its output and queries its own PRF oracle with input N^* and gets back K_e^* , where N^* is a nonce different from those used in previous PRF queries. Then, for each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}$ computes $K_{e,i} \leftarrow F(K_i, N^*)$ and checks if $K_{e,i} = K_e^*$ holds. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}$ guesses that it was interacting with the real PRF and outputs 1; otherwise, it outputs 0, guessing that it was interacting with a truly random function.

If $\mathcal{B}$ had access to real F_K , then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\mathcal{B}^{F_K} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $K_{e,i} = K_e^*$ check. On the other hand, if $\mathcal{B}$ was in the ideal world, the probability that a random output K_e^* equals $F(K_i, N^*)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^\kappa$. Since $|\mathcal{Q}^*| \leq q_h$, we have $\Pr[\mathcal{B}^f \Rightarrow 1] \leq q_h/2^\kappa$. Then, recalling that $\text{Adv}_F^{\text{prf}}(\mathcal{B}) = \Pr[\mathcal{B}^{F_K} \Rightarrow 1] - \Pr[\mathcal{B}^f \Rightarrow 1]$ we have

$$\text{Adv}_F^{\text{prf}}(\mathcal{B}) \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h}{2^\kappa}, \quad (65)$$

and it follows from equations (63)~(65) that

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}) + \frac{q_h}{2^\kappa}. \quad (66)$$

Now it is left to bound $\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1]$. Suppose $\mathcal{A}$ wins by querying $\text{Ver}(N^*, AD^*, (C^*, T^*))$ and that the winning tuple decrypts to M^* . We know $\mathcal{A}$ did not query (N^*, AD^*, M^*) previously to the encryption oracle because by the tidyness of $\mathbf{E}$, if $\mathcal{A}$ did make such a query, result of $\mathbf{E}.\text{Enc}(K_e, N^*, M^*)$ would be exactly C^* . Then, since $T^* = T_T[(N^*, AD^*, M^*)]$, that previous encryption oracle would have returned (C^*, T^*) which is a contradiction with $(C^*, T^*) \notin \mathcal{Q}$ part of the winning condition. Therefore, the only way $\mathcal{A}$ can win is by querying Ver and trying to “guess” T^* such that $T^* = T_T[(N^*, AD^*, M^*)]$.

The probability that $\mathcal{A}$ wins with its first verification query is $\frac{1}{2^t}$ since we know $T_T[(N^*, AD^*, M^*)]$ would be sampled after $\mathcal{A}$ has made its guess T^* . For its second verification query, assuming the first was unsuccessful, $\mathcal{A}$ can either query new (N^*, AD^*) pair, leading again to the winning probability $\frac{1}{2^t}$. Otherwise, it can try to guess $T_T[(N^*, AD^*, M^*)]$ from its first verification query. The probability that it wins is $(1 - \frac{1}{2^t})\frac{1}{2^t-1} = \frac{1}{2^t}$, where the first multiplicative term is the probability that $\mathcal{A}$ did not win in its first query. For the subsequent verification queries the adversary can only follow the same strategy, either trying to guess the tag for new (N^*, AD^*, M^*) tuple or trying to guess the tag for one of the “old” tuples. In both cases, the probability that it succeeds is $\frac{1}{2^t}$. Therefore, in total,

$$\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] \leq \frac{q_v}{2^t}. \quad (67)$$

The proof is concluded by combining equations (62), (66) and (67). $\square$

B.19 Proof of Theorem 19 (CMT Security of chaSIV)

Proof. In order to win the $\text{CMT}_{\text{chaSIV}}^{\mathcal{A}}$ game (see Figure 5), $\mathcal{A}$ needs to output two tuples (K_1, N_1, AD_1, M_1) , (K_2, N_2, AD_2, M_2) such that they are encrypted to the same ciphertext and $(K_1, N_1, AD_1) \neq (K_2, N_2, AD_2)$. By construction of chaSIV, this implies that the hashes of (K_1, N_1, AD_1, M_1) and (K_2, N_2, AD_2, M_2) collide, which breaks collision resistance of $\mathbf{H}$. Therefore, we can construct $\mathcal{B}$ as follows: it first invokes $\mathcal{A}$, then extracts (K_1, N_1, AD_1) and (K_2, N_2, AD_2) from $\mathcal{A}$'s output. At the end, it outputs the tuple pair $((K_1, N_1, AD_1, M_1), (K_2, N_2, AD_2, M_2))$. $\square$

B.20 Proof of Theorem 20 (CDY\$ Security of chaSIV)

Proof. We define two games $\mathbf{G}_0$ and $\mathbf{G}_1$ as shown in Figure 28.

Game $\mathbf{G}_0$ is the same as $\text{CDY}_{\text{chaSIV}, S_s}^{\mathcal{A}}$ (see Figure 6) equipped with a random oracle $\mathbf{H}$. Therefore, we have $\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] = \text{Adv}_{\text{chaSIV}, S_s}^{\text{cdy}}(\mathcal{A}) = \text{Adv}_{\text{chaSIV}}^{\text{cdy}\$}(\mathcal{A})$.

Game $\mathbf{G}_1$ is the same as game $\mathbf{G}_0$, except that the committing tag T , calculated by the challenger, is now independent of queries to the random oracle $\mathbf{H}$. We claim that $\mathbf{G}_0$ and $\mathbf{G}_1$ behave identically unless the **bad** event happens, where the random oracle $\mathbf{H}$ is ever queried with an input that is prefixed with the key K sampled by the challenger. This is because, if **bad** does not occur, then T in $\mathbf{G}_0$ is also independent of random oracle queries made by $\mathcal{A}$ and by the challenger (to compute T'). Therefore, by the Difference Lemma of the game-based proof technique [Sho04], we have

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\text{bad}]. \quad (68)$$

Next, similar to the proof of Theorem 5, we bound $\Pr[\text{bad}]$ by constructing a game $\mathbf{G}_*$ also shown in Figure 28. $\mathbf{G}_*$ is defined such that $\mathcal{A}$ wins in $\mathbf{G}_*$ if and only if $\mathcal{A}$ triggers **bad** in game $\mathbf{G}_0/\mathbf{G}_1$ when (1)

Game $\mathbf{G}_0, \mathbf{G}_1$	Oracle $H(X)$
$T_H[\cdot] \leftarrow \perp$; $K \xleftarrow{\$} \{0, 1\}^\kappa$; $N \xleftarrow{\$} \mathcal{N}$; $AD \xleftarrow{\$} \mathcal{AD}$; $M \xleftarrow{\$} \mathcal{M}$ $K_e \leftarrow F(K, N)$; $T \xleftarrow{\$} \{0, 1\}^t$ $T_H[(K, N, AD, M)] \leftarrow T$ $N_e \leftarrow T _r$; $C \leftarrow \text{E.Enc}(K_e, N_e, M)$ $(K', N', AD', M') \xleftarrow{\$} \mathcal{A}^H(C, T)$ $K'_e \leftarrow F(K', N')$; $T' \leftarrow H(K', N', AD', M')$ $N_e \leftarrow T _r$; $C' \leftarrow \text{E.Enc}(K'_e, N'_e, M')$ return $(C', T') = (C, T)$	if $X = (K, *)$: bad $\leftarrow \text{true}$ if $T_H[X] = \perp$: $T_H[X] \xleftarrow{\$} \{0, 1\}^t$ return $T_H[X]$
Game $\mathbf{G}_*$	Oracle $H(X)$
$T_H[\cdot] \leftarrow \perp$; $K \xleftarrow{\$} \{0, 1\}^\kappa$; $N \xleftarrow{\$} \mathcal{N}$; $AD \xleftarrow{\$} \mathcal{AD}$; $M \xleftarrow{\$} \mathcal{M}$ $K_e \leftarrow F(K, N)$; $T \xleftarrow{\$} \{0, 1\}^t$ $N_e \leftarrow T _r$; $C \leftarrow \text{E.Enc}(K_e, N_e, M)$ $\mathcal{Q}^* \leftarrow \emptyset$ $(K', N', AD', M') \xleftarrow{\$} \mathcal{A}^H(C, T)$ $H(K', N', AD', M')$ return $K \in \mathcal{Q}^*$	$(K^*, *) \leftarrow X$ $\mathcal{Q}^* \leftarrow^{\cup} \{K^*\}$ if $T_H[X] = \perp$: $T_H[X] \xleftarrow{\$} \{0, 1\}^t$ return $T_H[X]$

Fig. 28: Games $\mathbf{G}_0, \mathbf{G}_1$ and $\mathbf{G}_*$ in the proof of Theorem 20, where $\mathbf{G}_0$ contains boxed content while $\mathbf{G}_1$ does not.

$\mathcal{A}$ queried the random oracle H with an input prefixed with K or (2) $\mathcal{A}$ output (K', N', AD', M') with $K' = K$ and then $H(K', N', AD', C')$ was queried by the challenger. Therefore, we have

$$\Pr[\text{bad}] = \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]. \quad (69)$$

Then, we bound $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1]$ by constructing an adversary $\mathcal{B}$ against the PRF security of the underlying F . $\mathcal{B}$ starts by initializing the set $\mathcal{Q}^*$ and table T_H (to an empty set and empty table, respectively), and then it runs $\mathcal{A}$. Then it samples the tuple (N, AD, M) uniformly at random, and calls its own PRF oracle with input N to derive K_e . It continues by sampling T , setting $N_e \leftarrow T|_r$, calculating $C \leftarrow \text{E.Enc}(K_e, N_e, M)$ and running $\mathcal{A}$ with (C, T) . For $\mathcal{A}$'s each random oracle query $X = (K^*, *)$, $\mathcal{B}$ records K^* in $\mathcal{Q}^*$ and answers the query via lazy sampling. After $\mathcal{A}$ terminates and outputs (K', N', AD', M') , $\mathcal{B}$ “queries” the random oracle H with the input (K', N', AD', M') and continues as follows. For each key $K_i \in \mathcal{Q}^*$, $\mathcal{B}$ computes $K_{e,i} \leftarrow F(K_i, N)$ and checks if $K_{e,i} = K_e$. If this holds for any of the keys in $\mathcal{Q}^*$, $\mathcal{B}$ guesses that it was interacting with the real PRF and outputs 1; otherwise, it outputs 0, guessing that it was interacting with a truly random function.

If $\mathcal{B}$ had access to real PRF, then it simulates $\mathcal{A}$'s view in $\mathbf{G}_*$ perfectly and hence we have $\Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\mathcal{B}^{\text{F}^\kappa} \Rightarrow 1]$. Here we have an inequality to capture “false positives” in the above $K_{e,i} = K_e$ check. On the other hand, if $\mathcal{B}$ was in the ideal world, the probability that a random output K_e equals $F(K_i, N)$ for some $K_i \in \mathcal{Q}^*$ is at most $|\mathcal{Q}^*|/2^\kappa$. Since $|\mathcal{Q}^*| \leq q_h + 1$, we have $\Pr[\mathcal{B}^{\text{f}} \Rightarrow 1] \leq (q_h + 1)/2^\kappa$. Then, recalling that $\text{Adv}_F^{\text{prf}}(\mathcal{B}) = \Pr[\mathcal{B}^{\text{F}^\kappa} \Rightarrow 1] - \Pr[\mathcal{B}^{\text{f}} \Rightarrow 1]$ we have

$$\text{Adv}_F^{\text{prf}}(\mathcal{B}) \geq \Pr[\mathbf{G}_*^{\mathcal{A}} \Rightarrow 1] - \frac{q_h + 1}{2^\kappa}, \quad (70)$$

and it follows from equations (68)~(70) that

$$\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \text{Adv}_F^{\text{prf}}(\mathcal{B}) + \frac{q_h + 1}{2^\kappa}. \quad (71)$$

Finally, we observe that in game $\mathbf{G}_1$ the committing tag T in the challenge ciphertext (C, T) is independent of random oracle queries and hence independent of T' , so in order for $\mathcal{A}$ to win one of the random oracle queries must output a committing tag equal to the independent random T . This probability is bounded by $(q_h + 1)/2^t$. Therefore, we have

$$\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] \leq \frac{q_h + 1}{2^t}. \quad (72)$$

The proof is concluded by combining equations (71) and (72).

□